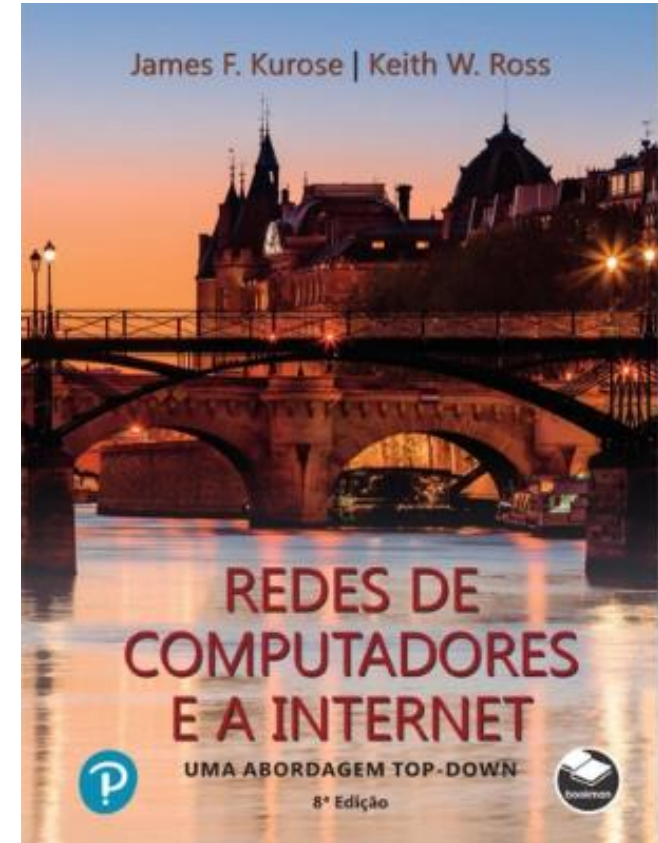


Capítulo 2

Camada de Aplicação



*Redes de Computadores e
a Internet: Uma
abordagem Top-Down*

8ª edição

Jim Kurose, Keith Ross

Pearson & Bookman, 2021

All material copyright 1996-2020
J.F Kurose and K.W. Ross, All Rights Reserved

Camada de Aplicação: roteiro

- **Princípios de aplicações de rede**
- A Web e o HTTP
- E-mail, SMTP, IMAP
- DNS: o serviço de diretório da Internet
- Aplicações P2P
- fluxos de vídeo e redes de distribuição de conteúdos
- programação de sockets com UDP e TCP



Camada de Aplicação: visão geral

Nossos objetivos:

- aspectos conceituais e de implementação de protocolos da camada de aplicação
 - modelos de serviço da camada de transporte
 - paradigma cliente servidor
 - paradigma *peer-to-peer*
- aprender sobre protocolos através do estudo de protocolos populares da camada de aplicação
 - HTTP
 - SMTP, IMAP
 - DNS
 - sistemas de *streaming* de vídeo, CDNs
- programação de aplicações de rede
 - API de sockets

Algumas aplicações de rede

- redes sociais
- Web
- mensagens de texto
- e-mail
- jogos em redes multiusuários
- *streaming* de vídeos armazenados (YouTube, Hulu, Netflix)
- compartilhamento de arquivos P2P
- voz sobre IP (ex., Skype)
- videoconferência em tempo real
- busca na Internet
- login remoto
- ...

Q: *quais são as suas favoritas?*

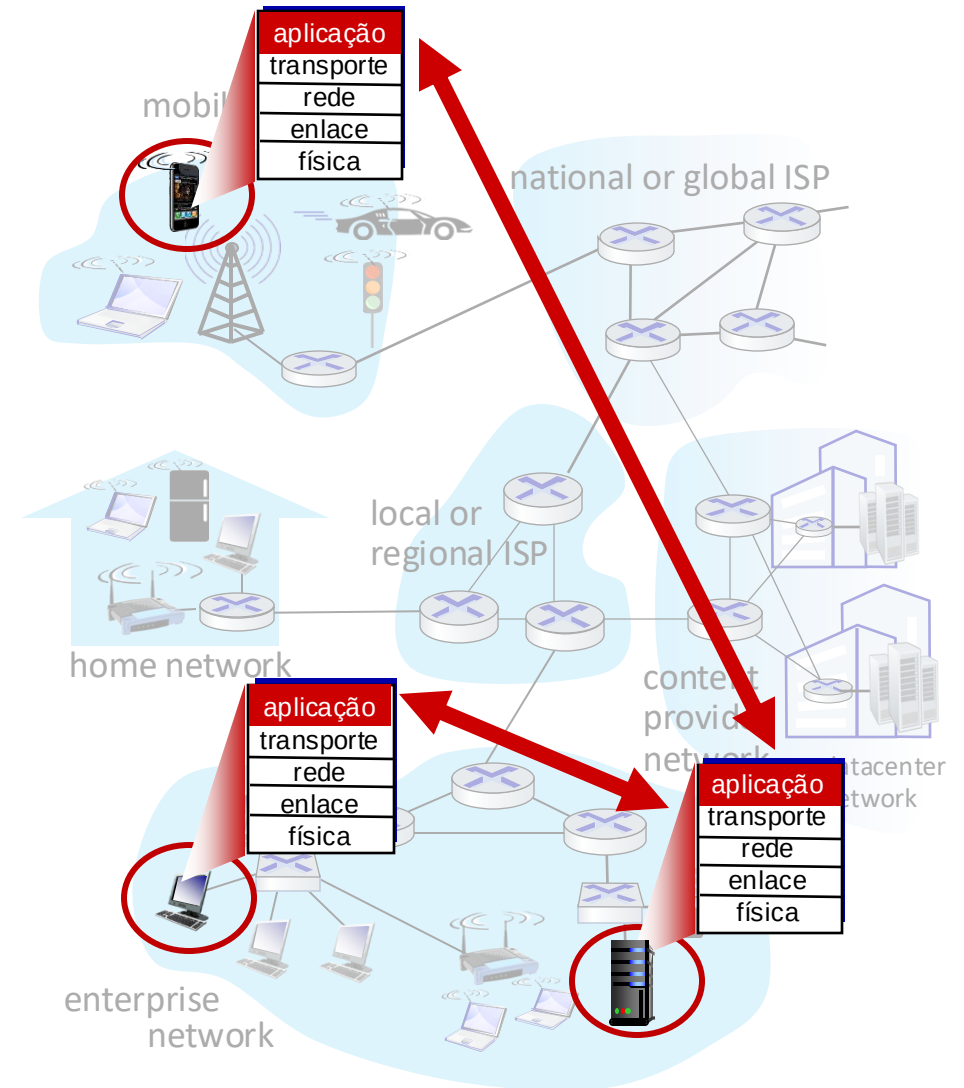
Criando uma aplicação de rede

escreva programas que:

- rodam em (diferentes) sistemas finais
- comunicam-se através da rede
- ex., software do servidor web se comunica com o software do navegador

não há necessidade de escrever software para os dispositivos do núcleo da rede

- dispositivos do núcleo da rede não executam aplicações dos usuários
- aplicações nos sistemas finais permitem rápido desenvolvimento e disseminação



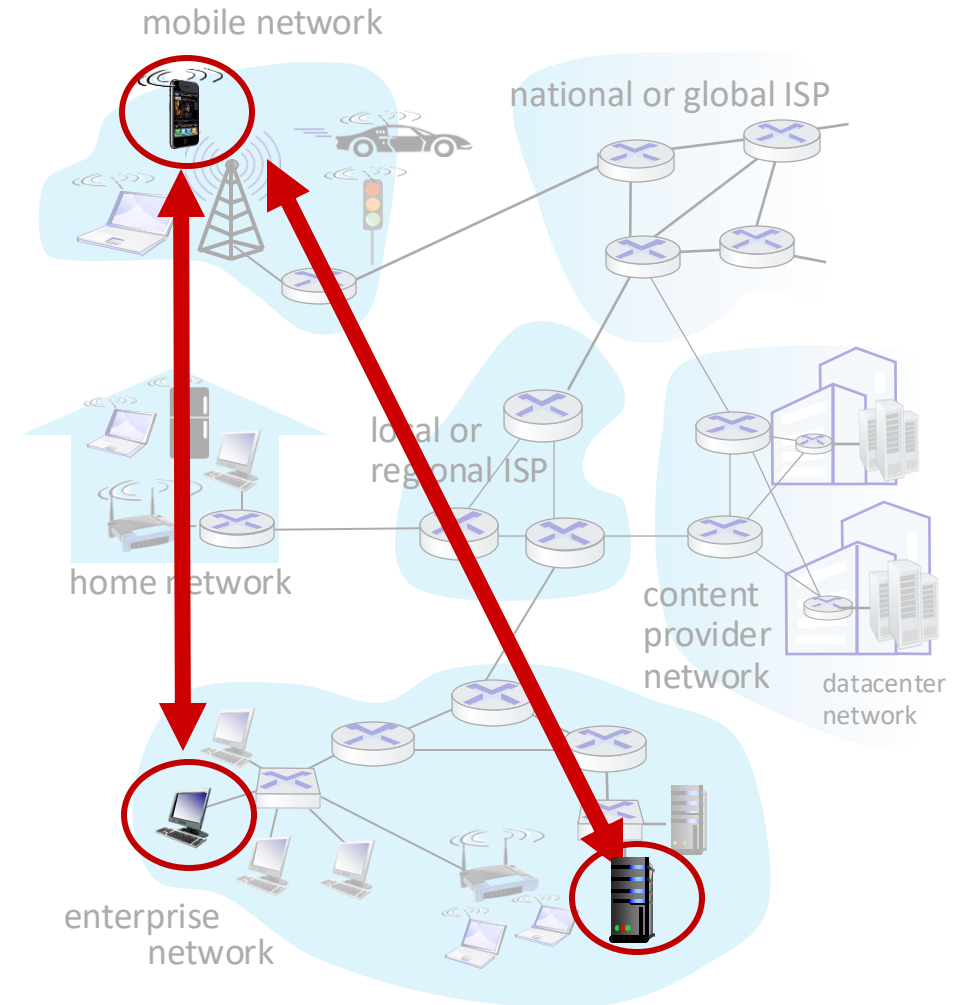
Paradigma cliente-servidor

servidor:

- sempre ligado
- endereço IP permanente
- frequentemente em *data centers*, visando escalabilidade

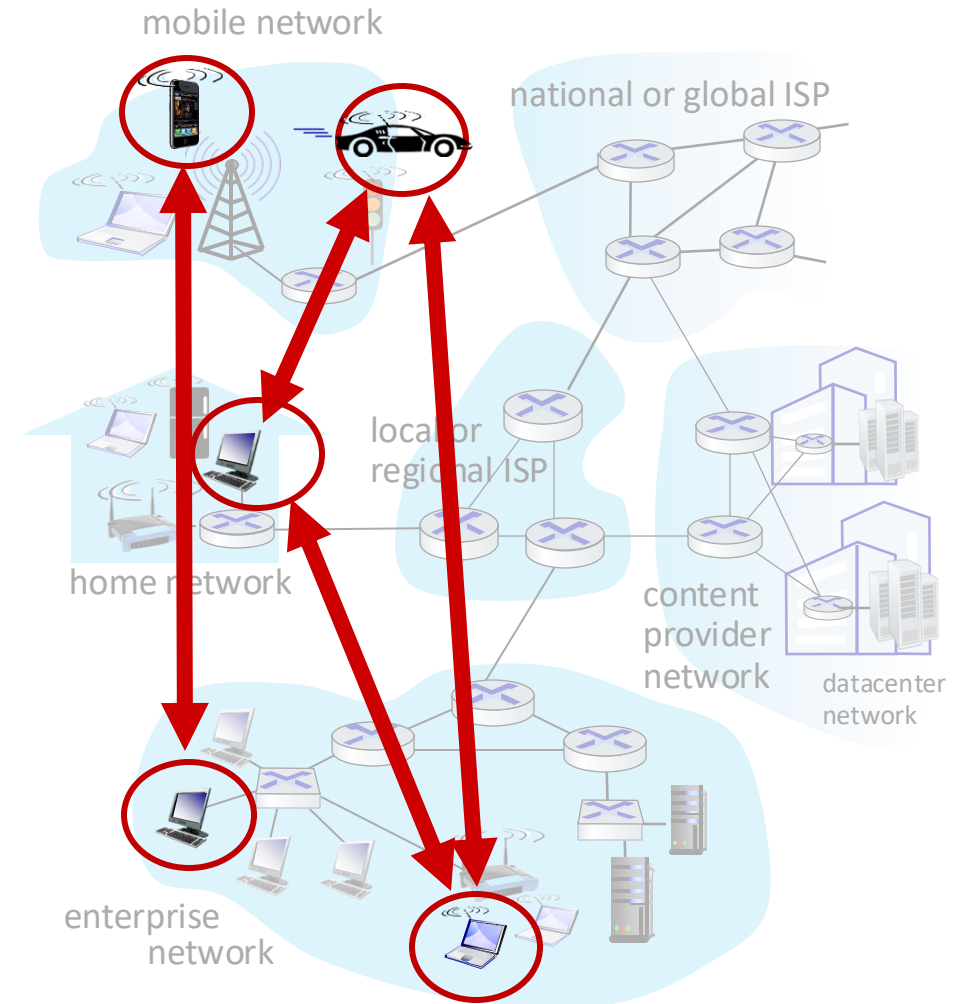
clientes:

- contactam, se comunicam com o servidor
- podem estar conectados intermitentemente
- podem ter endereços IP dinâmicos
- *não* se comunicam diretamente com outros clientes
- exemplos: HTTP, IMAP, FTP



Paradigma *peer-peer* (p2p)

- *não* há servidor sempre conectado
- sistemas finais arbitrários se comunicam diretamente
- pares solicitam serviços de outros pares e em troca proveem serviços para outros parceiros:
 - *autoescalabilidade* – novos pares trazem nova capacidade de serviço assim como novas demandas por serviços
- pares estão conectados intermitentemente e mudam endereços IP
 - gerenciamento complexo
- exemplo: compartilhamento de arquivos P2P



Comunicação entre processos

- processo*: programa que executa num sistema final
- processos no mesmo sistema final se comunicam usando **comunicação entre processos** (definida pelo sistema operacional)
 - processos em sistemas finais distintos se comunicam trocando **mensagens**

clientes, servidores

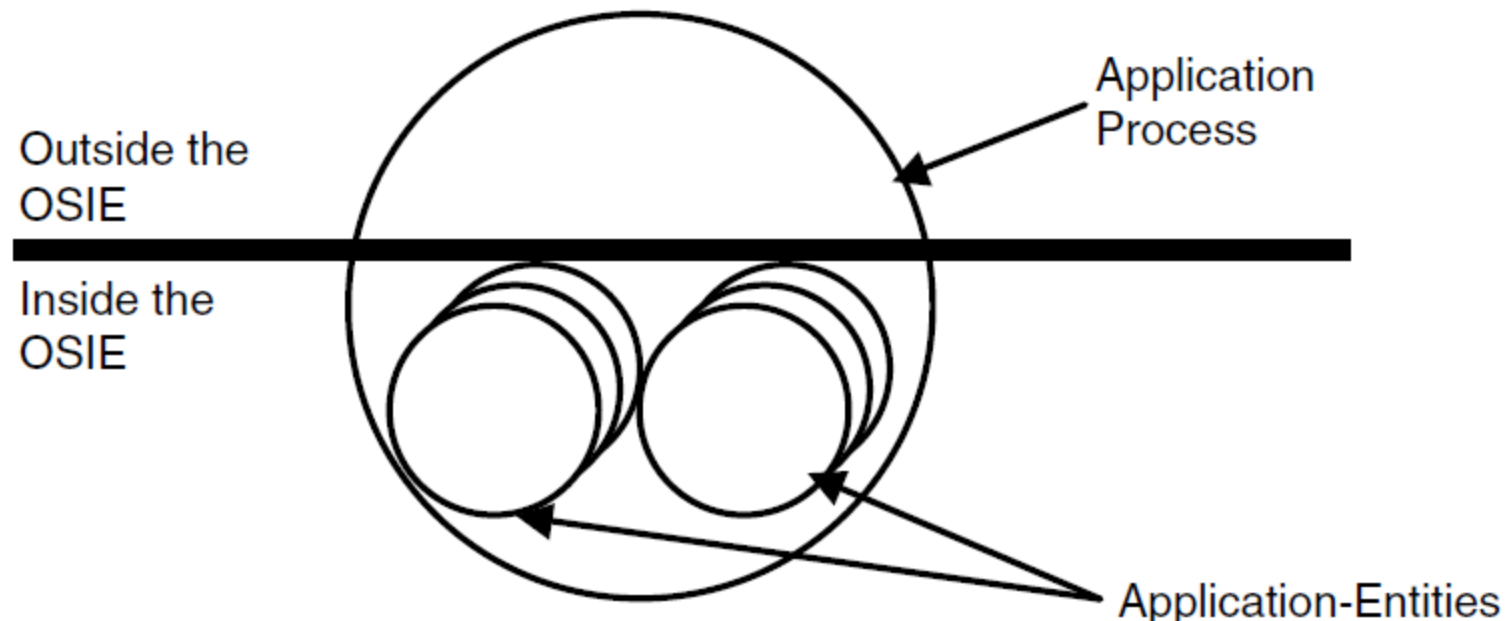
processo cliente: processo que inicia a comunicação

processo servidor: processo que espera ser contactado

- nota: aplicações com arquiteturas P2P possuem processos clientes e processos servidores

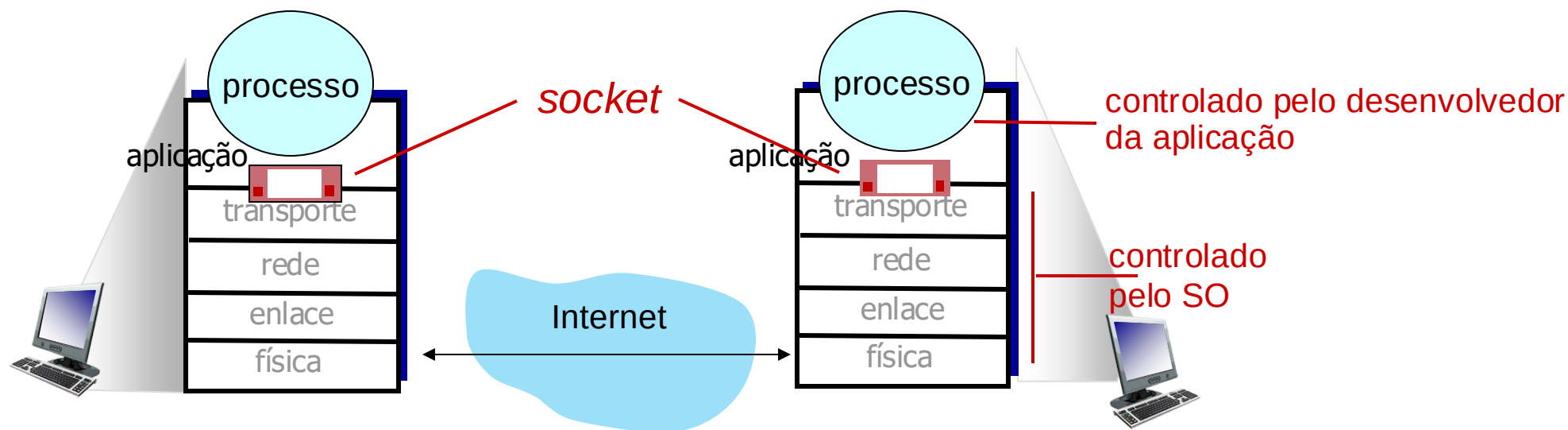
Relacionamento entre as Aplicações e a Camada de Aplicação

- Grande contribuição do OSI para as camadas superiores.
- Protocolos de aplicação (*application entities*) fazem parte do processo da aplicação, mas há uma parte que está fora do ambiente OSI (OSIE – OSI *Environment*)



Sockets

- os processos enviam/ recebem mensagens para/dos seus **sockets**
- um socket é análogo a uma porta
 - processo transmissor envia a mensagem através da porta
 - o processo transmissor assume existência de uma infra de transporte no outro lado da porta que faz com que a mensagem chegue ao socket do processo receptor
 - envolve dois sockets: um de cada lado



Endereçamento de processos

- para receber mensagens, processo precisa *identificador*
- dispositivo possui um endereço IP único de 32-bits
- Q: o endereço IP do hospedeiro no qual o processo está sendo executado é suficiente para identificar o processo?
 - R: não, *muitos* processos podem estar em execução no mesmo hospedeiro
- *identificador* inclui o *endereço IP* e *números de portas* associadas com o processo no hospedeiro.
- exemplo de números de portas:
 - servidor HTTP: 80
 - servidor de e-mail: 25
- para enviar mensagem HTTP para o servidor web gaia.cs.umass.edu:
 - *endereço IP*: 128.119.245.12
 - *número de porta*: 80
- mais sobre isso em breve...

Um protocolo da camada de aplicação define:

- tipos de mensagens trocadas,
 - ex., requisições, respostas
- sintaxe das mensagens:
 - campos presentes nas mensagens e como são identificados
- semântica das mensagens
 - significado da informação nos campos
- regras para quando e como os processos enviam e recebem as mensagens

protocolos abertos:

- definidos em RFCs, todos têm acesso à definição dos protocolos
- permitem a interoperabilidade
- ex., HTTP, SMTP

protocolos proprietários:

- ex., Skype

Qual o serviço de transporte que uma aplicação necessita?

integridade dos dados

- algumas apls (ex., transf. de arquivos, transações web) requerem uma transferência 100% confiável
- outras (ex. áudio) podem tolerar algumas perdas

retardos

- algumas apls (ex., telefonia Internet, jogos interativos) requerem baixos retardos para serem “viáveis”

vazão (*throughput*)

- algumas apls (ex., multimídia) requerem quantia mínima de vazão para serem “viáveis”
- outras apls (“apls elásticas”) conseguem usar qq quantia de banda disponível

segurança

- criptografia, integridade dos dados, ...

Requisitos de serviços de transporte por apps

aplicação	perda de dados	vazão	sensibilidade a atrasos?
transf. arquivos/ <i>download</i>	sem perdas	elástica	não
e-mail	sem perdas	elástica	não
documentos Web	sem perdas	elástica	não
áudio/video em tempo real	tolerante	áudio: 5kbps-1Mbps vídeo:10kbps-5Mbps	sim, 10's mseg
<i>streaming</i> áudio/vídeo	tolerante	igual aos acima	sim, alguns segs
jogos interativos	tolerante	kbps+	sim, 10's mseg
mensagens de texto	sem perdas	elástica	sim e não

Serviços dos protocolos de transporte da Internet

serviço TCP:

- *transporte confiável* entre processos remetente e receptor
- *controle de fluxo*: remetente não vai “afogar” receptor
- *controle de congestionamento*: estrangula remetente quando a rede estiver carregada
- *orientado a conexão*: apresentação requerida entre cliente e servidor
- *não provê*: garantias temporais ou de banda mínima, segurança

serviço UDP:

- *transferência de dados não confiável* entre processos remetente e receptor
- *não provê*: confiabilidade, controle de fluxo, controle de congestionamento, garantias temporais, garantia de vazão, segurança, nem estabelecimento da conexão

Q: qual o interesse pelo UDP?

Serviços dos protocolos de transporte da Internet

aplicação	Protocolo da camada de aplicação	protocolo de transporte
transf. arquivo/ <i>download</i>	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
documentos Web	HTTP 1.1 [RFC 7320]	TCP
telefonia Internet	SIP [RFC 3261], RTP [RFC 3550], ou proprietário	TCP ou UDP
<i>streaming</i> áudio/vídeo	HTTP [RFC 7320], DASH	TCP
jogos interativos	WOW, FPS (proprietário)	UDP ou TCP

Tornando o TCP seguro

Sockets TCP & UDP básicos:

- sem criptografia
- senhas em texto aberto enviadas aos sockets atravessam a Internet em texto aberto (!)

Segurança da camada de transporte (TLS – Transport Layer Security)

- provê conexões TCP criptografadas
- integridade dos dados
- autenticação dos pontos terminais

TLS implementado na camada de aplicação

- apps usam bibliotecas TLS, que por sua vez, usam TCP
- textos abertos enviados para o *socket* atravessam a Internet *criptografadas*
- vide Capítulo 8

Há também o **DTLS**, baseado no TLS que torna seguras as comunicações com o UDP.

Camada de Aplicação: roteiro

- Princípios de aplicações de rede
- **A Web e o HTTP**
- E-mail, SMTP, IMAP
- DNS: o serviço de diretório da Internet
- Aplicações P2P
- fluxos de vídeo e redes de distribuição de conteúdos
- programação de sockets com UDP e TCP



A Web e o HTTP

Primeiro, uma rápida revisão...

- páginas web consistem de *objetos*, cada um dos quais pode estar armazenado em diferentes servidores Web
- um objeto pode ser um arquivo HTML, uma imagem JPEG, um applet Java, um arquivo de áudio,...
- páginas web consistem de um *arquivo base HTML* que inclui *diversos objetos referenciados, cada um* endereçável por uma *URL*, ex.,

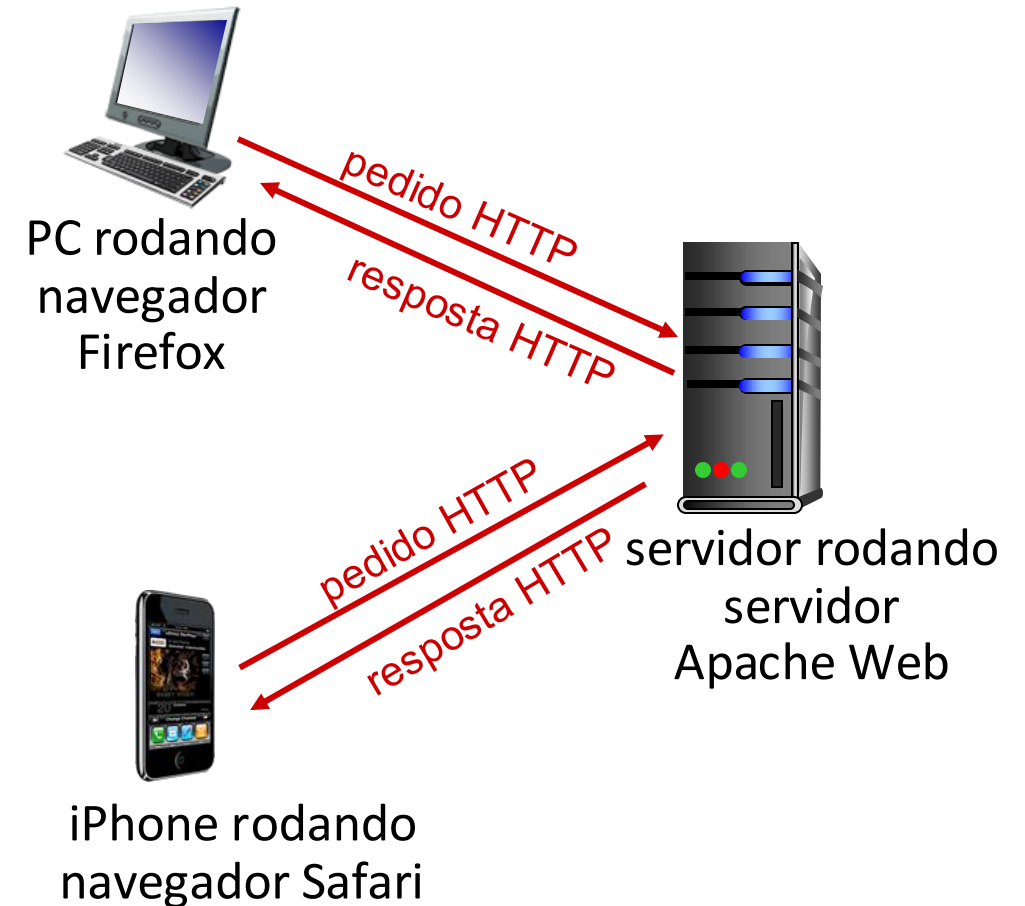
www.someschool.edu
nome do hospedeiro

/someDept/pic.gif
nome do caminho

Visão geral do HTTP

HTTP: *hypertext transfer protocol*

- protocolo da camada de aplicação da Web
- modelo cliente/servidor:
 - *cliente*: navegador que pede, recebe (usando o protocolo HTTP) e “apresenta” objetos Web
 - *servidor*: servidor Web envia (usando o protocolo HTTP) objetos em resposta aos pedidos



Visão geral do HTTP (continuação)

HTTP usa o TCP:

- cliente inicia conexão TCP (cria socket) ao servidor, porta 80
- servidor aceita conexão TCP do cliente
- mensagens HTTP (mensagens do protocolo da camada de apl) trocadas entre navegador (cliente HTTP) e servidor Web (servidor HTTP)
- encerra conexão TCP

HTTP é “sem estado”

- servidor *não* mantém informação sobre pedidos anteriores do cliente

nota

protocolos que mantêm “estado” são complexos!

- história passada (estado) tem que ser guardada
- Caso caia servidor/cliente, suas visões do “estado” podem ficar inconsistentes, devem ser reconciliadas

HTTP: dois tipos de uso de conexões TCP

HTTP não-persistente

1. abre conexão TCP
2. no máximo um objeto é enviado pela conexão TCP
3. encerra a conexão TCP

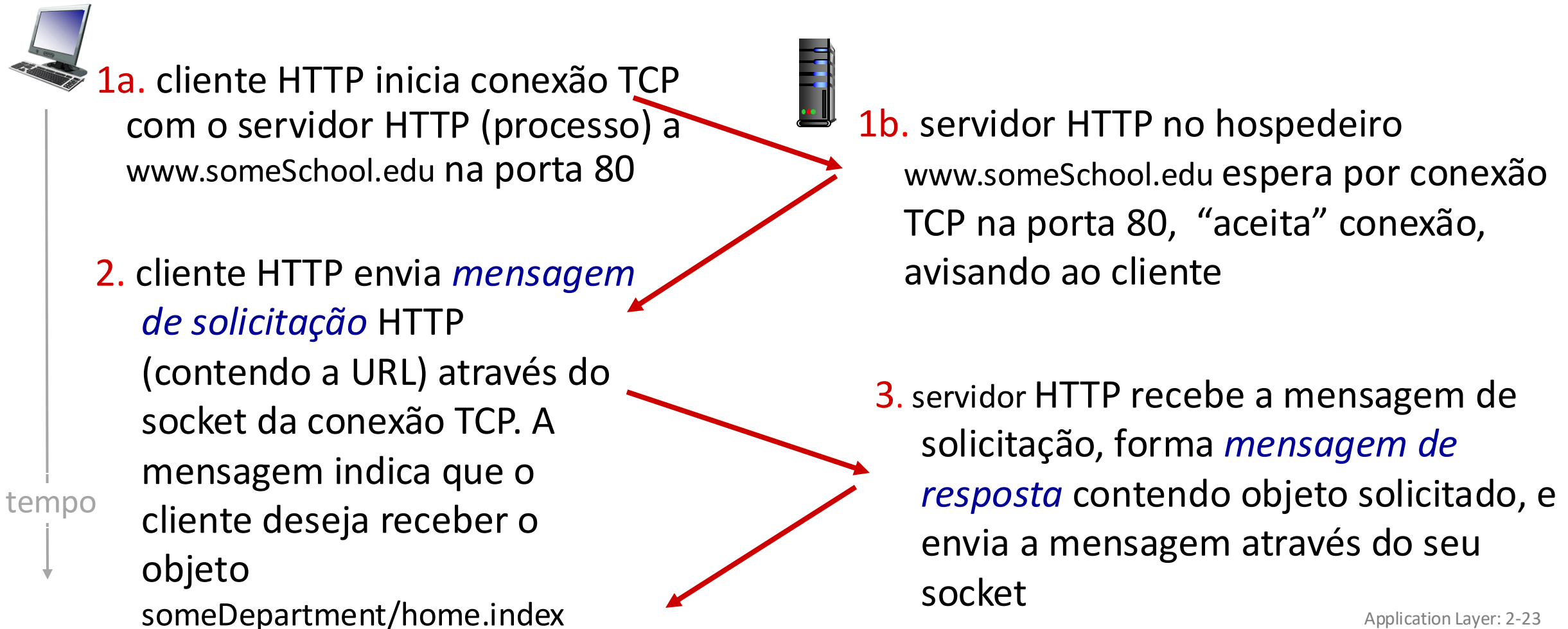
baixar múltiplos objetos
requer o uso de múltiplas
conexões

HTTP persistente

- abre conexão TCP com o servidor
- múltiplos objetos podem ser enviados sobre uma *única* conexão TCP entre cliente e servidor
- encerra a conexão TCP

HTTP não-persistente: exemplo

Usuário digita a URL: `www.someSchool.edu/someDepartment/home.index`
(contendo texto, referências a 10 imagens jpeg)



HTTP não-persistente: exemplo (cont.)

Usuário digita a URL: `www.someSchool.edu/someDepartment/home.index`
(contendo texto, referências a 10 imagens jpeg)



4. servidor HTTP encerra
conexão TCP.

5. cliente HTTP recebe mensagem de
resposta contendo arquivo html,
apresenta html. Analisando arquivo
html, encontra 10 objetos jpeg
referenciados

6. Passos 1 a 5 repetidos para
cada um dos 10 objetos jpeg

tempo
↓

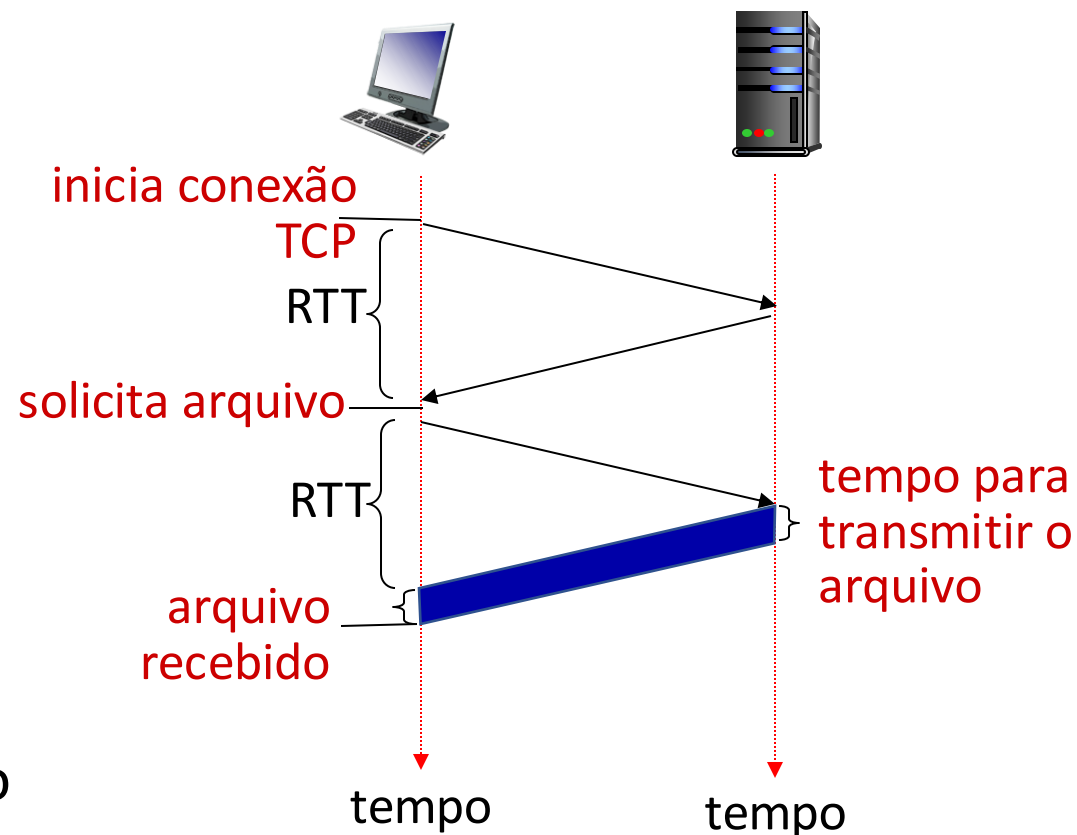
HTTP não-persistente: tempo de resposta

definição de RTT (*Round Trip Time*):

intervalo de tempo entre a ida e a volta de um pequeno pacote entre um cliente e um servidor

tempo de resposta HTTP (por objeto):

- um RTT para iniciar a conexão TCP
- um RTT para o pedido HTTP e o retorno dos primeiros bytes da resposta HTTP
- tempo de transmissão do objeto/arquivo



Tempo de resposta do HTTP não-persistente = $2RTT +$ tempo de transmissão do arquivo

HTTP Persistente (HTTP 1.1)

Questões com o HTTP não-persistente:

- requer 2 RTTs para cada objeto
- SO aloca recursos do hospedeiro (*overhead*) para cada conexão TCP
- os navegadores frequentemente abrem conexões TCP paralelas para recuperar os objetos referenciados

HTTP Persistente (HTTP1.1):

- o servidor deixa a conexão aberta após enviar a resposta
- mensagens HTTP seguintes entre o mesmo cliente/servidor são enviadas nesta conexão aberta
- o cliente envia os pedidos logo que encontra um objeto referenciado
- pode ser necessário apenas um RTT para todos os objetos referenciados (cortando o tempo de resposta pela metade)

Mensagem de requisição HTTP

- dois tipos de mensagem HTTP: *requisição, resposta*
- **mensagem de requisição HTTP:**
 - ASCII (formato legível por pessoas)

linha de requisição
(comandos GET, POST,
HEAD)

linhas de
cabeçalho

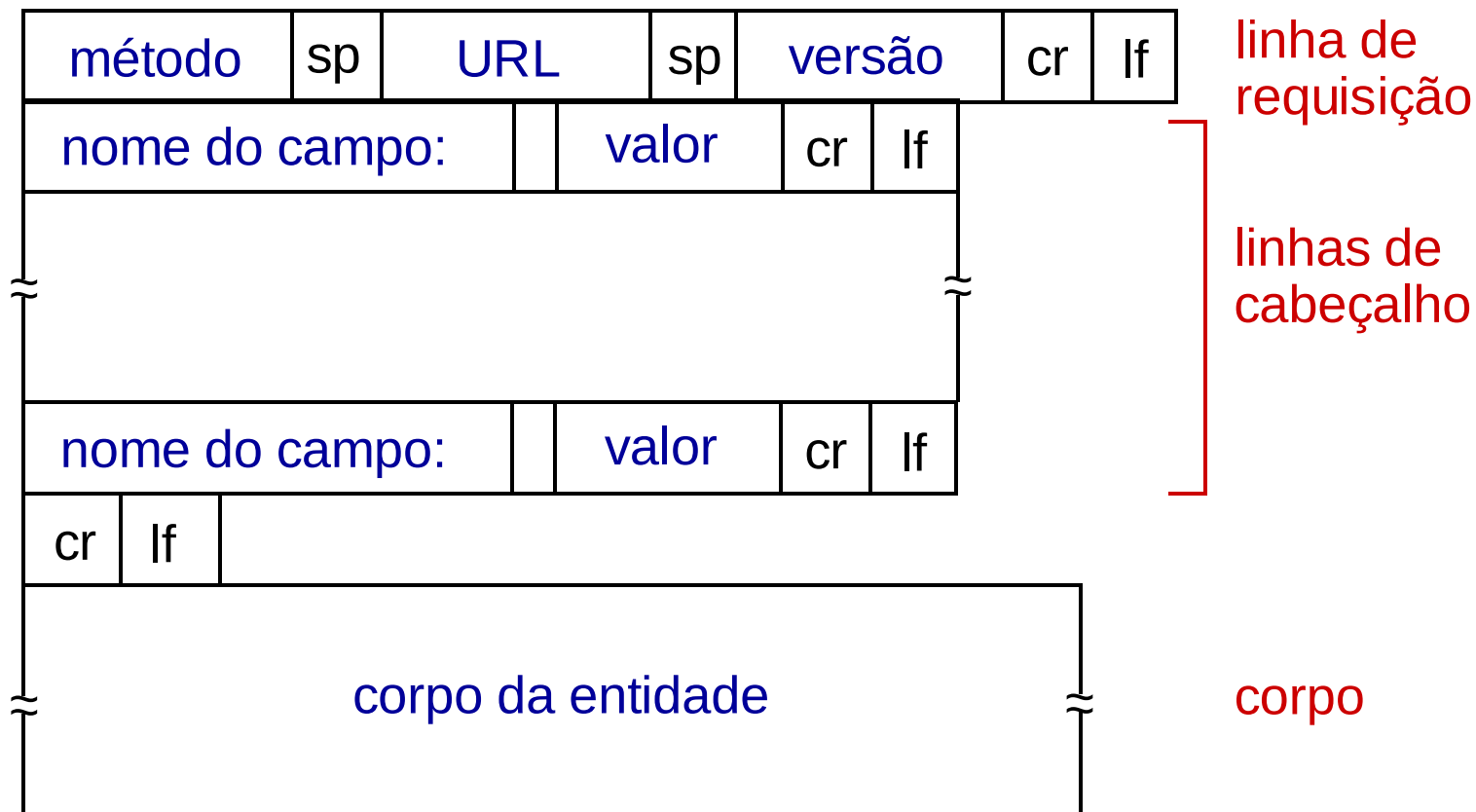
carriage return, line feed
no início da linha indica
o final das linhas de
cabeçalho

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

caractere de *carriage return*
caractere de *line-feed*

* Confira os exercícios interativos on-line para mais
exemplos: http://gaia.cs.umass.edu/kurose_ross/interactive/

Mensagem de requisição HTTP: formato geral



Outras mensagens de requisição HTTP

método POST:

- páginas Web frequentemente contêm formulário de entrada
- Conteúdo é enviado para o servidor no corpo da mensagem POST

método GET (para envio de dados ao servidor):

- inclui dados do usuário no campo URL da mensagem de requisição GET (após a '?'):

`www.somesite.com/animalsearch?monkeys&banana`

método HEAD:

- solicita (apenas) cabeçalhos que seriam retornados se a URL especificada fosse solicitada por um método GET.

método PUT:

- carrega um novo arquivo (objeto) no servidor
- substitui completamente o arquivo existente numa URL especificada com o conteúdo no corpo da mensagem de solicitação

Mensagem de resposta HTTP

linha de status (protocolo
código de status frase de
status)

linhas de
cabeçalho

dados, ex., arquivo HTML
solicitado

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02
      GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-
      1\r\n
\r\n
dados dados dados dados dados ...
```

* Confira os exercícios interativos online para mais exemplos: http://gaia.cs.umass.edu/kurose_ross/interactive/

Códigos de status das respostas HTTP

- código de status aparece na 1a linha da mensagem de resposta do servidor para o cliente.
- alguns códigos típicos:

200 OK

- sucesso, objeto pedido segue mais adiante nesta mensagem

301 Moved Permanently

- objeto pedido mudou de lugar, nova localização especificado mais adiante nesta mensagem (no campo Location:)

400 Bad Request

- mensagem de pedido não entendida pelo servidor

404 Not Found

- documento pedido não se encontra neste servidor

505 HTTP Version Not Supported

- versão de HTTP do pedido não usada por este servidor

Experimente o HTTP (lado do cliente)

1. Faça um netcat para o seu servidor Web favorito:

- ```
%nc -c -v gaia.cs.umass.edu 80
```
- abre conexão TCP para a porta 80 (porta padrão para servidores HTTP) em gaia.cs.umass.edu.
  - qualquer coisa digitada é enviada para a porta 80 do servidor gaia.cs.umass.edu

2. digite uma solicitação GET HTTP:

```
GET /kurose_ross/interactive/index.php HTTP/1.1
Host: gaia.cs.umass.edu
```

- digitando isso (deve teclar ENTER duas vezes), está enviando este pedido GET mínimo (porém completo) ao servidor HTTP

3. examine a mensagem de resposta enviada pelo servidor HTTP!

(ou use Wireshark para ver as msgs de pedido/resposta HTTP capturadas)



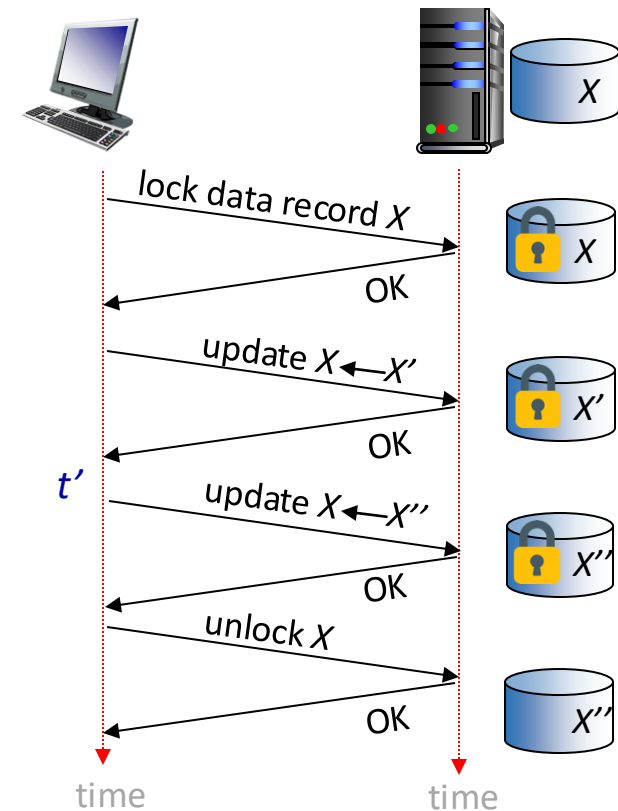
# Mantendo estado usuário/servidor: cookies

Relembre: a interação

GET/resposta é *sem estado*

- não há noção de trocas de mensagens HTTP em diversos passos para completar uma “transação” Web
  - não há necessidade para cliente/servidor rastrear o “estado” numa troca em múltiplos passos.
  - todas as solicitações HTTP são independentes uma das outras
  - não há necessidade do cliente/servidor “recuperar” uma transação apenas parcialmente completada

um protocolo com estado: cliente faz duas mudanças em X, ou nenhuma



**Q:** o que ocorre se a conexão de rede do cliente cair no instante  $t'$  ?

# Mantendo estado usuário/servidor: cookies

sítios Web e navegadores dos clientes usam *cookies* para manter algum estado entre as transações

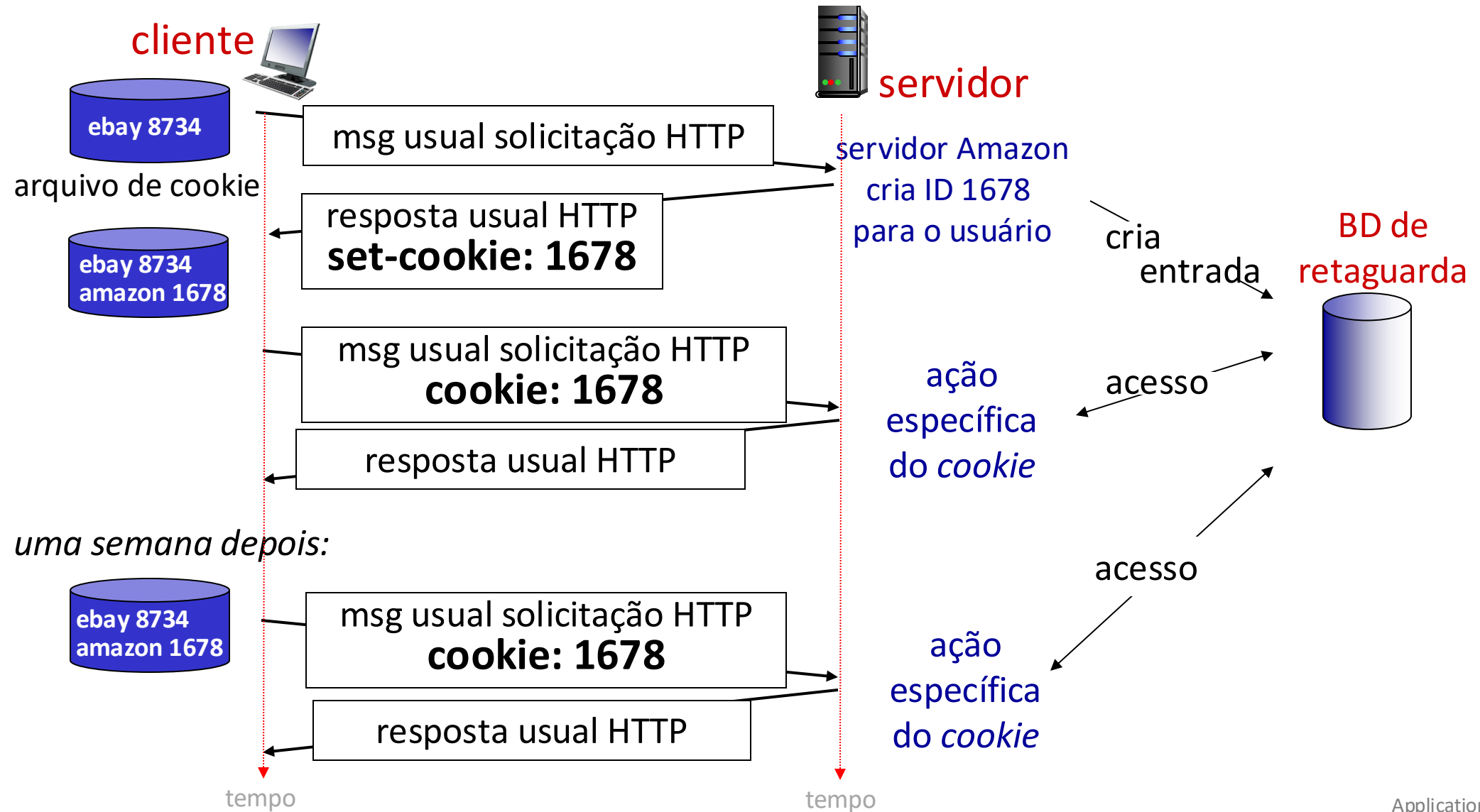
## *quatro componentes:*

- 1) linha de cabeçalho do cookie na mensagem de resposta HTTP
- 2) linha de cabeçalho do cookie numa próxima mensagem de pedido HTTP
- 3) arquivo do cookie mantido no host do usuário e gerenciado pelo navegador do usuário
- 4) BD de retaguarda no sítio Web

## Exemplo:

- Suzana usa navegador num laptop, visita um sítio específico de comércio eletrônico pela primeira vez
- quando as solicitações iniciais HTTP chegam no sítio, o sítio cria:
  - uma ID única (“cookie”)
  - uma entrada para a ID no BD de retaguarda
- solicitações HTTP seguintes de Suzana para este sítio conterá o valor do ID do cookie, permitindo ao sítio “identificar” Suzana.

# Mantendo estado usuário/servidor: cookies



# Cookies HTTP: comentários

## *Para que os cookies podem ser usados:*

- autorização
- carrinhos de compra
- recomendações
- estado da sessão do usuário (Webmail)

## *Desafio: Como manter o estado:*

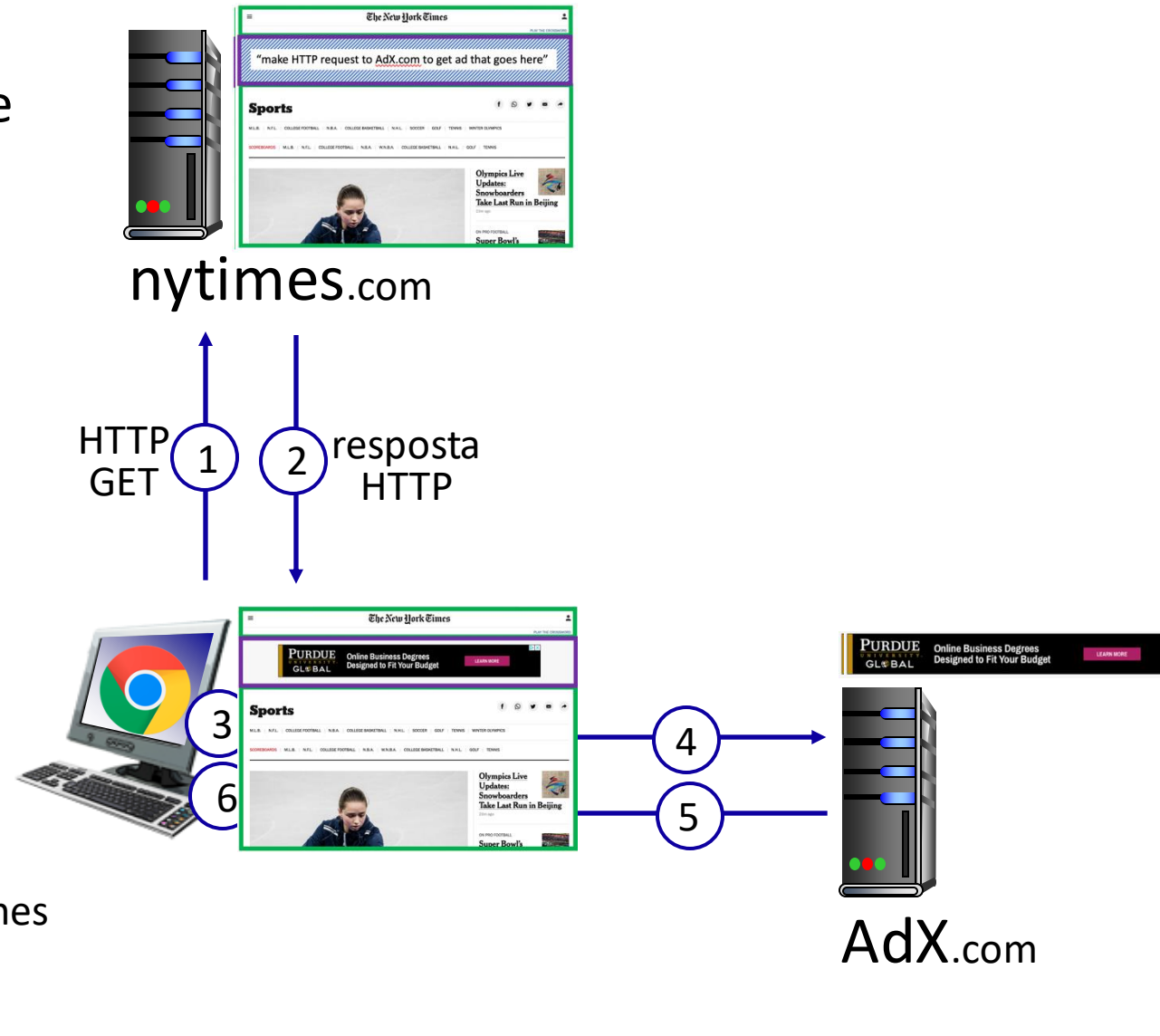
- Pontos finais do protocolo: mantêm o estado no transmissor/receptor para múltiplas transações
- *Cookies*: mensagens HTTP transportam o estado

## *cookies e privacidade:* nota

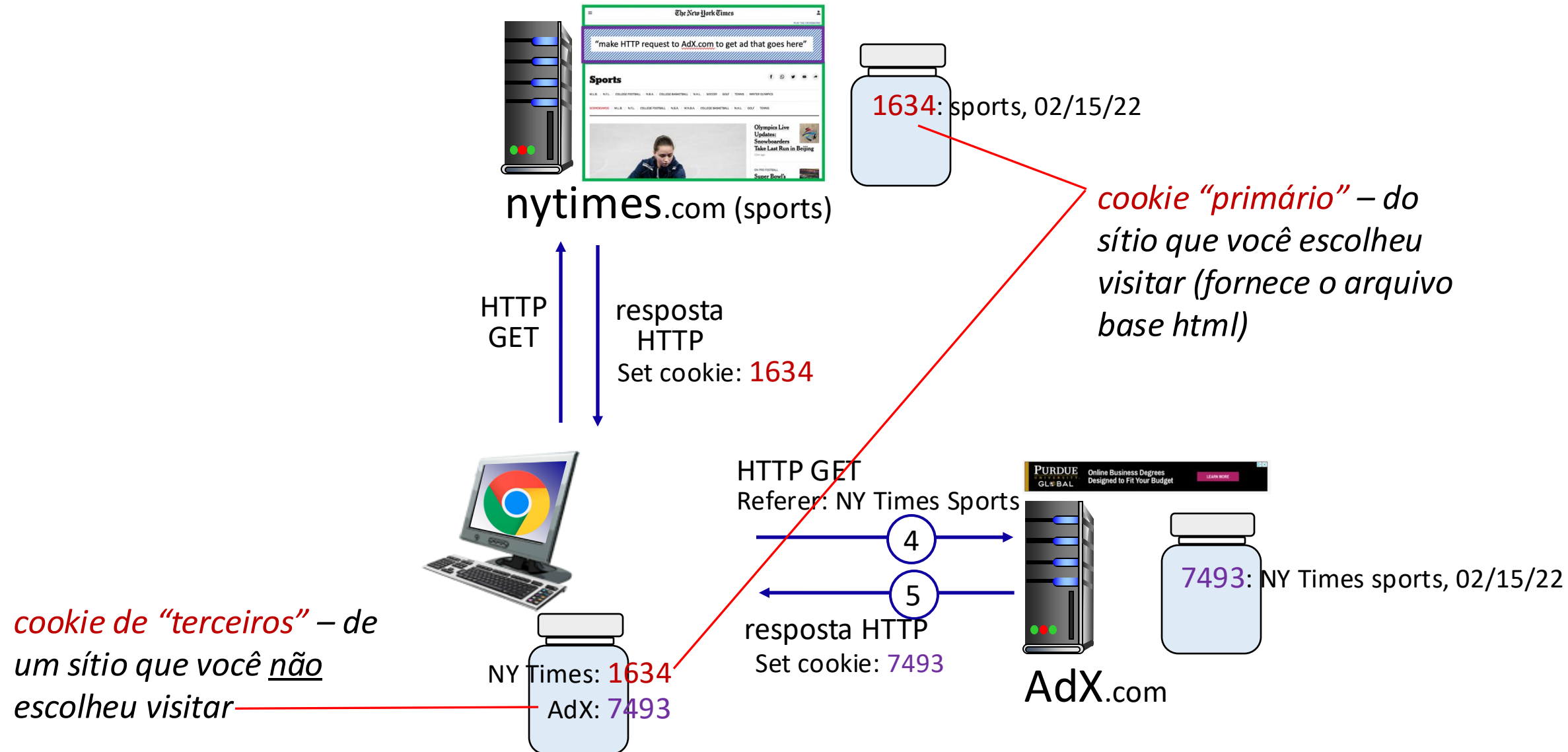
- *cookies* permitem que os sítios aprendam muito sobre você.
- *cookies* persistentes de terceiros (*cookies* de rastreamento) permitem que uma identidade comum (valor do *cookie*) seja rastreado entre diferentes sítios web

# Exemplo: apresentando a página do NY Times

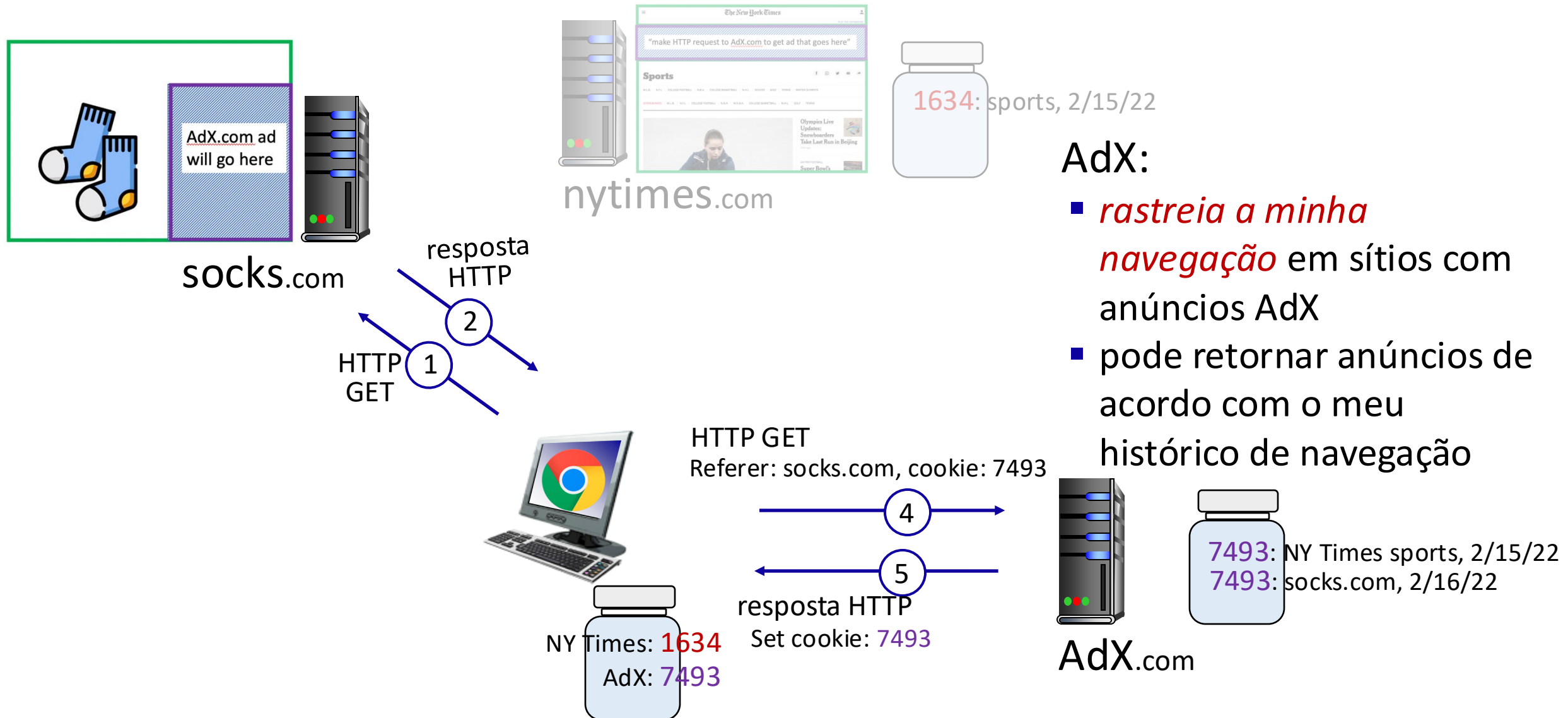
- 1 obtém o arquivo base html de nytimes.com
- 2
- 4 recupera anúncio de AdX.com
- 5
- 7 apresenta a página composta



# Cookies: rastreando o comportamento de um usuário

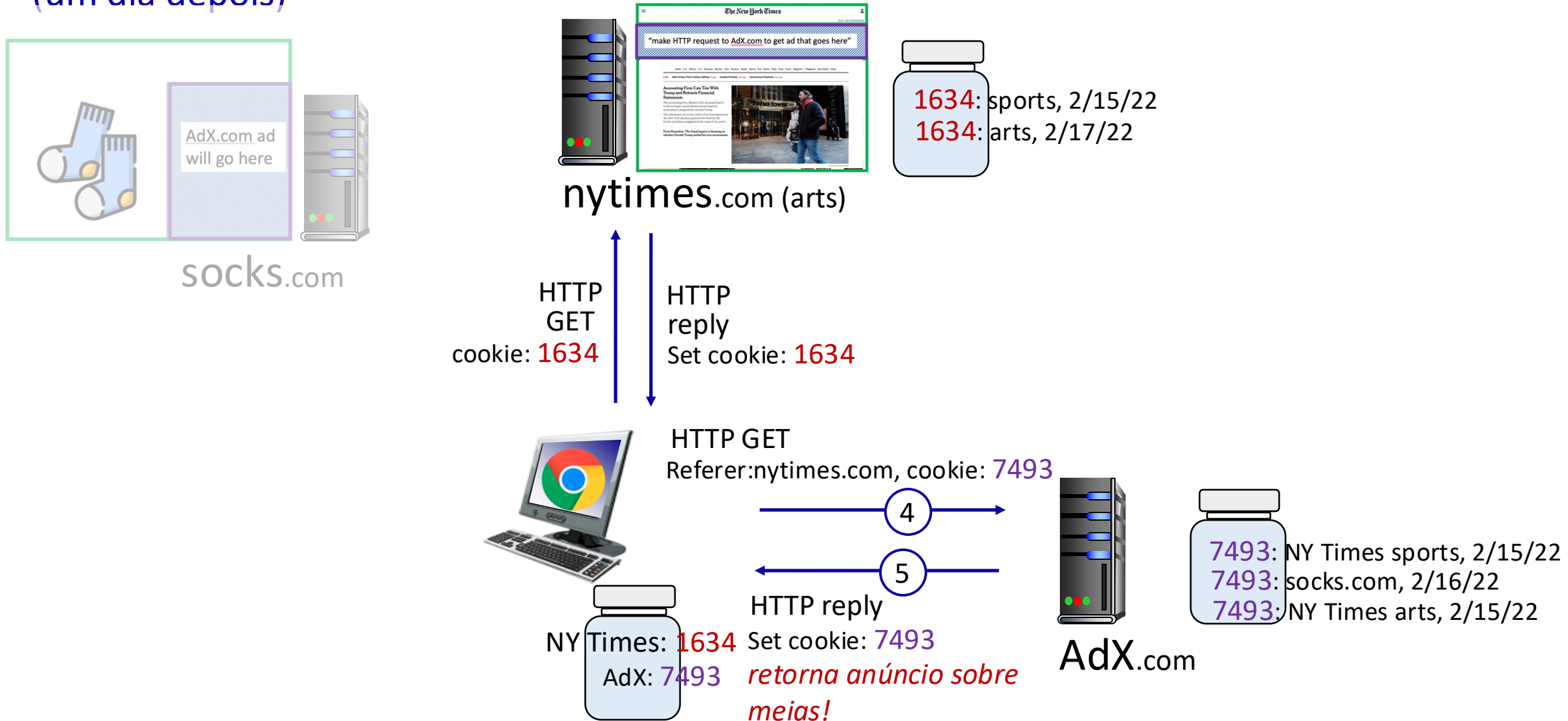


# Cookies: rastreando o comportamento de um usuário



# Cookies: rastreando o comportamento de um usuário

(um dia depois)





# Cookies: rastreando o comportamento de um usuário

Cookies podem se usados para:

- rastrear o comportamento do usuário em um dado sítio (**cookies primários**)
- rastrear o comportamento do usuário em diversos sítios (**cookies de terceiros**) sem que o usuário nunca tenha escolhido visitar o site rastreador (!)
- o rastreamento pode ser *invisível* ao usuário:
  - ao invés de apresentar um anúncio, o HTTP GET ao rastreador poderia ser um *link* invisível

rastreamento de terceiros através de cookies:

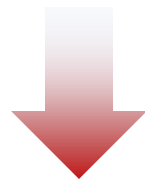
- desabilitados por default nos navegadores Firefox, Safari
- serão desabilitados no navegador Chrome em 2024 (**seria em 2023**)

# A GDPR (LGPD europeia) e os cookies

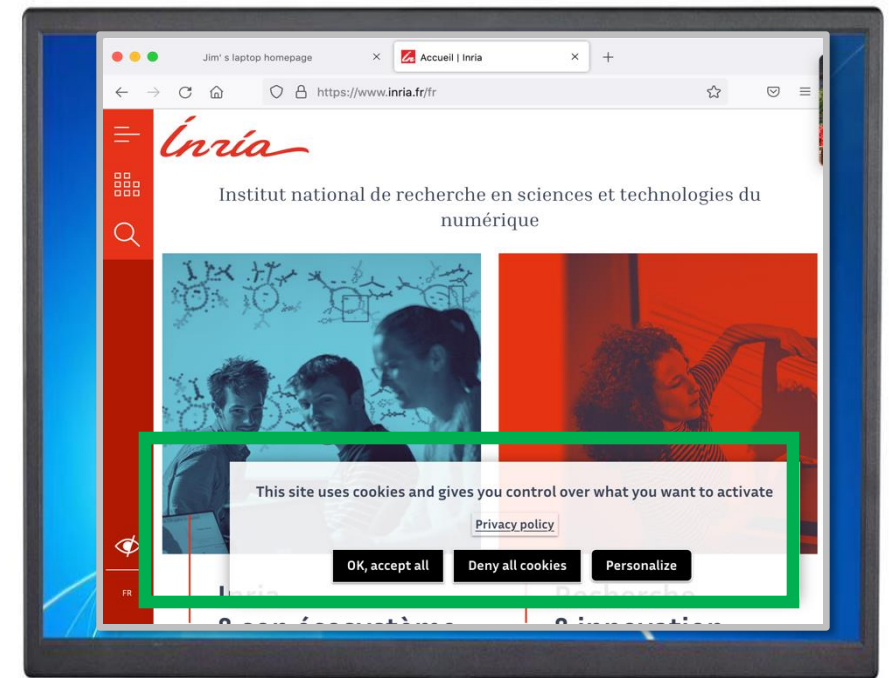
“As pessoas singulares podem ser associadas a identificadores por via eletrônica [...] tais como endereços IP, testemunhos de conexão (cookie) ou outros identificadores [...].

Estes identificadores podem deixar vestígios que, em especial quando combinados com identificadores únicos e outras informações recebidas pelos servidores, podem ser utilizados para a definição de perfis e a identificação das pessoas singulares.”

GDPR, considerando 30 (Maio 2018)



quando os cookies podem identificar uma pessoa, eles são considerados dados pessoais, sujeitos às regulações da GDPR sobre dados pessoais



*O usuário pode controlar explicitamente se os cookies são permitidos ou não*

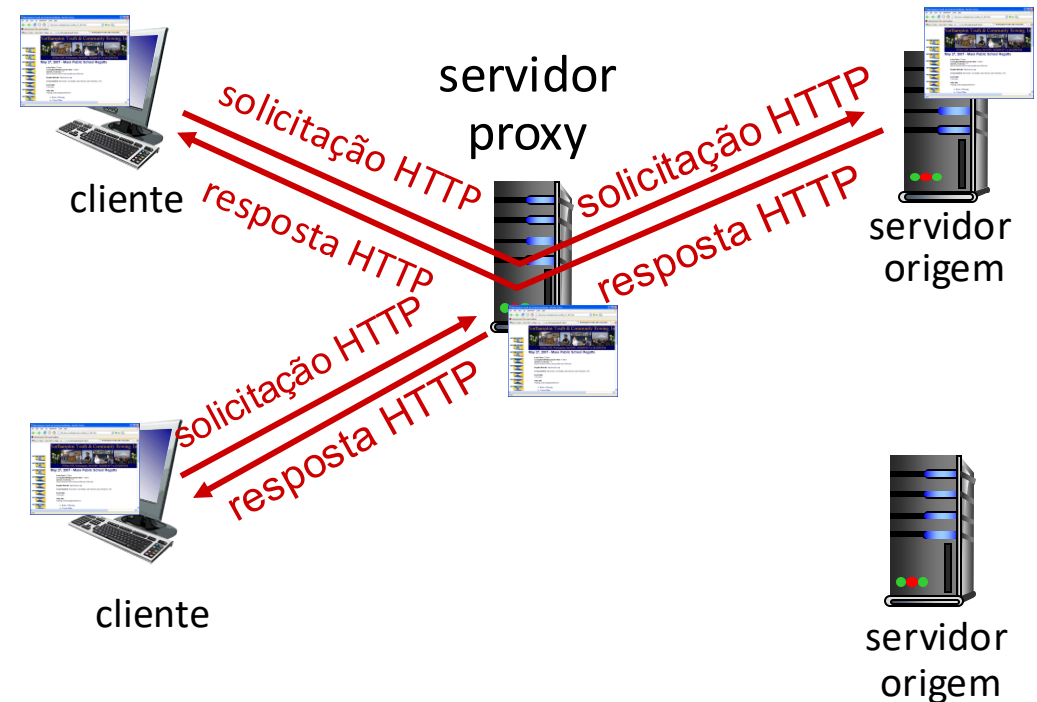
# LGPD (Lei Geral da Proteção de Dados) e Cookies

- A LGPD não menciona explicitamente os cookies, mas diz:
  - Art. 12 § 2º Poderão ser igualmente considerados como dados pessoais, para os fins desta Lei, aqueles utilizados para formação do perfil comportamental de determinada pessoa natural, se identificada.
- Recentemente (Out/2022) a Autoridade Nacional de Proteção de Dados (ANPD) publicou o Guia Orientativo “Cookies e proteção de dados pessoais”:
  - Este guia indica como utilizar cookies de modo a cumprir o disposto na LGPD.
  - Orienta, por exemplo, sobre o uso dos Banners de cookies.
  - [www.anpd.gov.br](http://www.anpd.gov.br)

# Caches Web (servidores *proxy*)

**Objetivo:** atender pedido do cliente sem envolver servidor de origem

- usuário configura o navegador para acessar o *cache Web*
- navegador envia todas as solicitações HTTP para o cache
  - *se* objeto estiver no cache: cache retorna o objeto para o cliente
  - *caso contrário* cache solicita objeto ao servidor origem, armazena o objeto recebido, depois retorna o objeto para o cliente



# Caches Web (servidores *proxy*)

- Cache Web age tanto como cliente como servidor
  - servidor para o cliente que faz a solicitação original
  - cliente para o servidor origem
- servidor informa à cache sobre a permissão de realizar o cache no cabeçalho da resposta

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

*Para que* usar cache Web?

- reduz o tempo de resposta para as solicitações dos clientes
  - cache mais próximo do cliente
- reduz o tráfego no enlace de acesso institucional
- a Internet fica mais densa com caches
  - permitem que provedores de conteúdo “pobres” entreguem conteúdo de forma mais efetiva

# Exemplo de *Cache*

## *Cenário:*

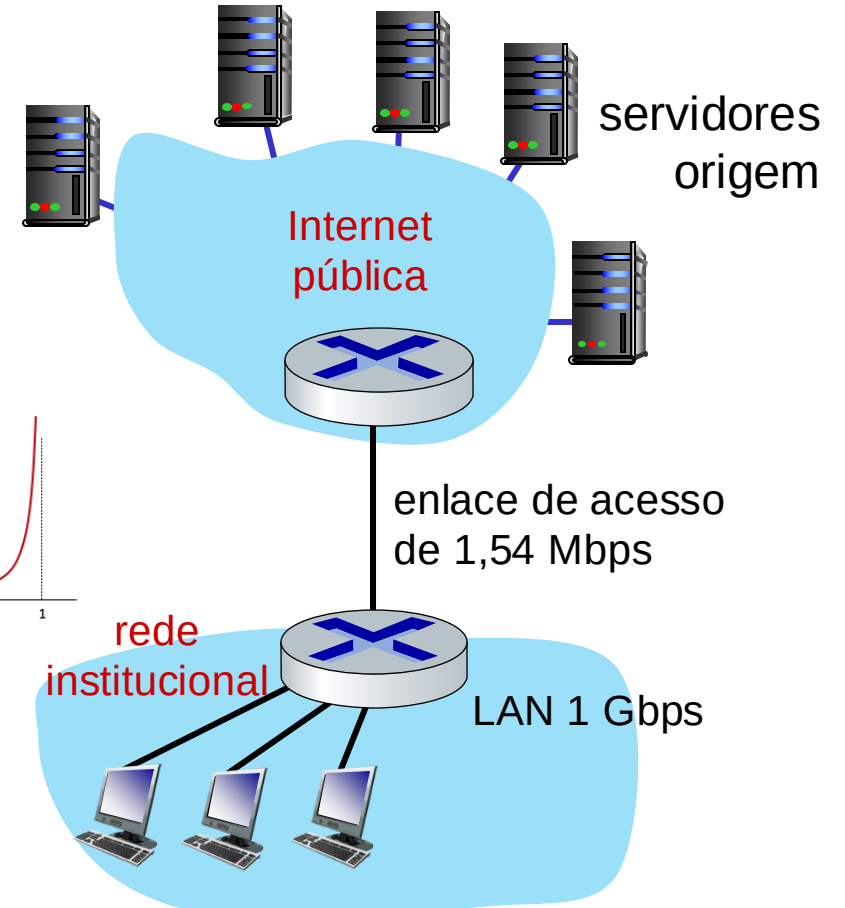
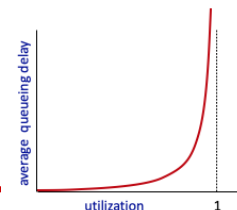
- taxa do enlace de acesso: 1,54 Mbps
- RTT do roteador institucional para o servidor: 2 seg
- tamanho dos objetos Web: 100 kbits
- taxa média de solicitações dos navegadores para os servidores originais: 15/seg
  - taxa média para os servidores: 1,50 Mbps

## *Desempenho:*

- utilização da LAN : 0,0015
- utilização do enlace de acesso = 0,97
- atraso fim-a-fim = atraso da Internet +  
atraso do acesso + atraso da LAN  
= 2 seg + minutos +  $\mu$ secs

*problema:*

*grandes atrasos  
com alta  
utilização!*



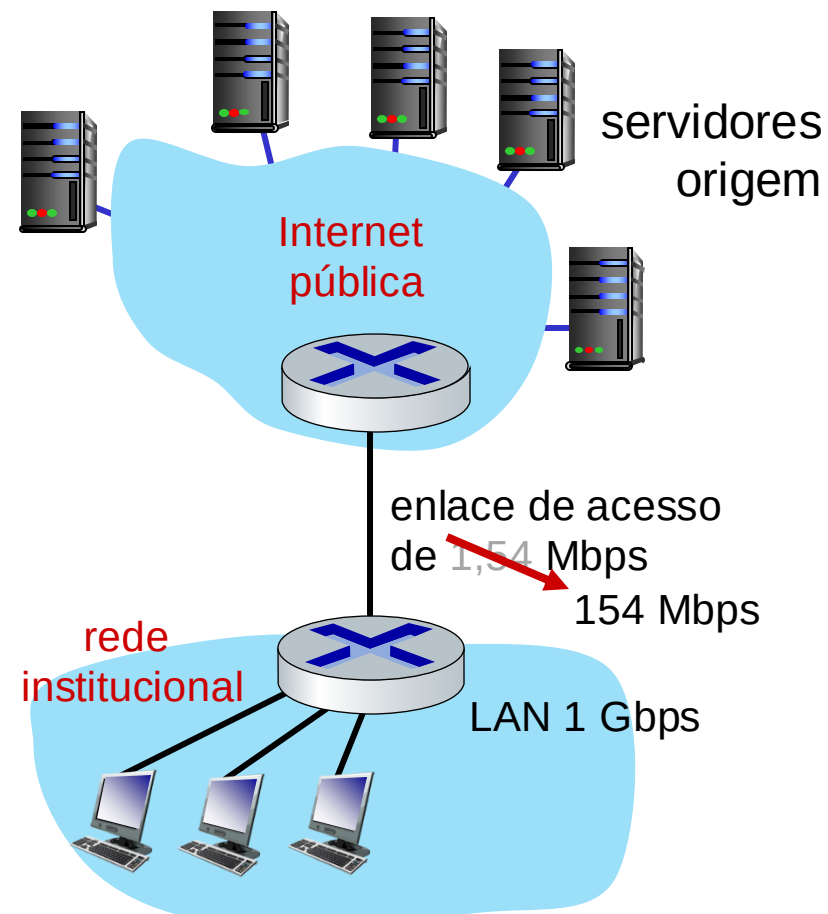
# Exemplo de *Cache*: contratar acesso mais rápido

## *Cenário:*

- taxa do enlace de acesso: ~~1,54~~ 154 Mbps
- RTT do roteador institucional para o servidor: 2 seg
- tamanho dos objetos Web: 100 kbits
- taxa média de solicitações dos navegadores para os servidores originais: 15/seg
- taxa média para os servidores: 1,50 Mbps

## *Desempenho:*

- utilização da LAN: 0,0015
- utilização do enlace de acesso = ~~0,97~~ 0,0097
- atraso fim-a-fim = atraso da Internet +  
atraso do acesso + atraso da LAN  
= 2 seg + ~~minutos~~  $\mu$ segs  
msecs



*Custo:* enlace de acesso mais rápido (caro!)

# Exemplo de *Cache*: instalação de um cache web

## *Cenário:*

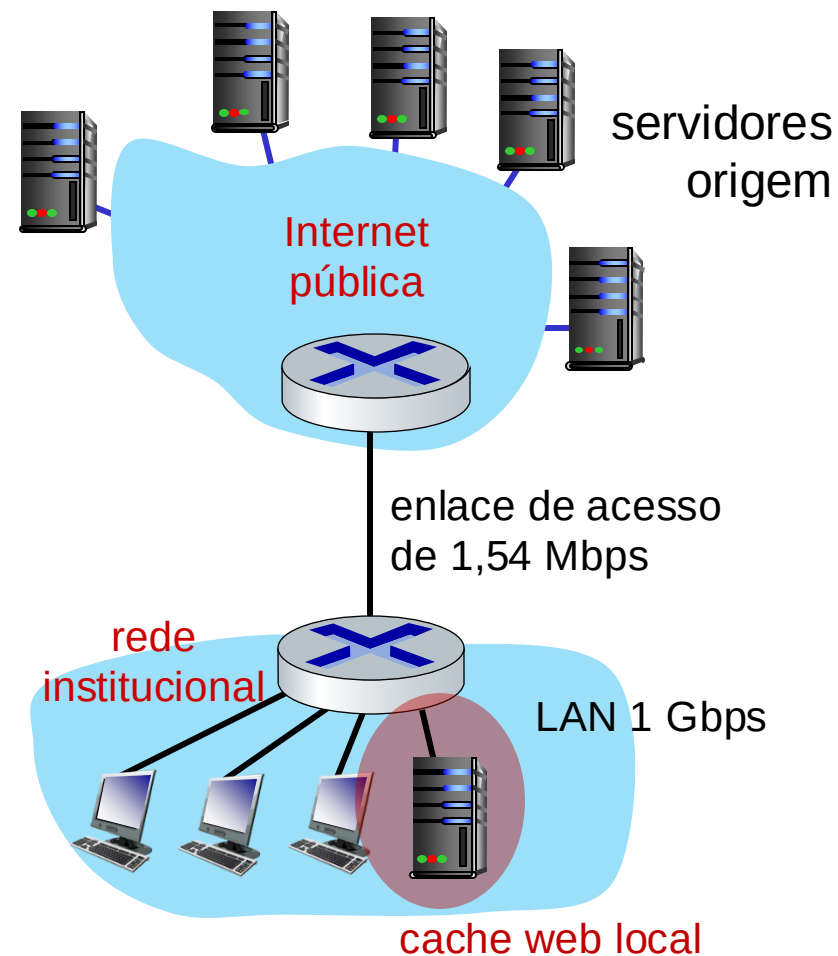
- taxa do enlace de acesso: 1,54 Mbps
- RTT do roteador institucional para o servidor: 2 seg
- tamanho dos objetos Web: 100 kbits
- taxa média de solicitações dos navegadores para os servidores originais: 15/seg
  - taxa média para os servidores: 1,50 Mbps

## *Desempenho:*

- utilização da LAN: ?
- utilização do enlace de acesso = ?
- atraso médio fim-a-fim = ?

*Como calcular a utilização e atraso no enlace?*

*Custo:* cache web (barato!)

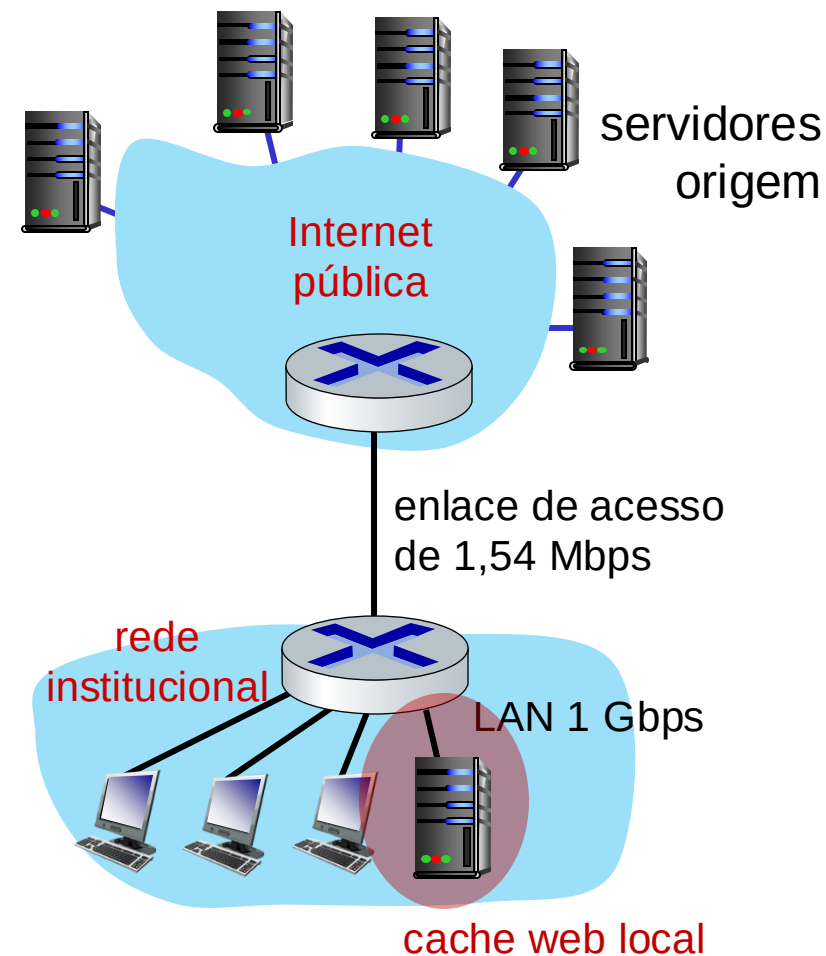




# Exemplo de *Cache*: instalação de um cache web

## Cálculo da utilização do enlace de acesso e atraso fim-a-fim com cache:

- assuma que a taxa de acerto seja de 0,4: 40% dos pedidos atendidos pelo cache, 60% dos pedidos atendidos pela origem
- enlace de acesso: 60% dos pedidos usam o enlace de acesso
- taxa de dados dos navegadores pelo enlace  
 $= 0,6 * 1,50 \text{ Mbps} = 0,9 \text{ Mbps}$
- utilização  $= 0,9 / 1,54 = 0,58$
- atraso médio fim-a-fim  
 $= 0,6 * (\text{atraso até servidores origem})$   
 $+ 0,4 * (\text{atrasos até o cache})$   
 $= 0,6 (2,01) + 0,4 (\sim \text{msecs}) = \sim 1,2 \text{ segs}$



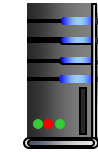
*menor atraso médio fim-a-fim do que com enlace de 154 Mbps (e mais barato!)*

# Cache no navegador: GET Condicional

cliente



servidor



**Meta:** não enviar objeto se o cache já tiver uma versão atualizada

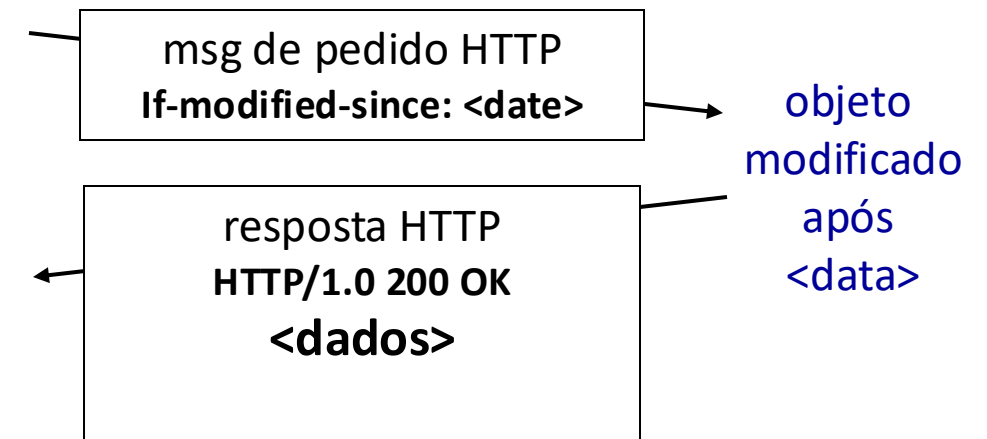
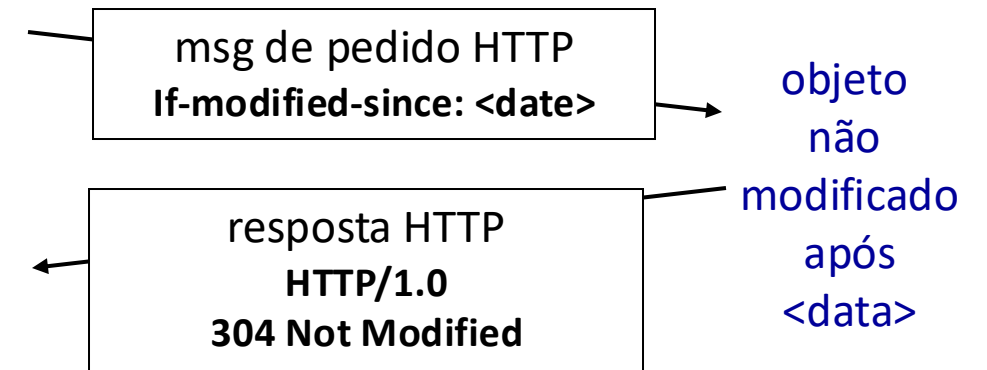
- sem atraso de transmissão do objeto
- menor utilização do enlace

■ **cliente:** especifica data da cópia no cache no pedido HTTP

**If-modified-since: <date>**

■ **servidor:** resposta sem objeto se cópia no cache estiver atualizada:

**HTTP/1.0 304 Not Modified**



# HTTP/2

*Objetivo chave:* atraso reduzido em solicitações HTTP de múltiplos objetos

HTTP1.1: introduziu múltiplos GET em série sobre uma única conexão TCP

- servidor responde *em ordem* (escalonamento FCFS: *first-come-first-served*) às solicitações de GET
- com FCFS, objeto pequeno pode ter que esperar sua vez de transmissão atrás de grande(s) objeto(s) (bloqueio pelo cabeça da fila (*HOL - head-of-line*))
- recuperação de perdas (retransmissão de segmentos TCP perdidos) interrompe a transmissão de objetos

# HTTP/2

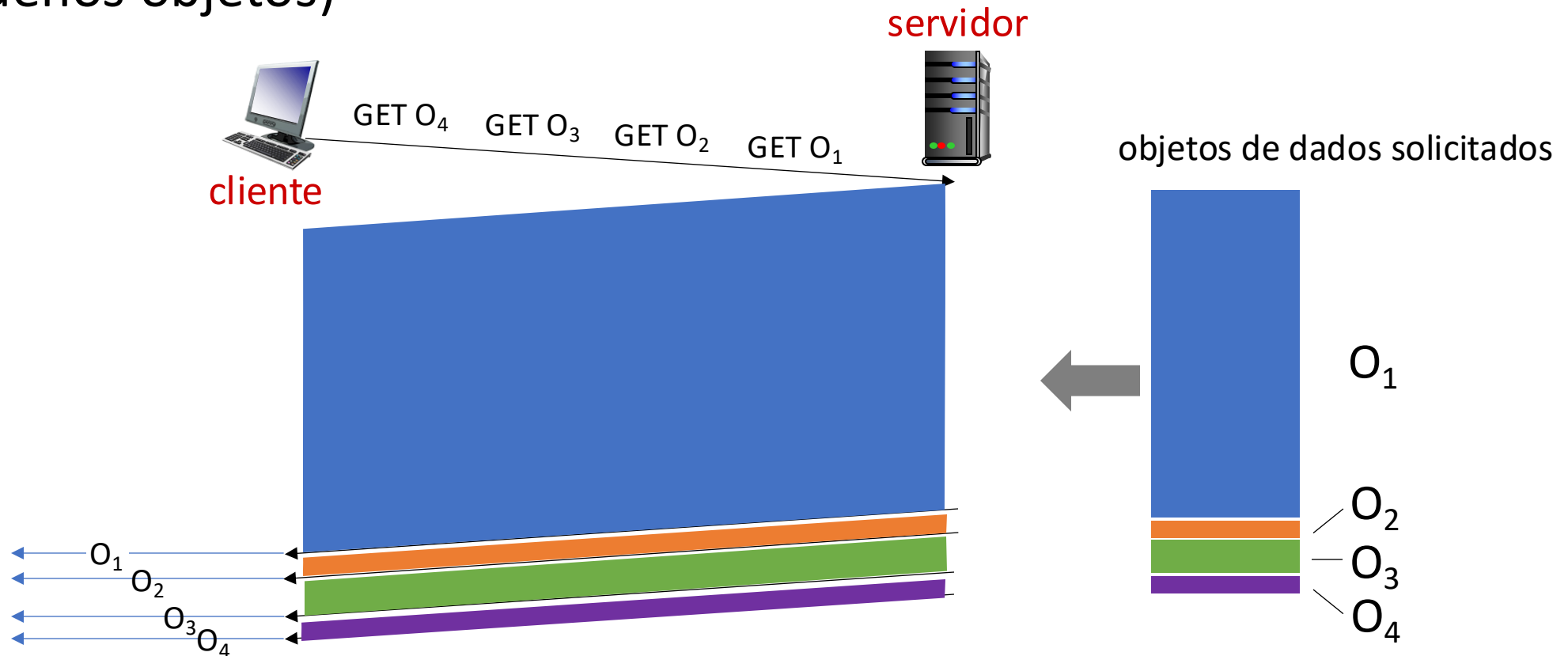
*Objetivo chave:* atraso reduzido em solicitações HTTP de múltiplos objetos

HTTP/2: [RFC 7540, 2015] maior flexibilidade do *servidor* em enviar objetos para o cliente:

- métodos, códigos de status, muitos campos de cabeçalhos inalterados em relação ao HTTP 1.1
- ordem de transmissão dos objetos solicitados baseados na prioridade especificada pelo cliente (não necessariamente FCFS)
- *envia* objetos não solicitados pelo cliente
- divide objetos em quadros, escalona quadros para mitigar o bloqueio HOL

# HTTP/2: mitigando o bloqueio HOL

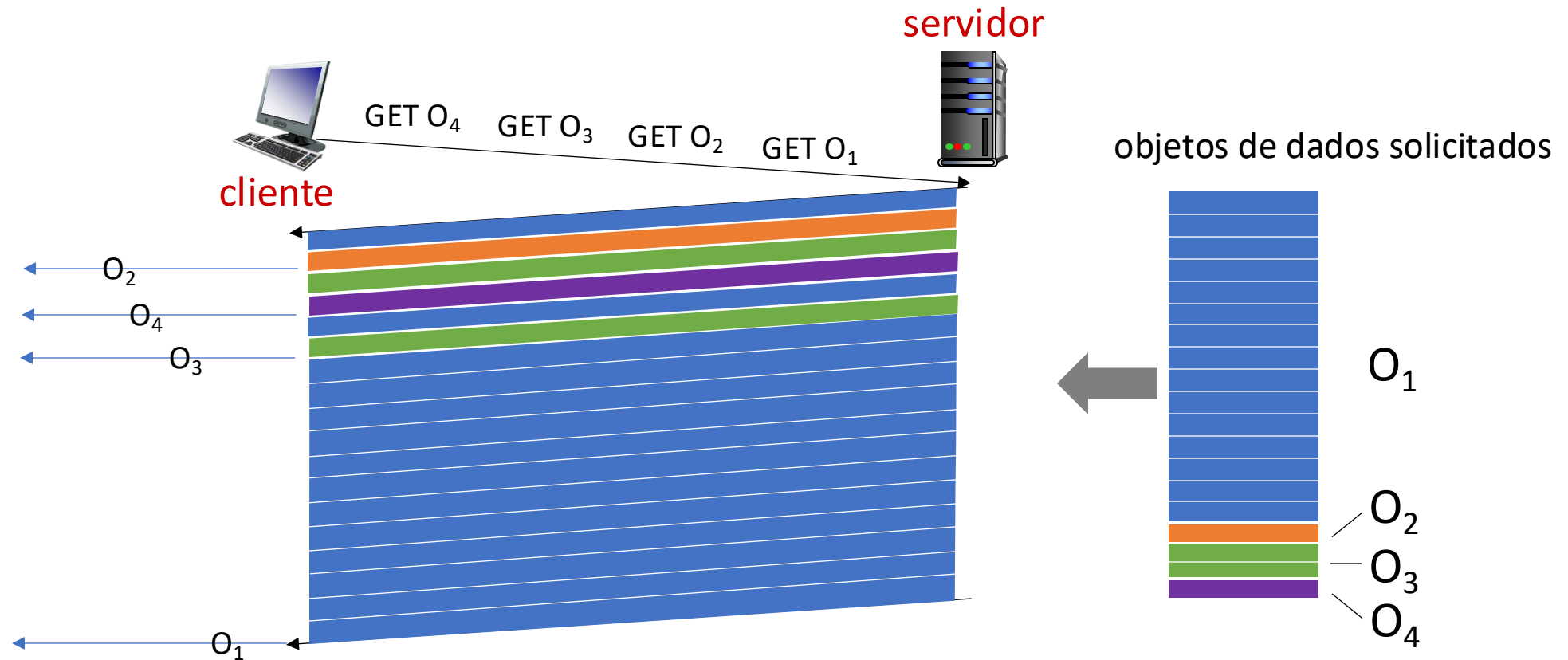
HTTP 1.1: cliente solicita 1 grande objeto (ex., arquivo de vídeo, e 3 pequenos objetos)



*objetos entregues na ordem das solicitações:  $O_2$ ,  $O_3$ ,  $O_4$  esperam atrás do  $O_1$*

# HTTP/2: mitigando o bloqueio HOL

HTTP/2: objetos divididos em quadros, transmissão intercalada dos quadros



*$O_2$ ,  $O_3$ ,  $O_4$  entregues rapidamente,  $O_1$  levemente atrasado*

# HTTP/2 para HTTP/3

HTTP/2 sobre uma única conexão TCP significa:

- recuperação das perdas ainda interrompe todas as transmissões de objetos
  - como no HTTP 1.1, navegadores são incentivados a abrir múltiplas conexões TCP em paralelo para reduzir o bloqueio e aumentar a vazão total
- sem segurança sobre conexão TCP básica
- **HTTP/3**: adiciona segurança, controle de erro e congestionamento por objeto (maior paralelismo) sobre UDP
  - mais sobre o HTTP/3 na camada de transporte

# Camada de Aplicação: roteiro

- Princípios de aplicações de rede
- A Web e o HTTP
- E-mail, SMTP, IMAP
- DNS: o serviço de diretório da Internet
- Aplicações P2P
- fluxos de vídeo e redes de distribuição de conteúdos
- programação de sockets com UDP e TCP





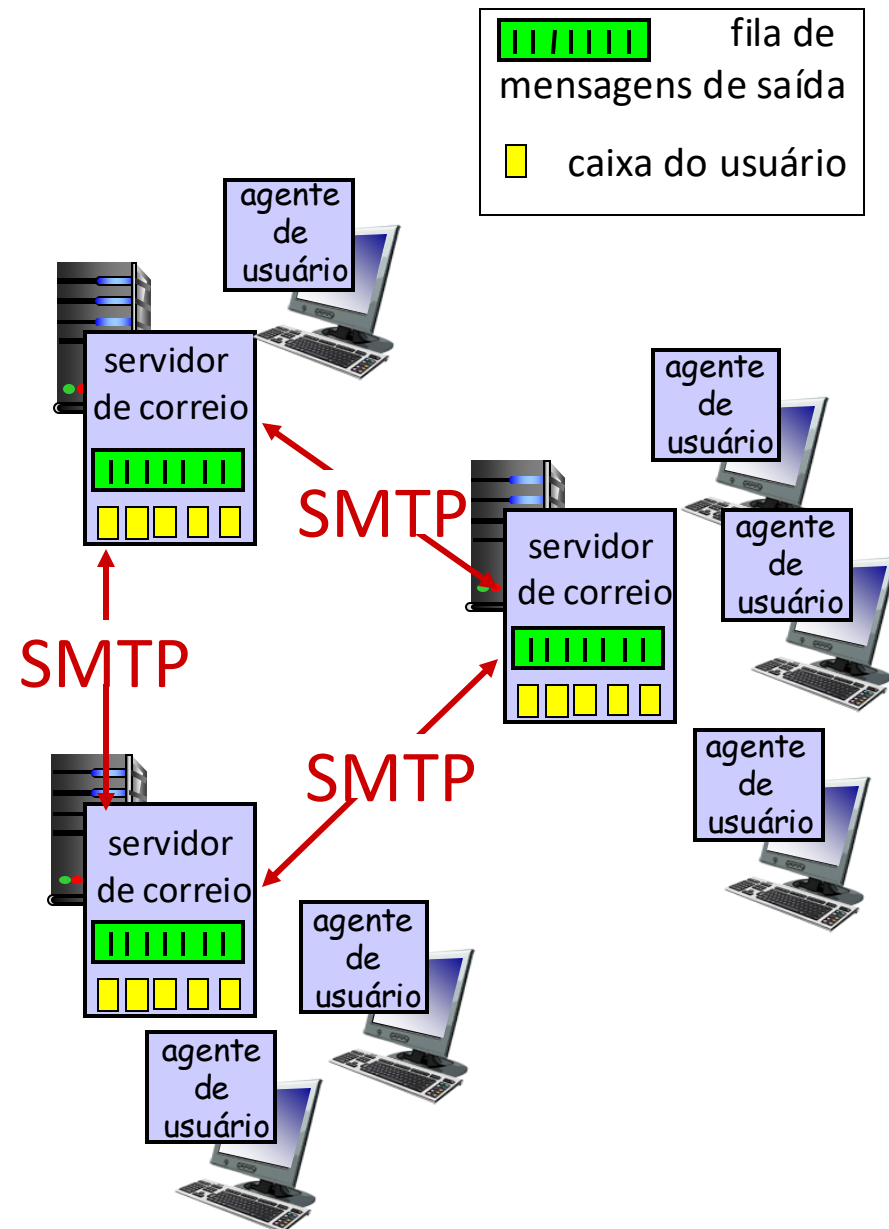
# E-mail

## Três componentes principais:

- agentes de usuário
- servidores de correio (mail)
- *Simple Mail Transfer Protocol*: SMTP

## Agente do Usuário

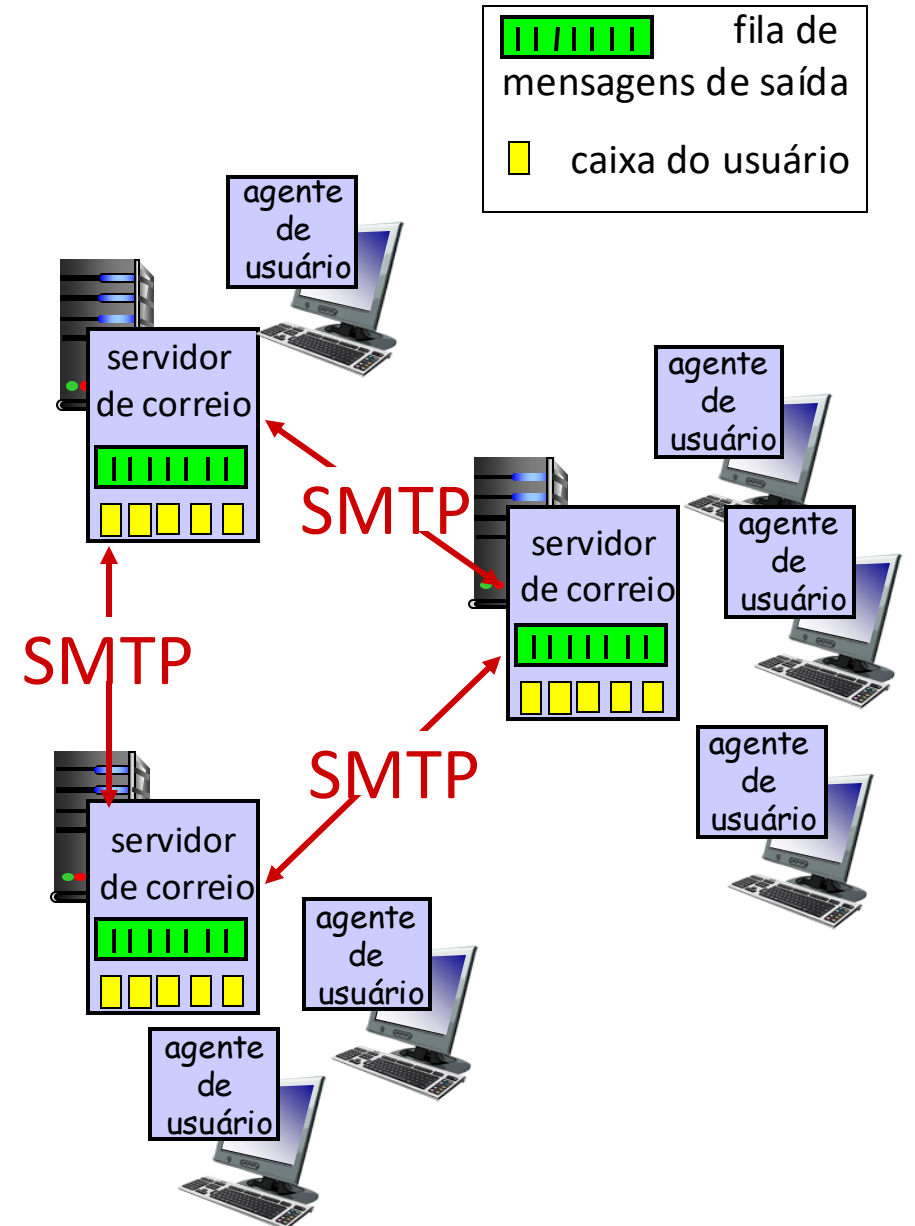
- conhecido como “leitor de correio”
- compõe, edita, lê mensagens de correio
- ex., Outlook, cliente de mail do iPhone
- mensagens de saída e recebidas são armazenadas no servidor



# E-mail: servidores de correio

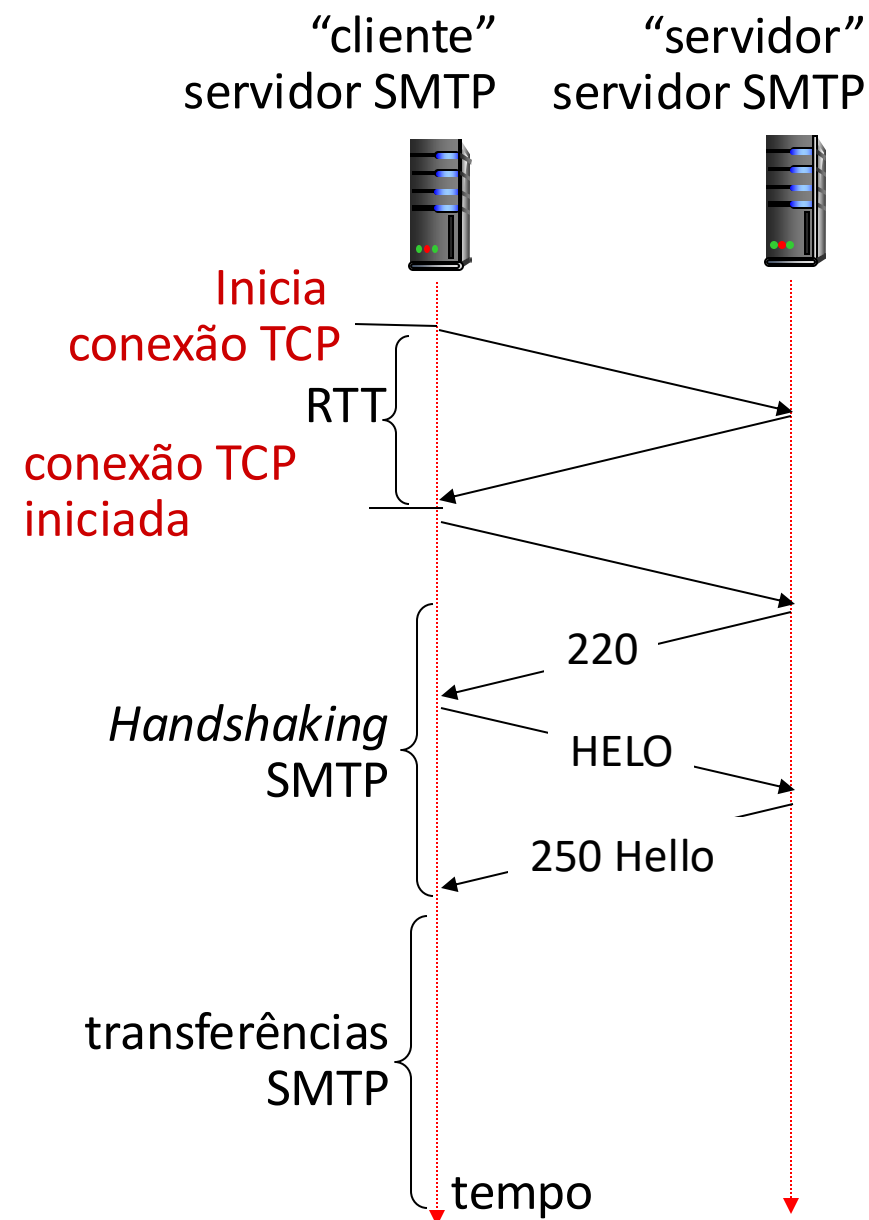
## servidores de correio:

- caixa de correio (*mailbox*) contém mensagens recebidas pelo usuário
- *fila de mensagens* contém mensagens de saída (a serem enviadas)
- *protocolo SMTP* entre servidores de correio para transferir mensagens de correio
  - *cliente*: servidor de correio que envia
  - “*servidor*”: servidor de correio que recebe



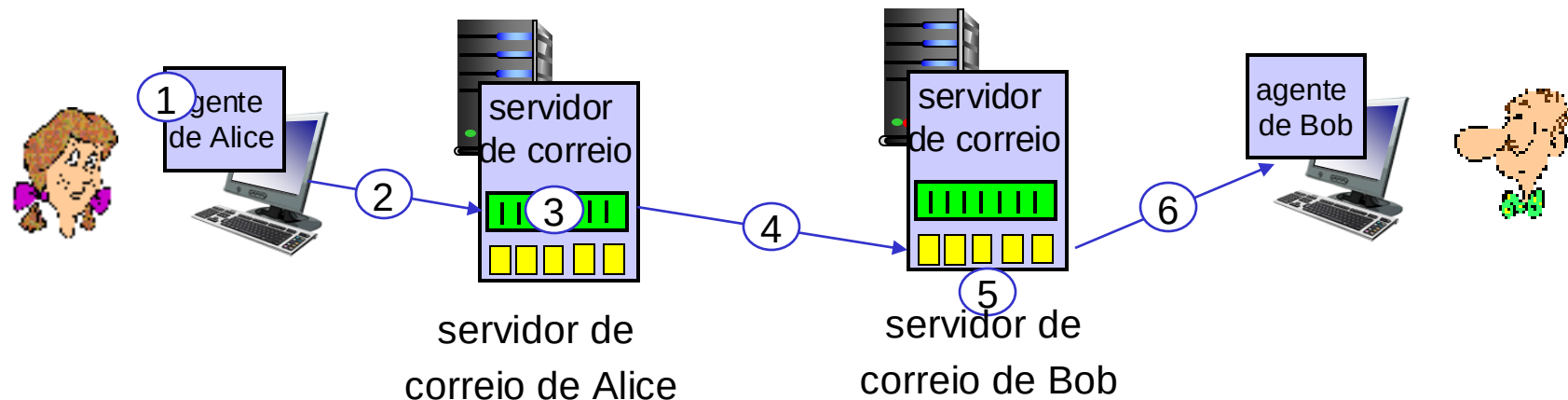
# RFC 5321 do SMTP

- usa TCP para a transferência confiável de msgs de correio do cliente (servidor que inicia a conexão) ao servidor, porta 25
  - transferência direta: servidor remetente (agindo como cliente) ao servidor destinatário
- três fases da transferência
  - *Handshaking* SMTP (saudação)
  - transferência das mensagens SMTP
  - Encerramento SMTP
- interação comando/resposta (como o HTTP)
  - **comandos**: texto ASCII
  - **resposta**: código e frase de status



# Cenário: Alice envia e-mail para Bob

- 1) Alice usa o UA para compor mensagem de e-mail “para” bob@some school.edu
- 2) O UA de Alice envia a mensagem para o seu servidor de correio; a mensagem é colocada na fila de mensagens
- 3) O lado cliente do SMTP abre uma conexão TCP com o servidor de correio de Bob
- 4) O cliente SMTP envia a mensagem de Alice através da conexão TCP
- 5) O servidor de correio de Bob coloca a mensagem na caixa de entrada de Bob
- 6) Bob chama o seu UA para ler a mensagem



# Interação SMTP típica

S: 220 hamburger.edu

# SMTP: observações

## *comparação com o HTTP:*

- HTTP: recupera (*pull*)
  - SMTP: envia (*push*)
  - ambos têm interação comando/resposta, códigos de status
  - HTTP: cada objeto é encapsulado em sua própria mensagem de resposta
  - SMTP: múltiplos objetos de mensagem enviados numa mensagem de múltiplas partes
- SMTP usa conexões persistentes
  - SMTP requer que a mensagem (cabeçalho e corpo) sejam em ASCII de 7-bits
  - servidor SMTP usa CRLF.CRLF para reconhecer o final da mensagem

# Formato de uma mensagem de Correio

SMTP: protocolo para enviar mensagens de e-mail, definido na RFC 5321 (assim como o HTTP está definido na RFC 7231)

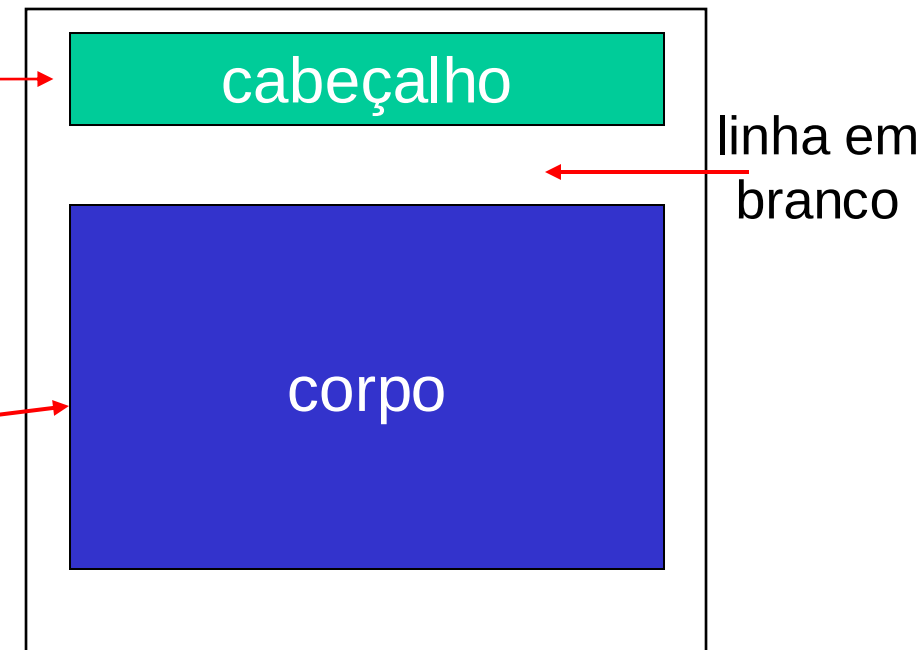
RFC 2822 define a *sintaxe* das mensagens de e-mail (como o HTML define a sintaxe dos documentos web)

- linhas de cabeçalho, ex.,

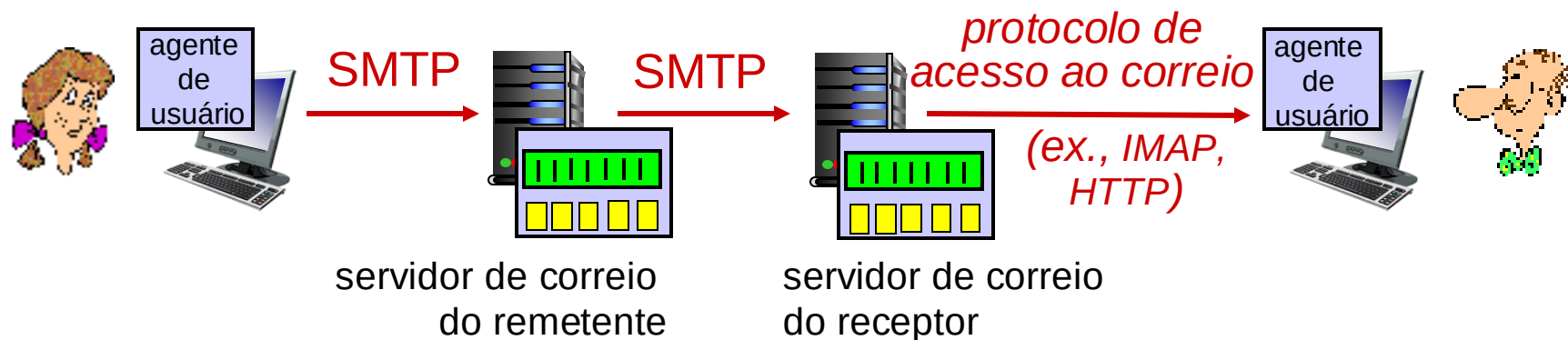
- To:
- From:
- Subject:

estas linhas, dentro do corpo da mensagem de correio são diferentes dos comandos SMTP MAIL FROM:, RCPT TO:!

- Corpo: da “mensagem”, apenas caracteres ASCII



# Protocolos de acesso ao correio



- **SMTP:** entrega/armazenamento de mensagens de correio no servidor do receptor
- protocolo de acesso ao correio: recuperação do servidor
  - **IMAP:** *Internet Mail Access Protocol* [RFC 3501]: mensagens são armazenadas no servidor, IMAP recupera, deleta, cria pastas de mensagens no servidor
- **HTTP:** gmail, Hotmail, Yahoo!Mail, etc. proveem interface web usando o SMTP (para enviar), IMAP (ou POP) para recuperar mensagens de correio



# Camada de Aplicação: roteiro

- Princípios de aplicações de rede
- A Web e o HTTP
- E-mail, SMTP, IMAP
- DNS: o serviço de diretório da Internet
- Aplicações P2P
- fluxos de vídeo e redes de distribuição de conteúdos
- programação de sockets com UDP e TCP



# DNS: *Domain Name System*

*peessoas:* muitos identificadores:

- CPF, nome, RG, passaporte

*hospedeiros, roteadores na Internet:*

- endereço IP (32 bits) - usado para endereçar os datagramas
- “nome”, ex., cs.umass.edu - usado por humanos

Q: como mapear entre endereço IP e o nome, e vice versa?

*Domain Name System (DNS):*

- *base de dados distribuída*  
implementada numa hierarquia de diversos *servidores de nomes*
- *protocolo da camada de aplicação:*  
hospedeiros, servidores de nomes se comunicam para *resolver* nomes (tradução endereço/nome)
  - nota: função chave da Internet, *implementada como protocolo da camada de aplicação*
  - complexidade na “borda” da rede

# DNS: serviços, estrutura

## Serviços DNS

- tradução de nome do hospedeiro para endereço IP
- apelidos para hospedeiros (*aliasing*)
  - nomes canônicos e apelidos
- apelidos para servidores de e-mail
- Distribuição de carga
  - Servidores Web replicados: conjunto de endereços IP para um mesmo nome

## *Q: Por que não centralizar o DNS?*

- ponto único de falha
- volume de tráfego
- base de dados centralizada e distante
- manutenção

## *R: não é escalável!*

- só os servidores DNS da Comcast: 600B de consultas DNS por dia
- só os servidores DNS da Akamai: 2,2T de consultas DNS por dia

# Pensando sobre o DNS

imensa base de dados distribuída:

- ~ bilhões de registros, cada um simples

lida com trilhões de consultas/dia:

- *muito* mais leituras do que escritas
- *desempenho é importante*: quase todas as interações da Internet interagem com o DNS – cada mseg conta!

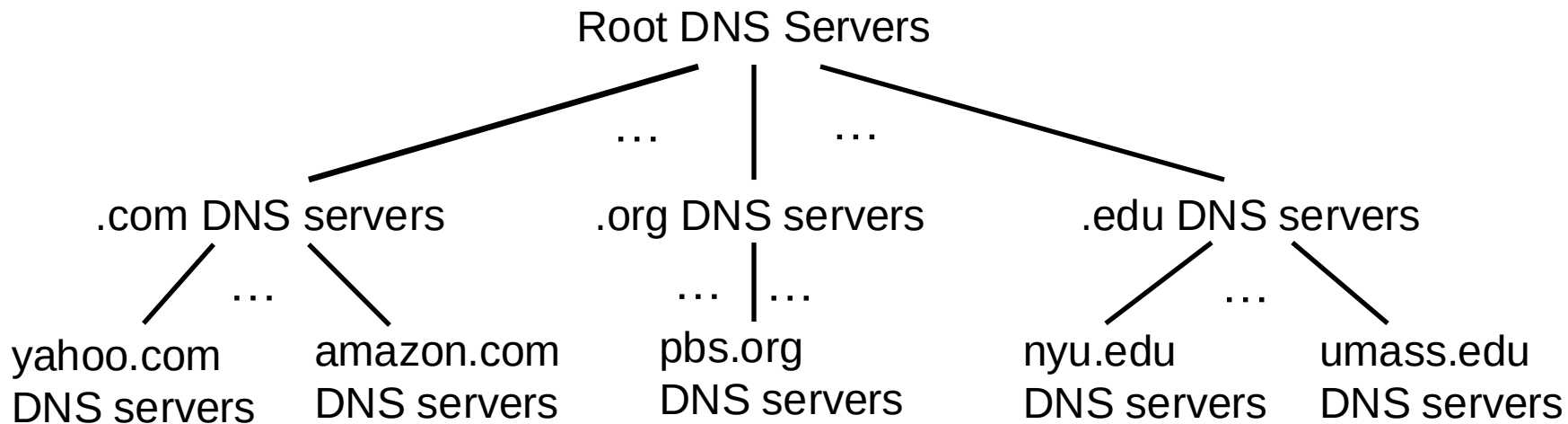
descentralizado organizacional e fisicamente:

- milhões de diferentes organizações são responsáveis por seus registros

“à prova de balas”: confiabilidade, segurança



# DNS: base de dados distribuída e hierárquica



*Raiz*

*Domínio de topo  
(Top Level Domain)*

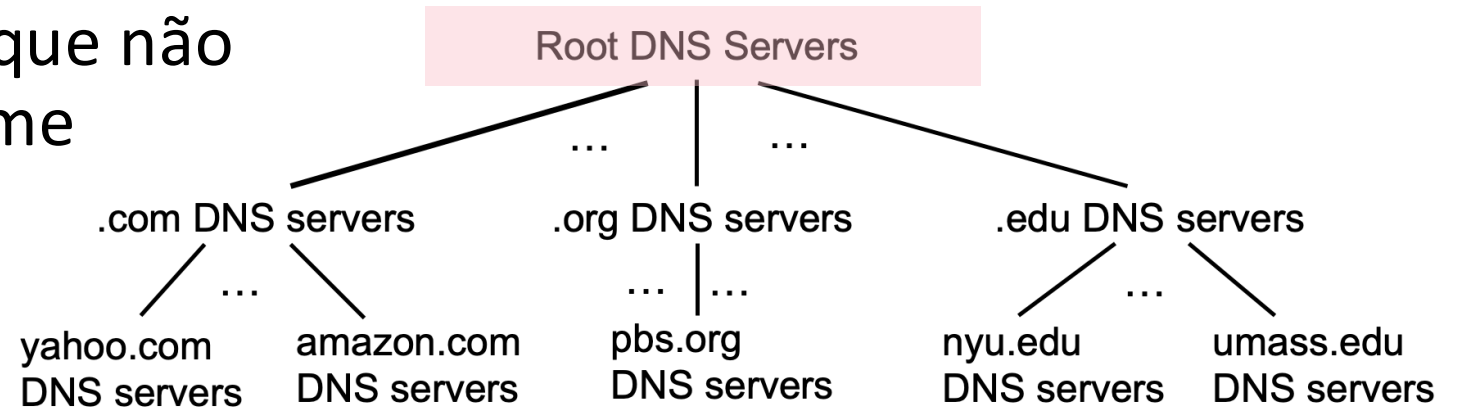
*Com Autoridade*

Cliente quer endereço IP de [www.amazon.com](http://www.amazon.com); 1ª aproximação:

- cliente consulta o servidor raiz para encontrar o servidor DNS .com
- cliente consulta o servidor DNS .com DNS para obter o servidor DNS do domínio amazon.com
- cliente consulta o servidor DNS amazon.com para obter o endereço IP de [www.amazon.com](http://www.amazon.com)

# DNS: servidores raiz

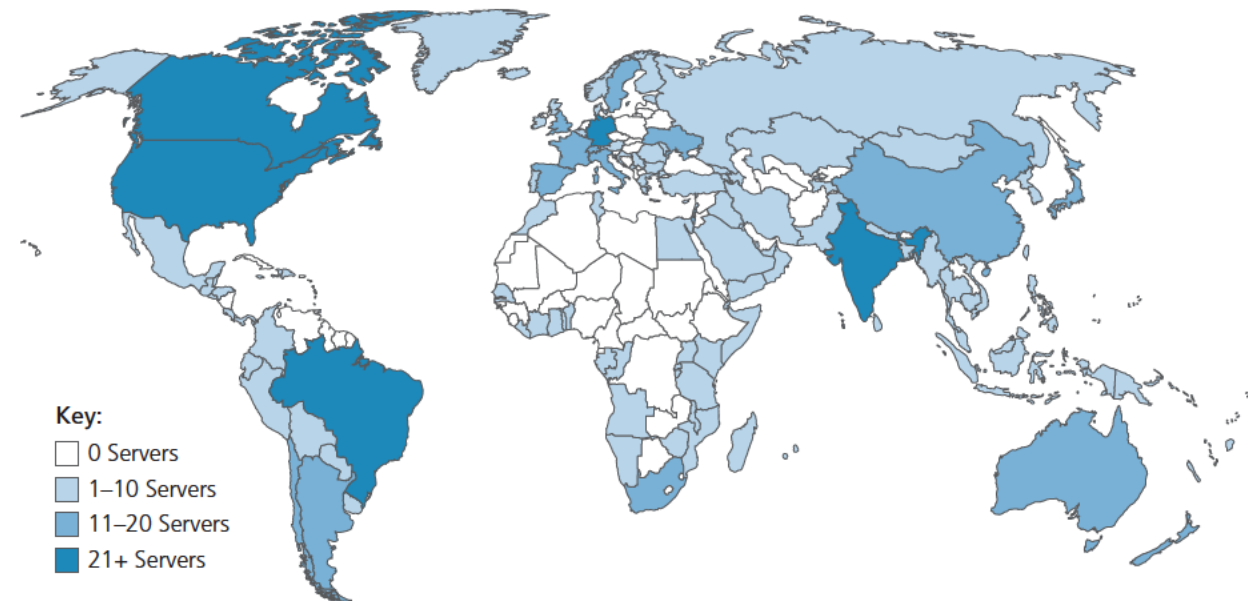
- oficial, último recurso de contato por servidores de nomes que não conseguem resolver o nome



# DNS: servidores raiz

- oficial, último recurso de contato por servidores de nomes que não conseguem resolver o nome
- função *extremamente importante* da Internet
  - a Internet não poderia funcionar sem ela!
  - DNSSEC – provê segurança (autenticação e integridade das mensagens)
- ICANN (*Internet Corporation for Assigned Names and Numbers*) gerencia o domínio DNS raiz

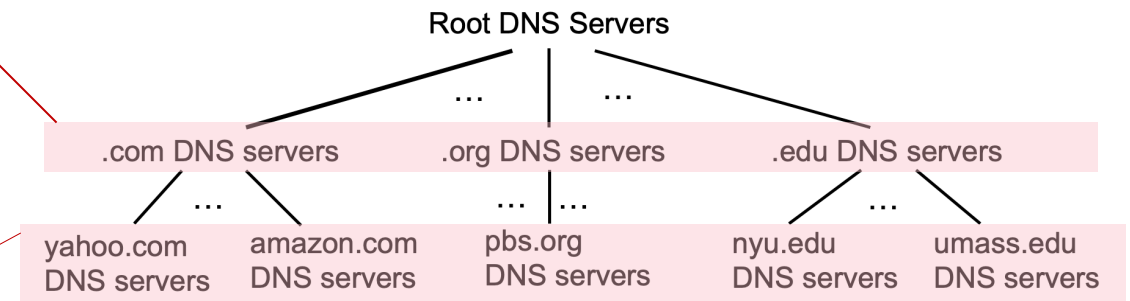
13 “servidores” lógicos de nomes raiz em todo o mundo. Cada “servidor” é replicado muitas vezes (~200 servidores nos Estados Unidos)



# DNS: servidores TLD e com Autoridade

## Servidores de Domínio de Topo (TLD - *Top-Level Domain*):

- responsáveis por .com, .org, .net, .edu, .aero, .jobs, .museums, e todos os domínios de topo de países, e.g.: .cn, .uk, .fr, .ca, .jp, .br
- Network Solutions: registro com autoridade para os TLDs .com, .net
- Educause: TLD .edu



## Servidores DNS com Autoridade:

- servidores DNS das organizações, provendo mapeamentos oficiais entre nomes de hospedeiros e endereços IP para os servidores da organização
- Podem ser mantidos pelas organizações ou seus provedores de acesso



# Servidores DNS locais

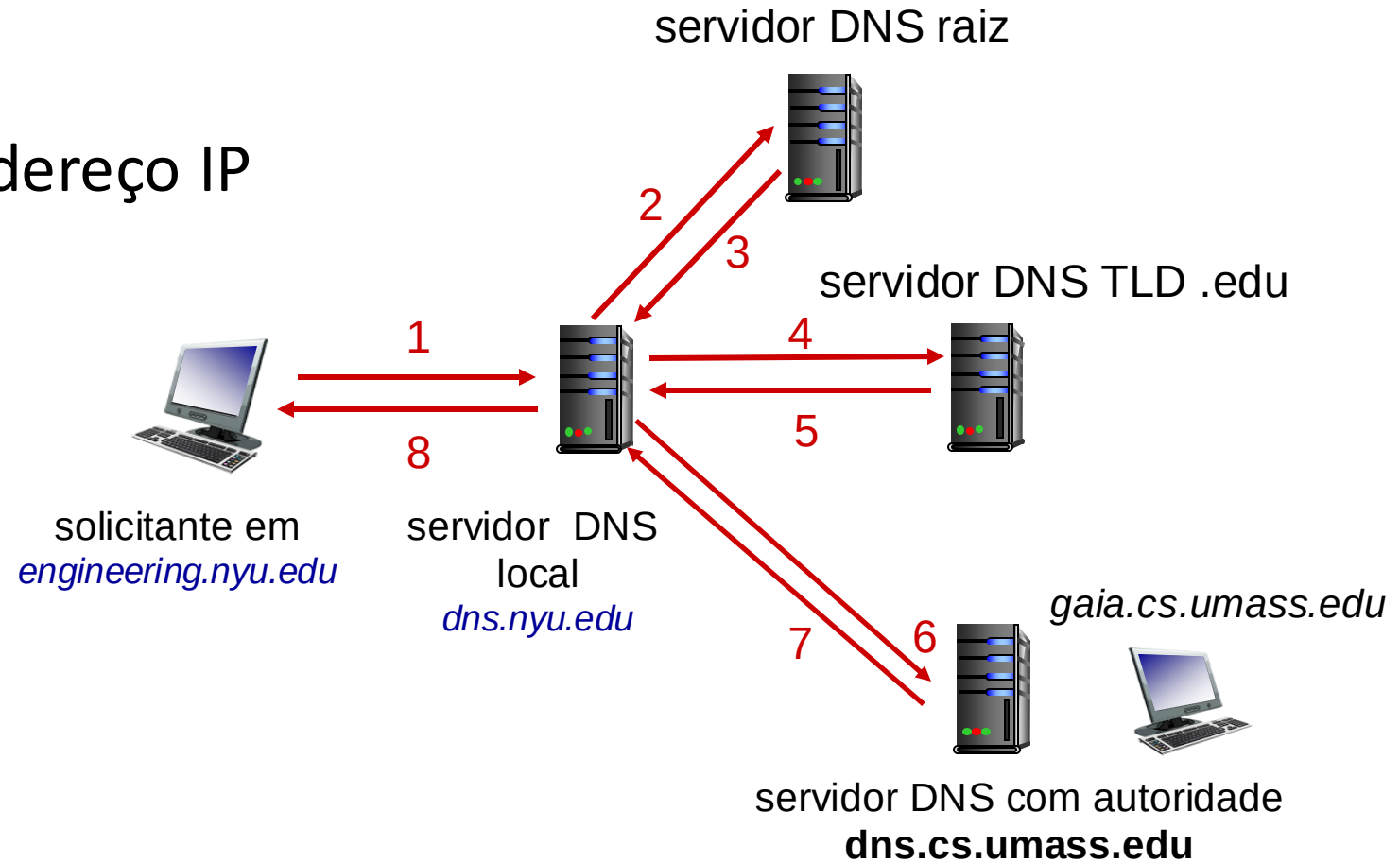
- quando o *host* faz uma consulta DNS, ela é enviada para o seu servidor DNS *local*
  - Servidor DNS local envia a resposta:
    - a partir da sua cache de pares recentes de tradução nome a endereço (possivelmente desatualizada!)
    - encaminhando a consulta para ser resolvida na hierarquia DNS
  - cada ISP possui um servidor DNS local; para encontrar o seu:
    - MacOS: `% scutil --dns`
    - Windows: `>ipconfig /all`
- o servidor DNS local não pertence estritamente à hierarquia

# Resolução de nome DNS: consulta iterativa

**Exemplo:** hospedeiro em `engineering.nyu.edu` quer o endereço IP de `gaia.cs.umass.edu`

## Consulta iterativa:

- servidor consultado responde com o nome de um servidor a ser contactado
- “Não conheço este nome, mas pergunte para esse servidor”

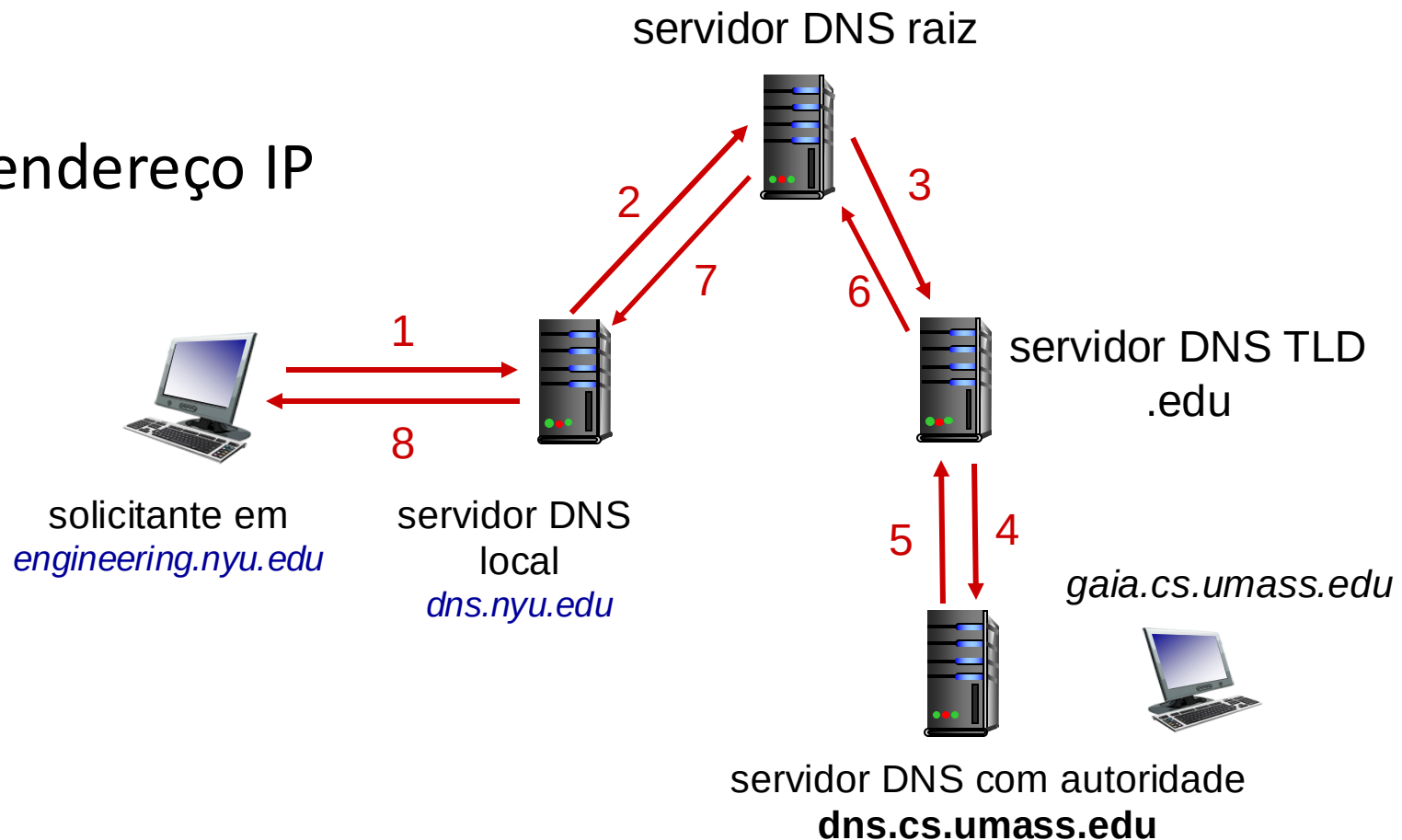


# Resolução de nome DNS: consulta recursiva

**Exemplo:** hospedeiro em `engineering.nyu.edu` quer o endereço IP de `gaia.cs.umass.edu`

## Consulta recursiva:

- transfere a responsabilidade de resolução do nome para o servidor de nomes contatado
- carga pesada nos níveis mais altos da hierarquia?



# Cache de informações do DNS

- assim que (qualquer) servidor de nomes aprende um mapeamento, ele o coloca em uma *cache*, e *imediatamente* retorna o mapeamento na cache em resposta a uma consulta
  - uso de *cache* melhora o tempo de resposta
  - as informações na cache são removidas após algum tempo (TTL)
  - os servidores TLD estão tipicamente em cache nos servidores de nomes locais
- informações na cache podem estar *desatualizadas*
  - se o hospedeiro mudar o seu endereço IP, este novo mapeamento não será conhecido em toda a Internet até que todos os TTLs expirem!
  - *tradução nome-para-endereço no estilo melhor-esforço!*

# Registros DNS

**DNS:** banco de dados distribuído que armazena registros de recursos **(RR)**

formato de um RR: (`nome`, `valor`, `tipo`, `ttl`)

## tipo=A

- `nome` é o nome do hospedeiro
- `valor` é o seu endereço IP

## tipo=NS

- `nome` é o domínio (ex., `foo.com`)
- `valor` é o nome de hospedeiro do servidor de nomes com autoridade para este domínio

## tipo=CNAME

- `nome` é o apelido para algum nome “canônico” (real)
- `www.ibm.com` é na verdade `serveeast.backup2.ibm.com`
- `valor` é o nome canônico

## tipo=MX

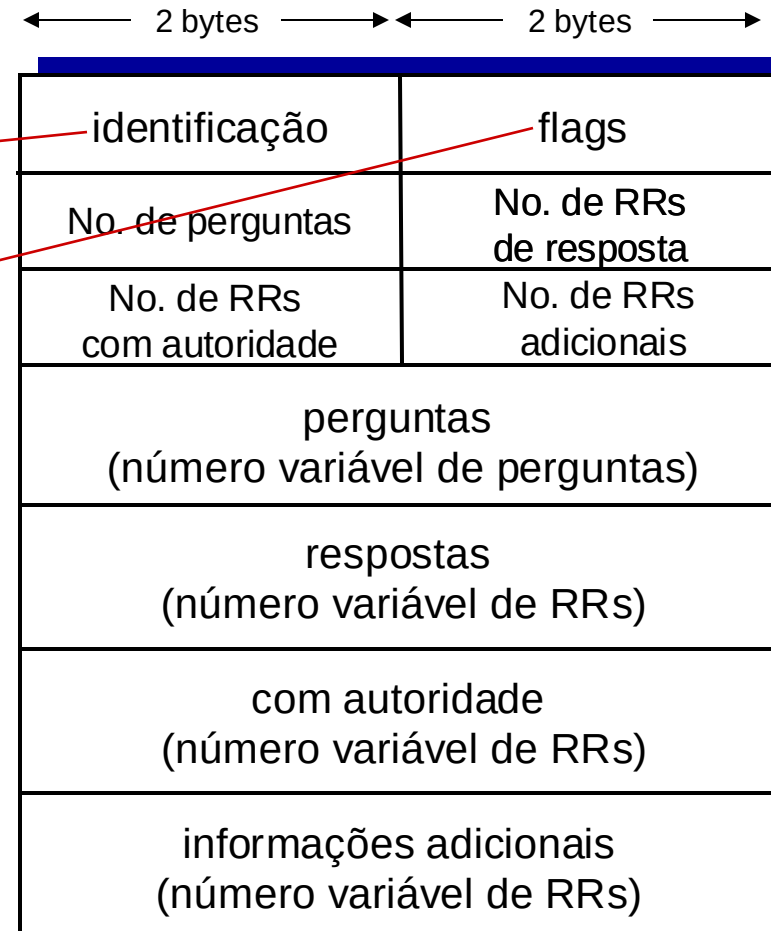
- `valor` é o nome do servidor de correio associado com o `nome`

# Mensagens do protocolo DNS

As mensagens DNS de *consulta* e *resposta*, têm o mesmo *formato*:

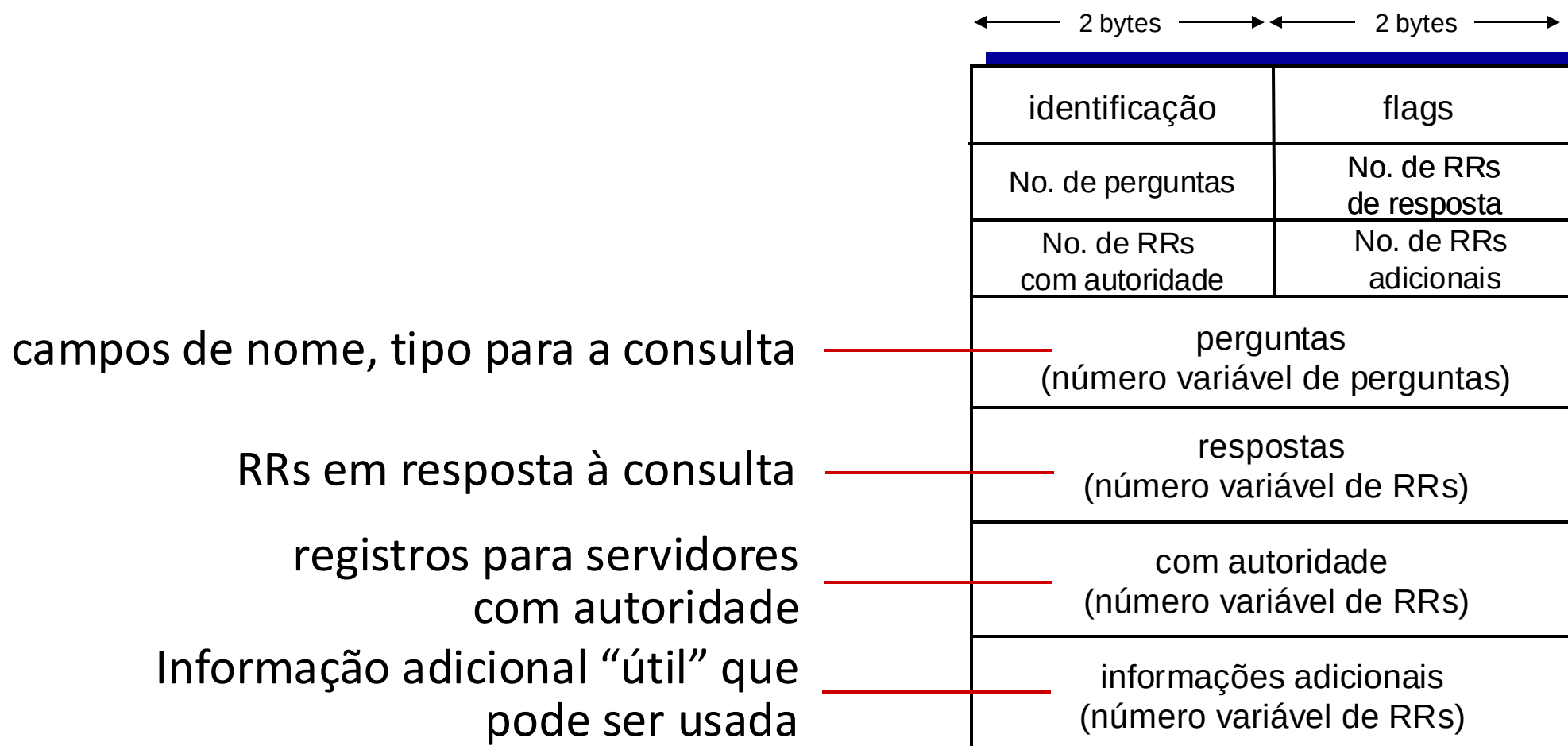
cabeçalho da mensagem:

- **identificação**: ID de 16 bits para pedido, resposta ao pedido usa mesmo ID
- **flags**:
  - pedido ou resposta
  - recursão desejada
  - recursão disponível
  - resposta é oficial



# Mensagens do protocolo DNS

As mensagens DNS de *consulta* e *resposta*, têm o mesmo *formato*:



# Inserindo registros no DNS

Exemplo: nova *startup* “Network Utopia”

- registra o nome networkutopia.com numa *entidade registradora DNS* (ex., Network Solutions)
  - provê nomes, endereços IP de seus servidores DNS com autoridade (primário e secundário)
  - registradora insere os RRs NS, A no servidor TLD .com:  
`(networkutopia.com, dns1.networkutopia.com, NS)`  
`(dns1.networkutopia.com, 212.212.212.1, A)`
- cria localmente o servidor com autoridade com endereço IP 212.212.212.1
  - registro tipo A para `www.networkutopia.com`
  - registro tipo MX para `networkutopia.com`



# Segurança DNS

## ataques DDoS

- bombardeia os servidores raiz com tráfego
  - até o momento não tiveram sucesso
  - filtragem do tráfego
  - servidores DNS locais cacheiam os IPs dos servidores TLD, permitindo que os servidores raízes não sejam consultados
- bombardeio a servidores TLD
  - potencialmente mais perigoso

## ataques de falsificação

- Intercepta as consultas DNS, retorna respostas falsas
  - envenenamento do DNS
  - RFC 4033: serviços de autenticação do DNSSEC

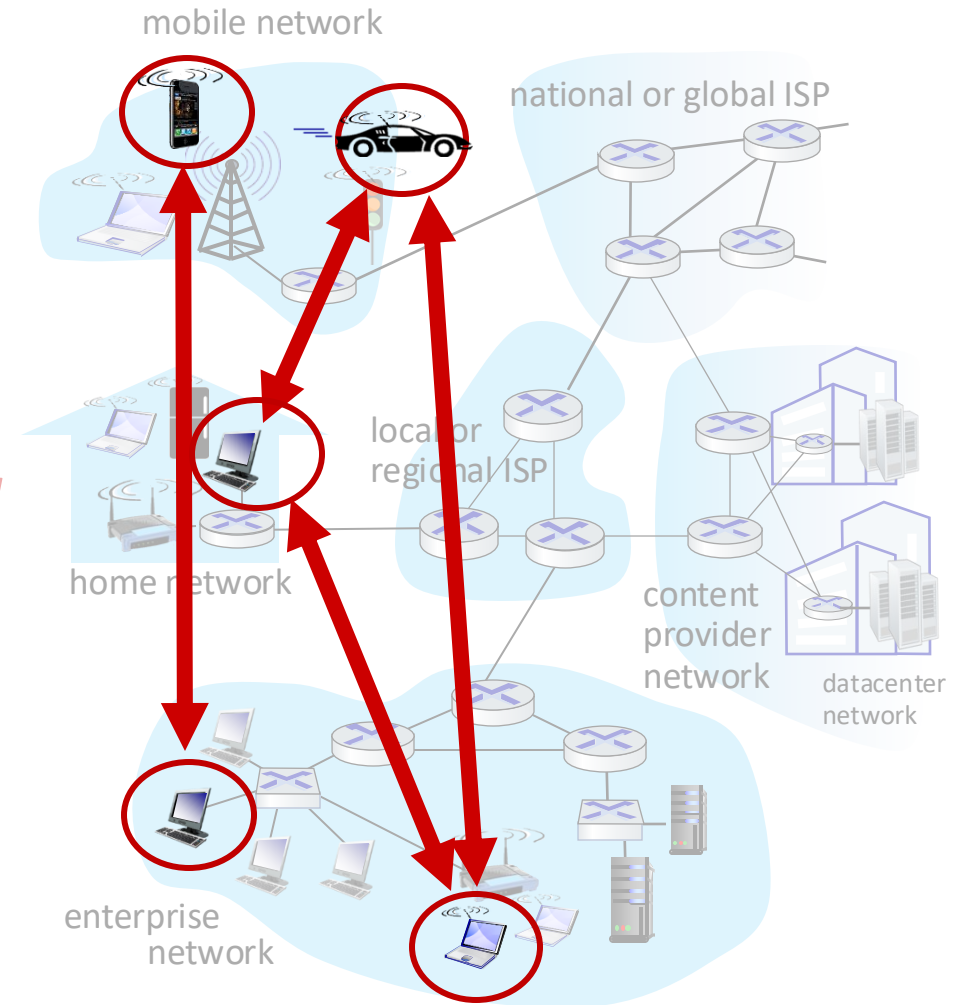
# Camada de Aplicação: roteiro

- Princípios de aplicações de rede
- A Web e o HTTP
- E-mail, SMTP, IMAP
- DNS: o serviço de diretório da Internet
- Aplicações P2P
- fluxos de vídeo e redes de distribuição de conteúdos
- programação de sockets com UDP e TCP



# Arquitetura peer-to-peer (P2P)

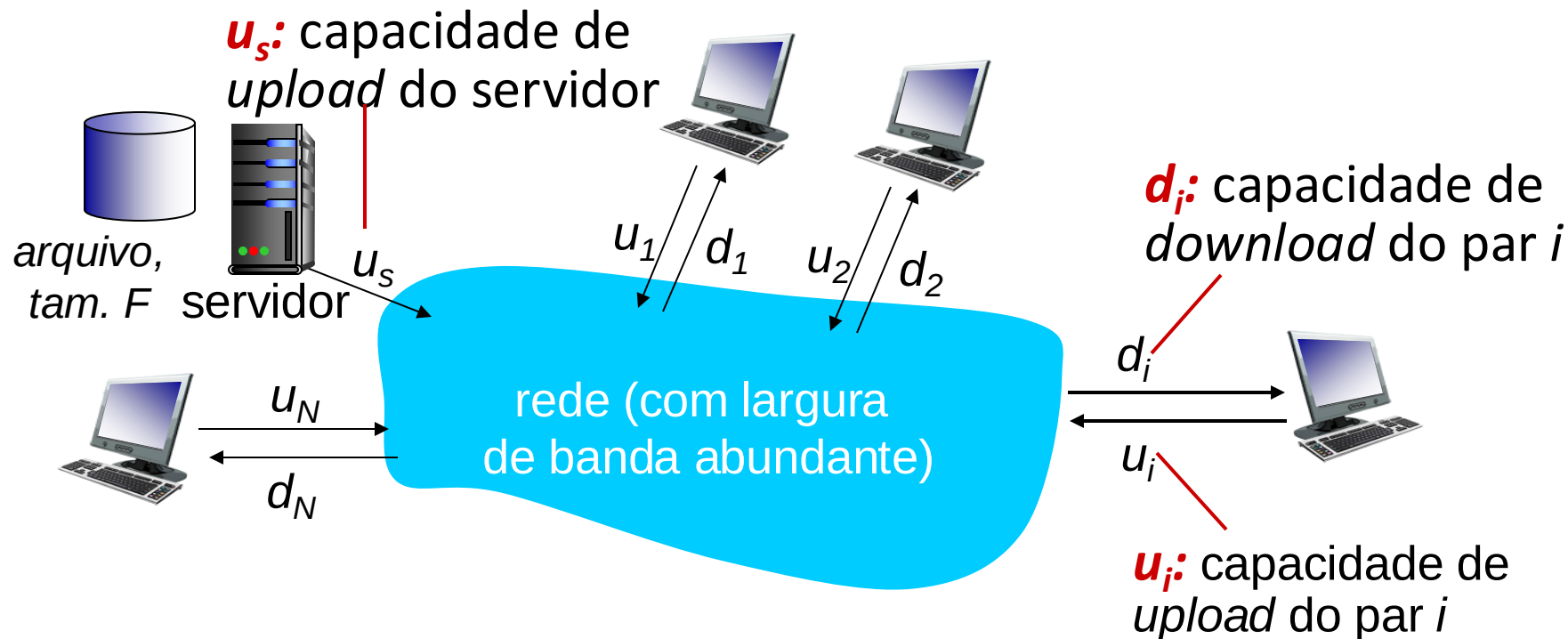
- *sem* servidor sempre ligado
- sistemas finais arbitrários se comunicam diretamente
- pares solicitam serviço de outros pares, em retribuição fornecem serviço a outros pares
  - *auto escalabilidade – novos pares trazem nova capacidade de serviço, e novas demandas por serviço*
- pares estão conectados de forma intermitente e mudam seus endereços IP
  - gerenciamento complexo
- exemplos: compartilhamento de arquivos P2P (BitTorrent), *streaming* (KanKan), VoIP (Skype)



# Distribuição de arquivo: cliente-servidor vs P2P

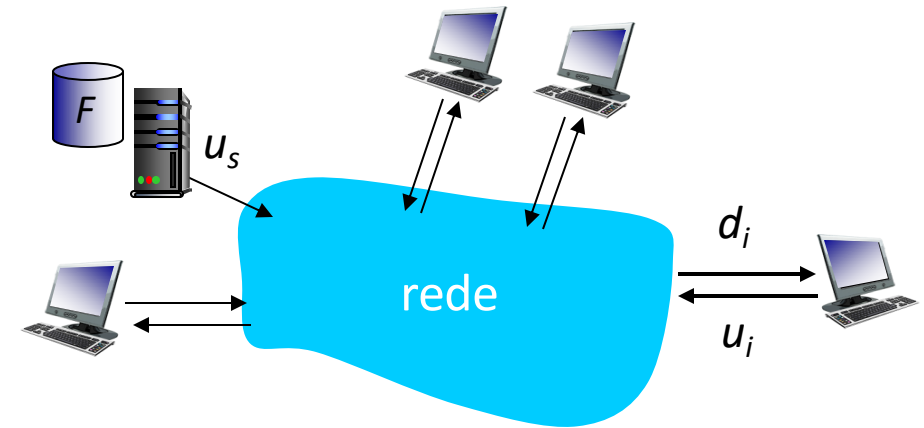
Q: quanto tempo leva para distribuir um arquivo (tamanho  $F$ ) de um servidor para  $N$  pares?

- capacidade de *upload/download* de um par é um recurso limitado



# Distribuição de arquivo: cliente-servidor

- **transmissão do servidor:** deve enviar em sequência (*upload*)  $N$  cópias do arquivo:
  - tempo para enviar uma cópia:  $F/u_s$
  - tempo para enviar  $N$  cópias:  $NF/u_s$
- **cliente:** cada cliente deve baixar (*download*) uma cópia do arquivo
  - $d_{min}$  = taxa mínima de *download*
  - tempo de *download* para o usuário com menor taxa:  $F/d_{min}$

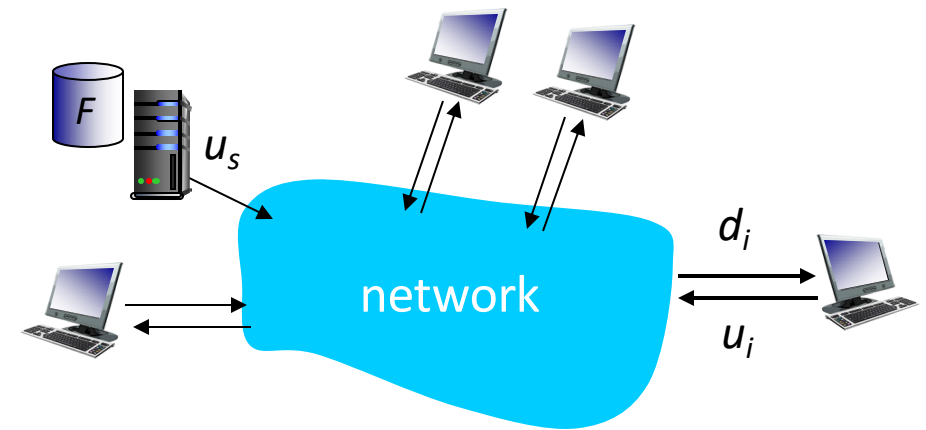


tempo para distribuir  $F$   
para  $N$  clientes usando  
abordagem cliente-servidor  $D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$

cresce linearmente com  $N$

# Distribuição de arquivo: P2P

- *transmissão do servidor*: deve enviar pelo menos uma cópia:
  - tempo para enviar uma cópia:  $F/u_s$
- *cliente*: cada cliente deve baixar uma cópia do arquivo
  - tempo de *download* para usuário com menor taxa:  $F/d_{min}$
- *clientes*: no total, devem baixar  $NF$  bits
  - taxa máx de *upload* (limita a taxa máxima de *download*) é  $u_s + \sum u_i$



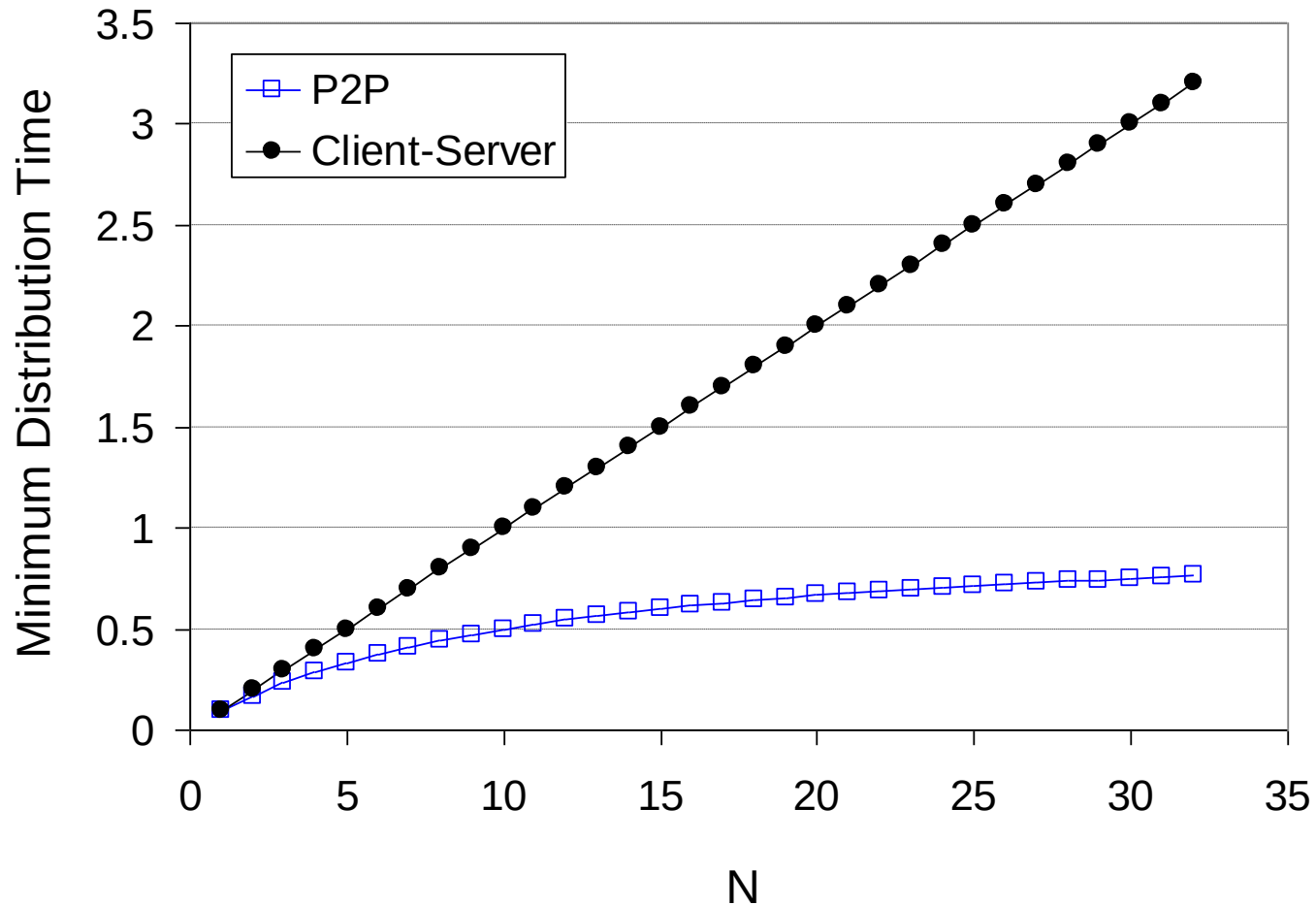
tempo para distribuir  $F$   
para  $N$  clientes usando  
abordagem P2P

$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

cresce linearmente com  $N$  ...  
... assim como este, cada par traz capacidade de serviço

# Cliente-servidor vs. P2P: exemplo

taxa de *upload* do cliente =  $u$ ,  $F/u = 1$  hora,  $u_s = 10u$ ,  $d_{min} \geq u_s$



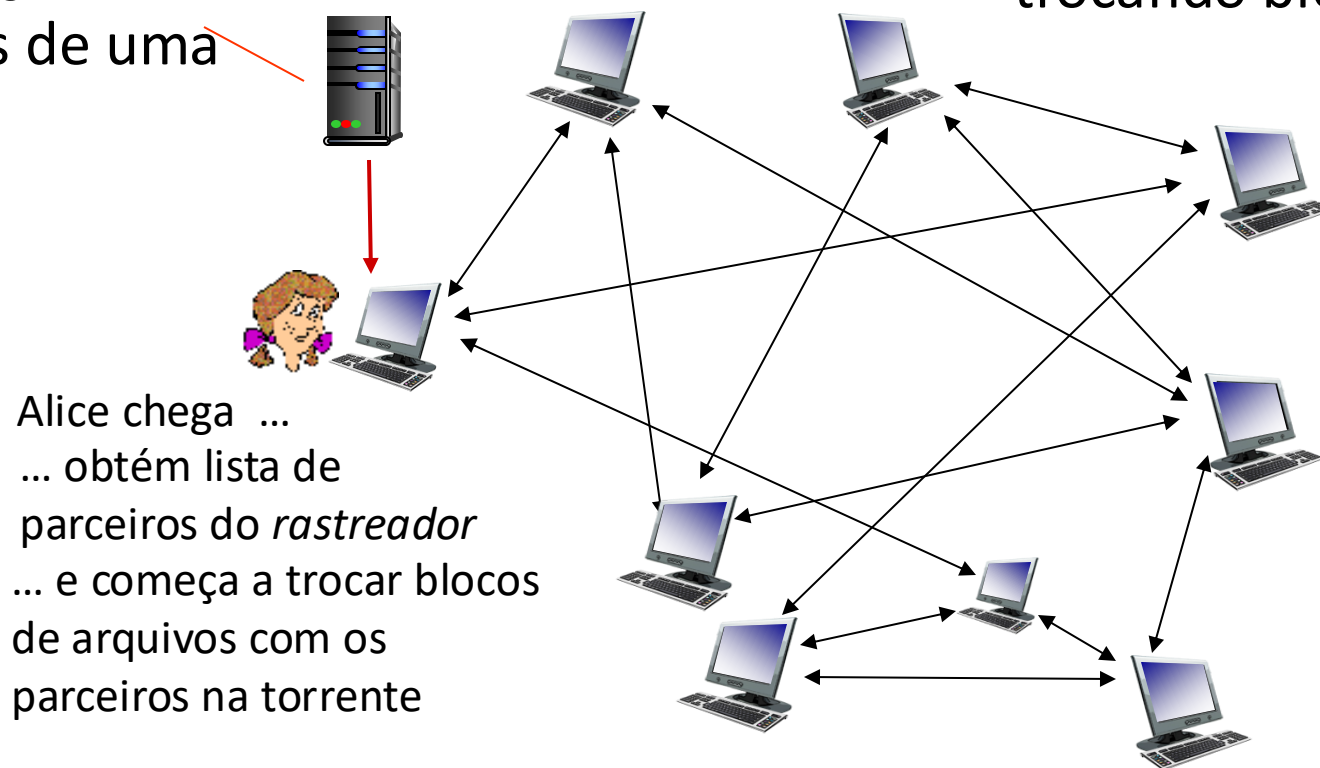
# Distribuição de arquivo P2P: BitTorrent

- arquivos divididos em blocos de 256kb
- Pares numa torrente enviam/recebem blocos do arquivo

*rastreador (tracker):*

registra pares  
participantes de uma  
torrente

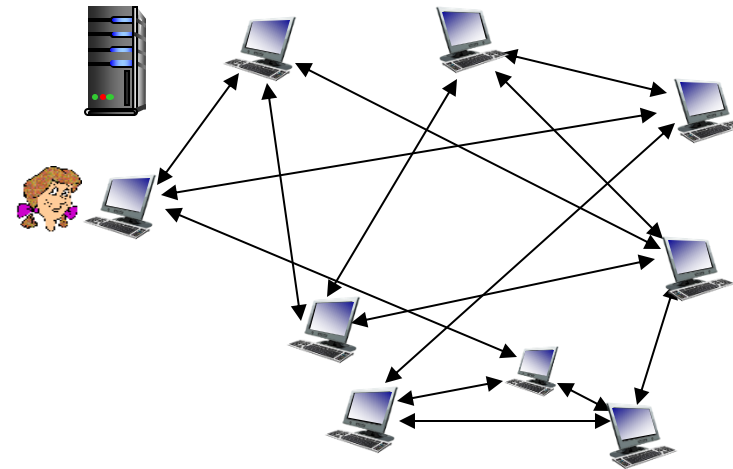
*torrente:* grupo de pares  
trocando blocos de um arquivo





# Distribuição de arquivo P2P: BitTorrent

- par que se une à torrente:
  - não tem nenhum bloco, mas irá acumulá-los com o tempo
  - registra com o *tracker* para obter lista dos pares, conecta a um subconjunto de pares (“vizinhos”)
- enquanto faz o download, par carrega blocos para outros pares
- par pode mudar os parceiros com os quais troca os blocos
- *churn*: pares podem entrar e sair
- quando o par obtiver todo o arquivo, ele pode (egoisticamente) sair ou permanecer (altruisticamente) na torrente



# BitTorrent: pedindo, enviando blocos de arquivos

## Pedindo blocos:

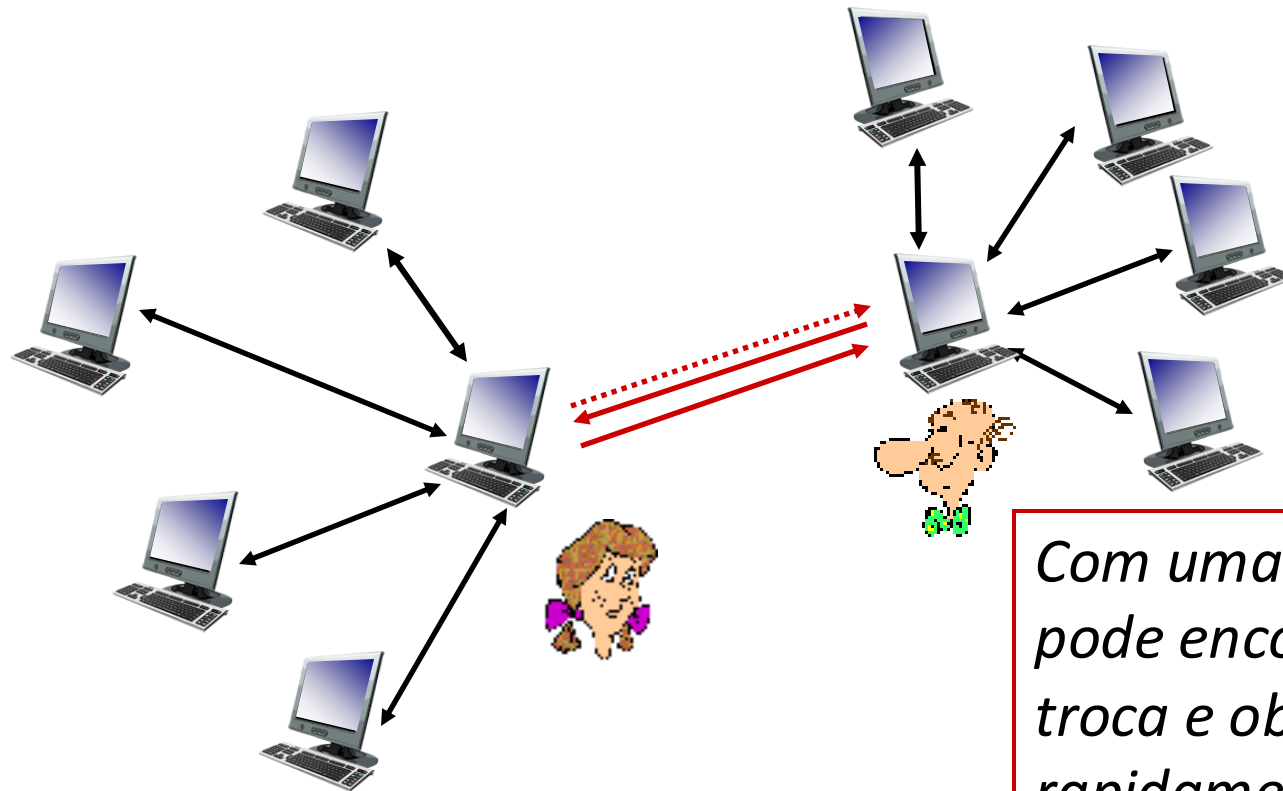
- num determinado instante, pares distintos possuem diferentes subconjuntos de blocos do arquivo
- periodicamente, um par (Alice) pede a cada vizinho a lista de blocos que eles possuem
- Alice envia pedidos para os pedaços que ainda não tem
  - começando pelos mais raros

## Enviando blocos: olho por olho!

- Alice envia blocos para os quatro vizinhos que estejam lhe enviando blocos *na taxa mais elevada*
  - outros pares foram sufocados por Alice (não recebem blocos dela)
  - reavalia os 4 mais a cada 10 segs
- a cada 30 segs: seleciona aleatoriamente outro par, começa a enviar-lhe blocos
  - “*optimistically unchoke*” este par
  - o par recém escolhido pode se unir aos 4 mais

# BitTorrent: toma lá, dá cá

- (1) Alice *“optimistically unchokes”* Bob
- (2) Alice se torna um dos quatro melhores provedores de Bob; Bob retribui
- (3) Bob se torna um dos quatro melhores provedores de Alice



*Com uma taxa de upload mais alta,  
pode encontrar melhores parceiros de  
troca e obter o arquivo mais  
rapidamente!*

# Camada de Aplicação: roteiro

- Princípios de aplicações de rede
- A Web e o HTTP
- E-mail, SMTP, IMAP
- DNS: o serviço de diretório da Internet
- Aplicações P2P
- fluxos de vídeo e redes de distribuição de conteúdos
- programação de sockets com UDP e TCP



# Streaming de vídeo e CDNs: contexto

- tráfego de *stream* de vídeo: maior consumidor de largura de banda da Internet
  - Netflix, YouTube, Amazon Prime: 80% do tráfego de ISPs residenciais (2020)
- **desafio:** escala – como alcançar ~1B usuários?
  - um único mega vídeo server não daria conta (por quê?)
- **desafio:** heterogeneidade
  - usuários diferentes têm diferentes características (ex.: cabeado x móvel; boa x ruim largura de banda)
- **solução:** *infraestrutura distribuída de camada de aplicação*

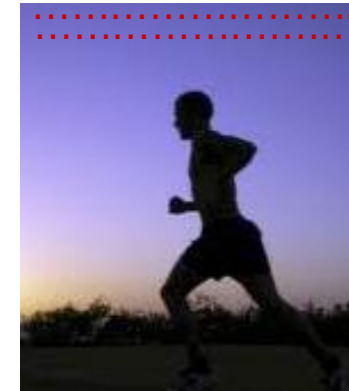


# Multimídia: vídeo

- vídeo: sequência de imagens apresentadas a uma taxa constante
  - e.g., 24 imagens/seg
- imagem digital: matriz de pixels
  - cada pixel representado por bits
- codificação: usa redundância **dentro** e **entre** imagens para diminuir # bits usados para codificar a imagem
  - espacial (dentro da imagem)
  - temporal (de uma imagem para a próxima)

*exemplo de codificação*

*espacial:* ao invés de enviar N valores com a mesma cor (roxo), envia apenas dois valores: valor da cor (roxo) e número (N) de valores repetidos (N)



quadro *i*

*exemplo de codificação*

*temporal:* ao invés de enviar o quadro completo  $i+1$ , envia apenas as diferenças do quadro  $i$

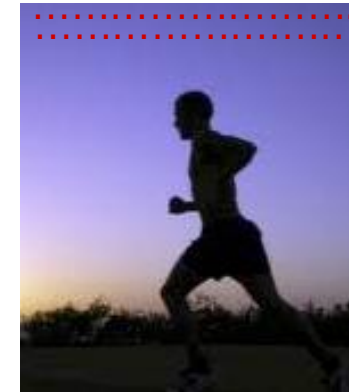


quadro *i+1*

# Multimídia: vídeo

- **CBR (*constant bit rate*):** codificação de vídeo a uma taxa constante
- **VBR (*variable bit rate*):** taxa de codificação de vídeo muda com a necessidade/redundância espacial ou temporal.
- **exemplos:**
  - MPEG 1 (CD-ROM) 1,5 Mbps
  - MPEG2 (DVD) 3-6 Mbps
  - MPEG4 (frequentemente usado na Internet, 64Kbps – 12 Mbps)

*exemplo de codificação espacial:* ao invés de enviar N valores com a mesma cor (roxo), envia apenas dois valores: valor da cor (roxo) e número (N) de valores repetidos (N)



quadro *i*

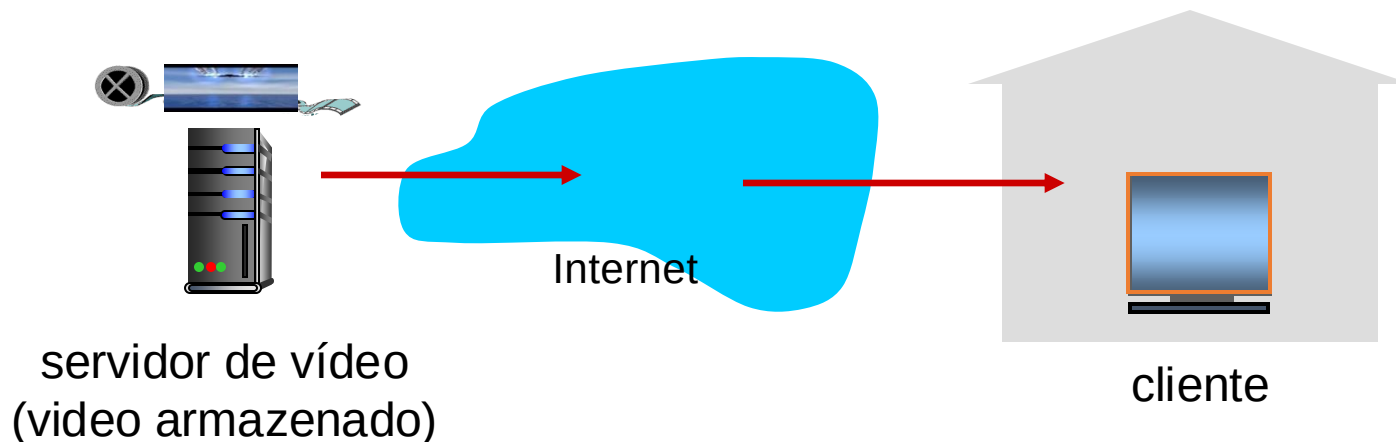
*exemplo de codificação temporal:* ao invés de enviar o quadro completo *i+1*, envia apenas as diferenças do quadro *i*



quadro *i+1*

# Streaming de vídeo armazenado

cenário simples:

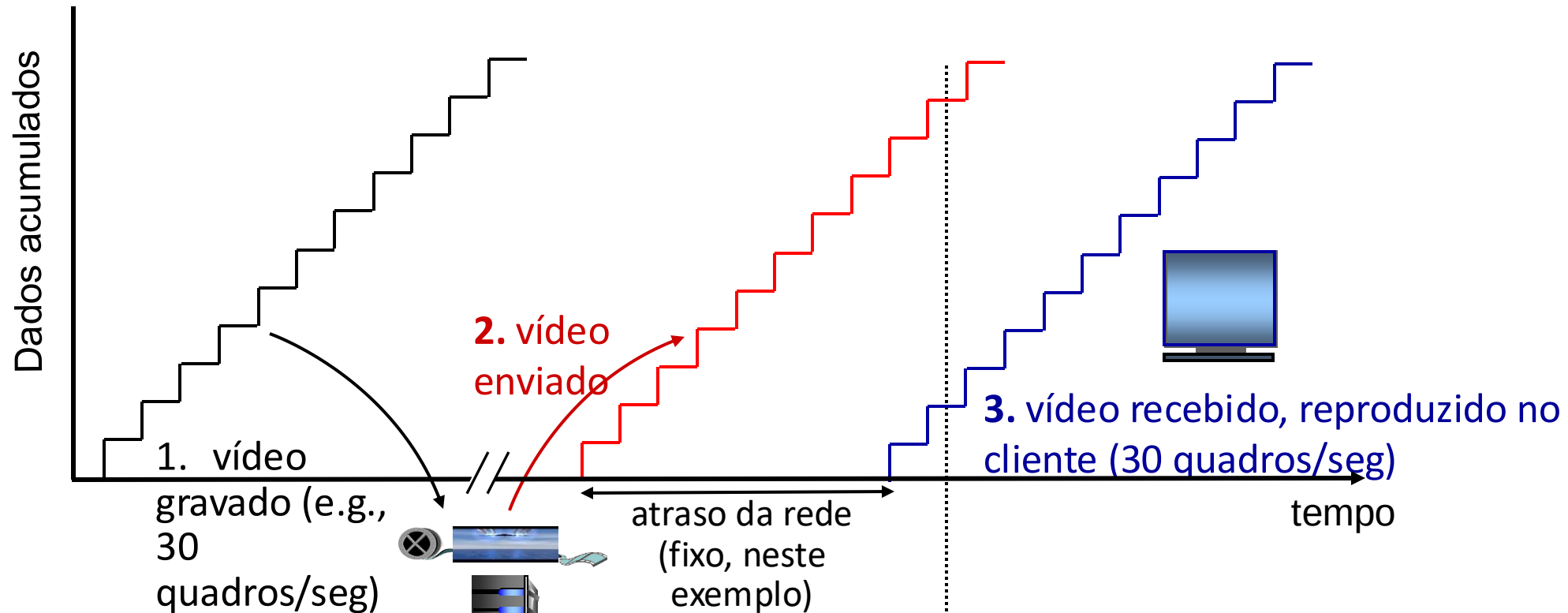


## Principais desafios:

- a largura de banda do servidor para o cliente irá *variar* com a mudança dos níveis de congestionamento de rede (na casa, na rede de acesso, no núcleo da rede, no servidor de vídeo)
- perda de pacotes e atraso devidos ao congestionamento irão atrasar a reprodução ou resultar numa qualidade de vídeo pobre.



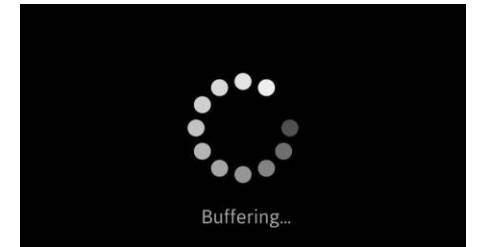
# Streaming de vídeo armazenado



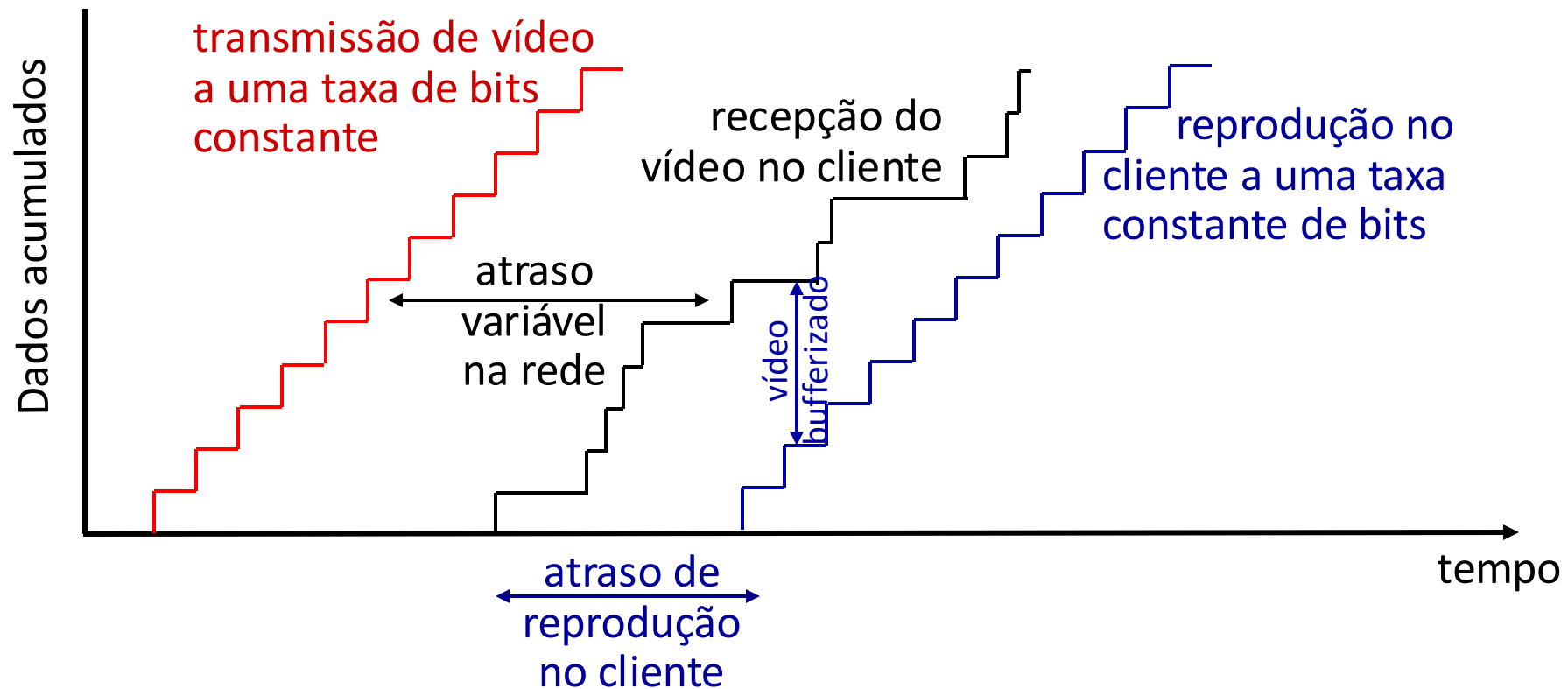
**streaming**: neste instante, o cliente está reproduzindo um trecho inicial do vídeo, enquanto o servidor ainda está enviando um trecho posterior do vídeo

# Streaming de vídeo armazenado: desafios

- **restrição de reprodução contínua**: uma vez iniciada a reprodução, ela deve casar com os tempos originais
  - ... mas, **os atrasos de rede são variáveis** (*jitter*), necessita **buffer no lado cliente** para atingir os objetivos de reprodução
- outros desafios:
  - interatividade do cliente: pausar, avançar rapidamente, voltar, saltar
  - pacotes de vídeo podem se perder, podem ser retransmitidos



# Streaming de vídeo armazenado: bufferização de reprodução



- *bufferização no lado do cliente e atraso de reprodução:*  
compensa o atraso adicionado pela rede, variação do atraso (*jitter*)

# Streaming multimídia: DASH

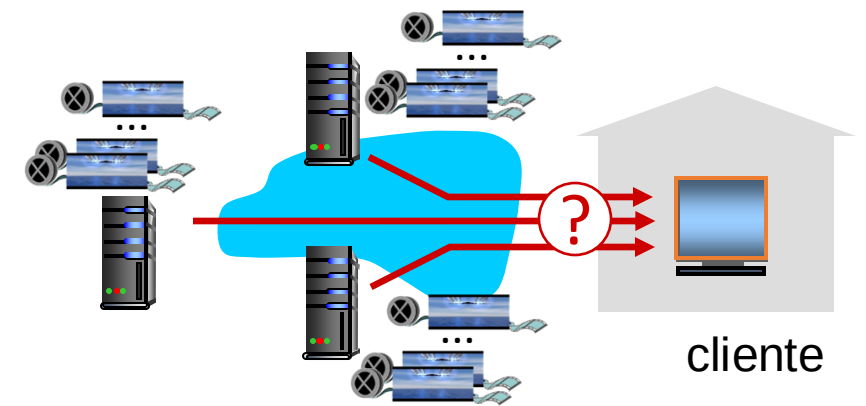
*D*ynamic, *A*daptive  
Streaming over *H*TTP

## ■ *servidor:*

- divide o arquivo de vídeo em diversos pedaços (*chunks*)
- cada pedaço é armazenado codificado em diferentes taxas
- codificações em taxas diferentes são armazenadas em arquivos diferentes
- arquivos replicados em diversos nós CDN
- **arquivo de manifesto**: provê URLs para os diferentes pedaços

## ■ *cliente:*

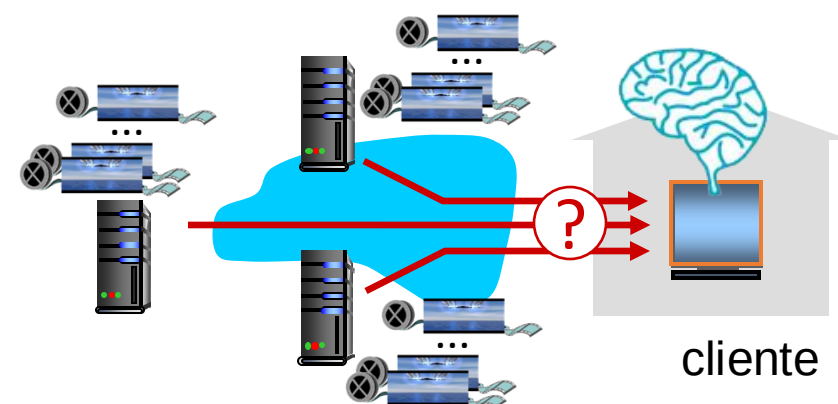
- mede periodicamente a banda entre servidor e cliente
- consulta manifesto, solicita um pedaço por vez
  - escolhe a taxa máxima suportada pela largura de banda atual
  - pode escolher diferentes taxas de codificação em instantes diferentes (dependendo a banda disponível no momento)



# Streaming multimídia: DASH

- “*inteligência*” no cliente: o cliente determina

- *quando* solicitar um pedaço (de modo a não haver nem esvaziamento nem estouro do buffer)
- *que taxa de codificação* solicitar (maior qualidade quando houver mais banda disponível)
- *de onde* solicitar o pedaço (pode solicitar do servidor URL que estiver mais “próximo” do cliente ou que tenha a maior banda disponível)



*Streaming vídeo* = codificação + DASH + *bufferização*  
para reprodução

# Redes de distribuição de conteúdo (CDNs)

- *CDNs* = *Content distribution networks*
- *desafio*: como enviar conteúdo (selecionado de milhões de vídeos) para centenas de milhares de usuários simultâneos?
- *opção 1*: grande “mega-servidor” único
  - ponto único de falha
  - ponto de congestionamento de rede
  - caminho longo (e possivelmente congestionado) para clientes distantes

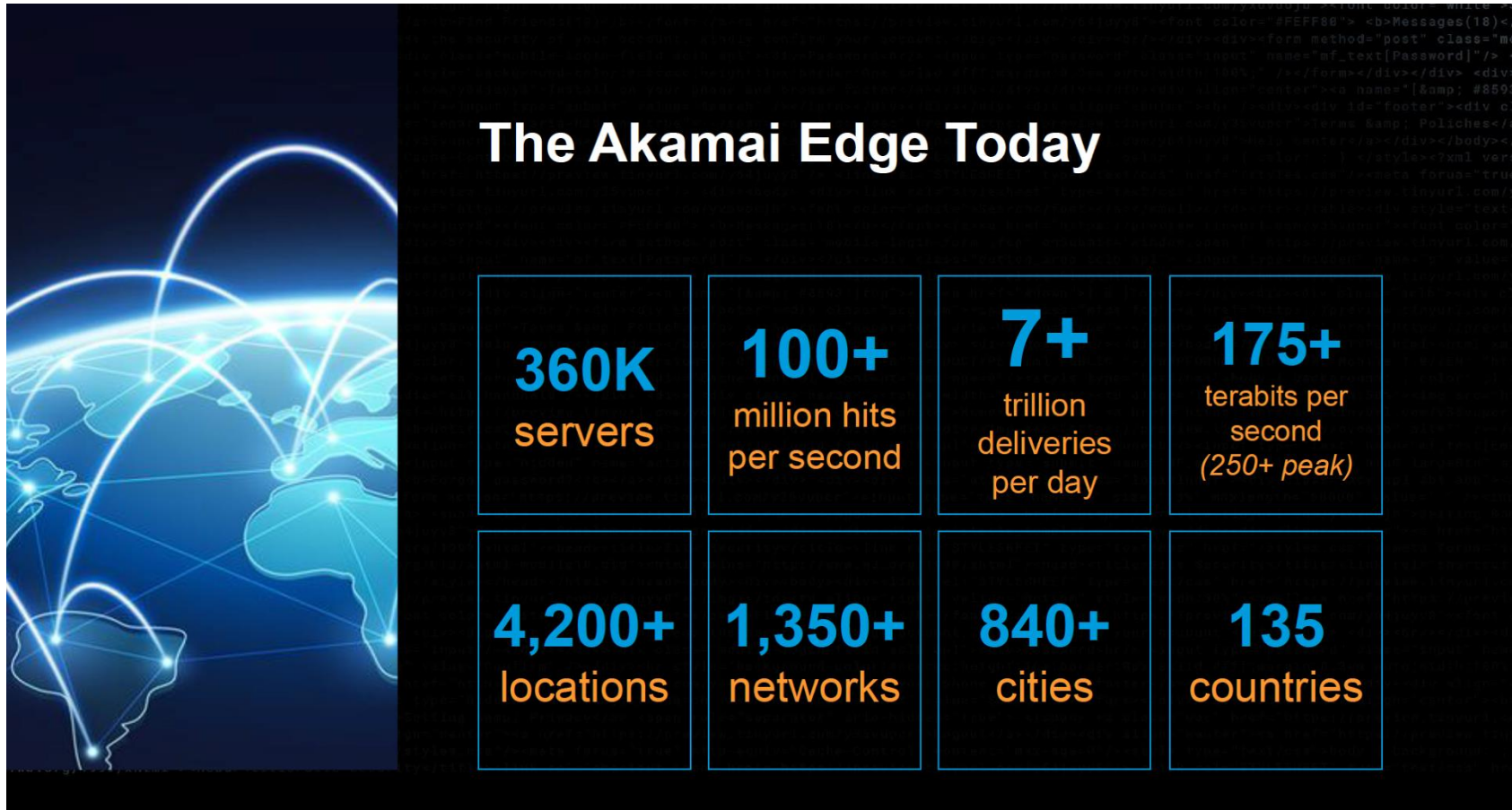
... simplesmente: esta solução *não escala*

# Redes de distribuição de conteúdo (CDNs)

- *desafio*: como enviar conteúdo (selecionado de milhões de vídeos) para centenas de milhares de usuários simultâneos?
- *opção 2*: armazenar/disponibilizar múltiplas cópias do vídeo em sites distribuídos geograficamente (*CDN*)
  - *ir fundo*: colocar servidores CDN em muitas redes de acesso
    - próximo aos usuários
    - Akamai, 240.000 servidores instalados em mais de 120 países (2015)
  - *levar para casa*: menor número (10's) de grandes *clusters* em POPs próximos (mas não dentro) das redes de acesso
    - usado pela Limelight



# Akamai hoje:

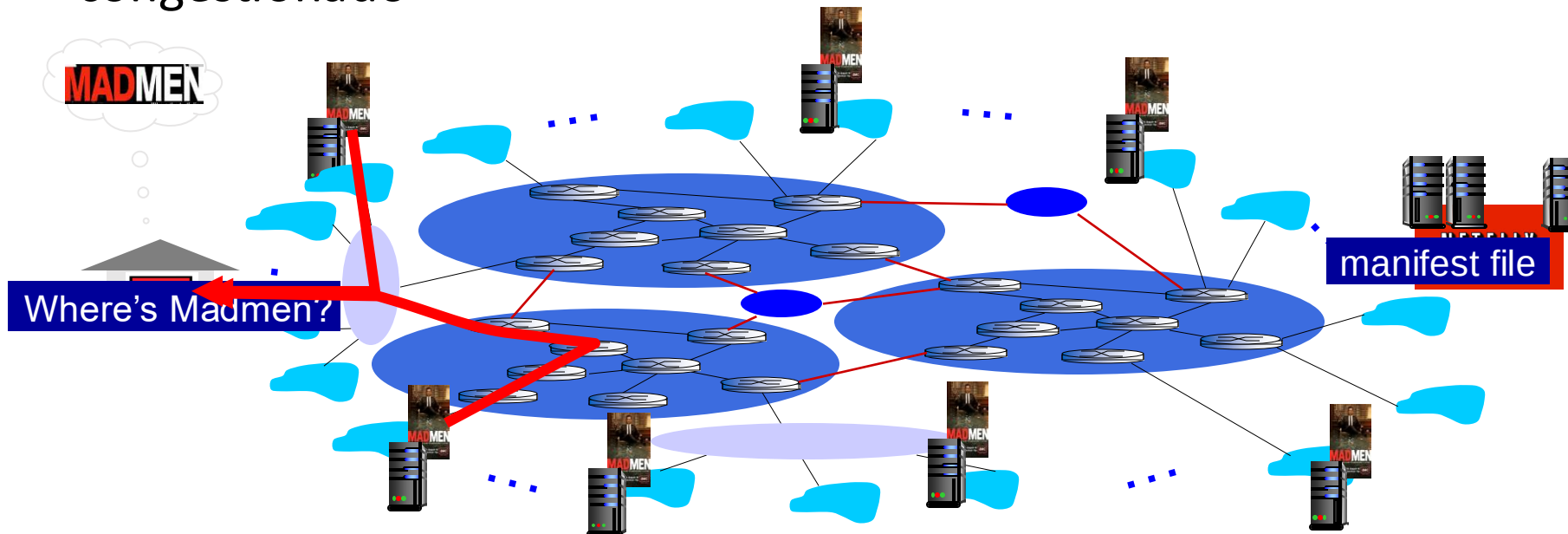


Source: <https://networkingchannel.eu/living-on-the-edge-for-a-quarter-century-an-akamai-retrospective-downloads/>



# Como funciona a Netflix?

- Netflix: armazena cópias do conteúdo (ex. MADMEN) em seus nós CDN OpenConnect (em todo o mundo)
- assinante solicita conteúdo, provedor de serviço retorna o manifesto
  - usando o manifesto, cliente recupera conteúdo na taxa máxima viável
  - pode escolher outra taxa ou cópia, se o caminho estiver congestionado



# Redes de distribuição de conteúdo (CDNs)



*desafios do OTT (Over The Top):* convivendo com uma Internet congestionada a partir da borda

- que conteúdo colocar em cada nó CDN?
- de qual nó CDN se deve recuperar o conteúdo? A que taxa?

# Camada de Aplicação: roteiro

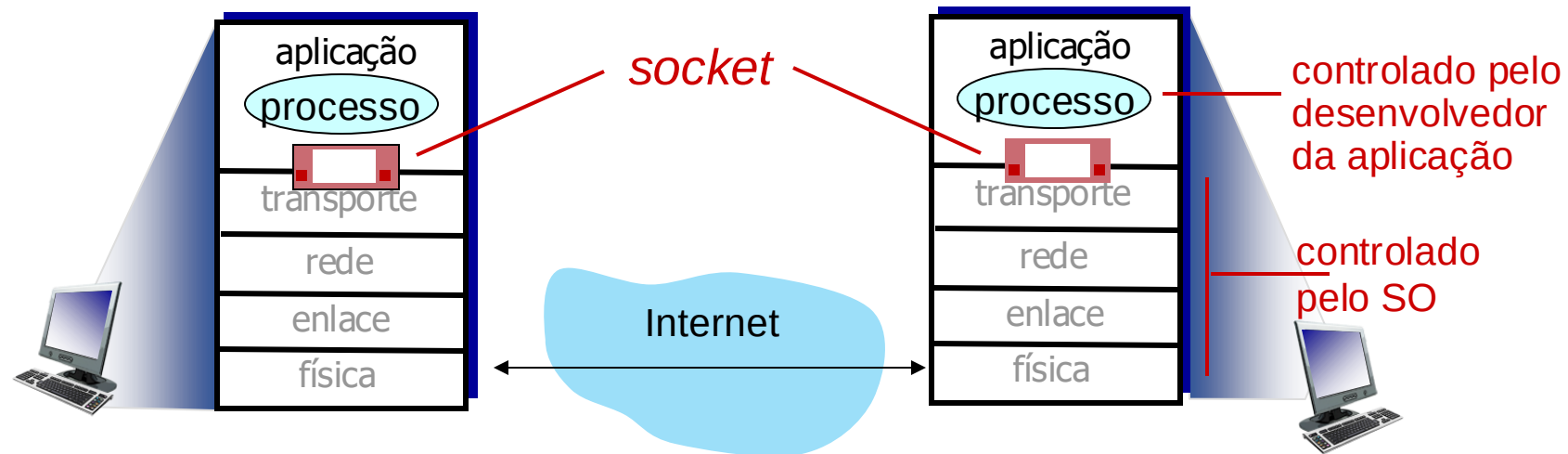
- Princípios de aplicações de rede
- A Web e o HTTP
- E-mail, SMTP, IMAP
- DNS: o serviço de diretório da Internet
- Aplicações P2P
- fluxos de vídeo e redes de distribuição de conteúdos
- programação de sockets com UDP e TCP



# Programação com *sockets*

*meta*: aprender a construir aplicações cliente/servidor que se comunicam usando sockets

*socket*: porta entre o processo de aplicação e o protocolo de transporte fim-a-fim



# Programação com *sockets*

Dois tipos de sockets para dois serviços de transporte:

- *UDP*: datagrama não confiável
- *TCP*: confiável, orientado a fluxos de bytes

Exemplo de aplicação:

1. o cliente lê uma linha de caracteres (dados) do seu teclado e envia os dados para o servidor
2. o servidor recebe os dados e converte os caracteres para maiúsculas
3. o servidor envia os dados modificados para o cliente
4. o cliente recebe os dados modificados e apresenta a linha na sua tela

# Programação com *sockets* com UDP

**UDP:** não tem “conexão” entre cliente e servidor

- não tem saudação (“*handshaking*”) antes de enviar os dados
- remetente coloca explicitamente endereço IP e porta do destino em cada pacote
- receptor deve extrair endereço IP e número da porta do remetente do datagrama recebido

**UDP:** dados transmitidos podem ser perdidos ou recebidos fora de ordem

**Ponto de vista da aplicação:**

- UDP provê transferência *não confiável* de grupos de bytes (“datagramas”) entre cliente e servidor

# Interação entre sockets cliente/servidor: UDP



**servidor** (rodando em serverIP)

cria socket, porta= x:  
`serverSocket =  
socket(AF_INET,SOCK_DGRAM)`

↓  
lê datagrama do  
`serverSocket`

↓  
escreve resposta  
no `serverSocket`  
especificando  
endereço,  
número de porta  
do cliente

**cliente**



cria socket:  
`clientSocket =  
socket(AF_INET,SOCK_DGRAM)`

↓  
Cria datagrama com IP e porta=x do  
servidor; envia o datagrama através do  
`clientSocket`

↓  
lê datagrama do  
`clientSocket`

↓  
fecha o  
`clientSocket`

# Aplicação exemplo: cliente UDP

## *UDPClient em Python*

|                                                                               |   |                                                                                              |
|-------------------------------------------------------------------------------|---|----------------------------------------------------------------------------------------------|
| inclui a biblioteca de sockets do Python                                      | → | <code>from socket import *</code>                                                            |
|                                                                               |   | <code>serverName = 'hostname'</code>                                                         |
|                                                                               |   | <code>serverPort = 12000</code>                                                              |
| cria socket UDP para o servidor                                               | → | <code>clientSocket = socket(AF_INET,</code><br><code>SOCK_DGRAM)</code>                      |
| obtem entrada do teclado do usuário                                           | → | <code>message = input('Input lowercase sentence:')</code>                                    |
| acrescenta o nome do servidor e número da porta à mensagem; envia pelo socket | → | <code>clientSocket.sendto(message.encode(),</code><br><code>(serverName, serverPort))</code> |
| lê caracteres de resposta do socket e converte em <i>string</i>               | → | <code>modifiedMessage, serverAddress =</code><br><code>clientSocket.recvfrom(2048)</code>    |
| imprime <i>string</i> recebido e fecha socket                                 | → | <code>Print(modifiedMessage.decode())</code><br><code>clientSocket.close()</code>            |



# Aplicação exemplo: servidor UDP

## *UDPServer em Python*

```
from socket import *
serverPort = 12000
cria socket UDP → serverSocket = socket(AF_INET, SOCK_DGRAM)
liga socket à porta local número 12000 → serverSocket.bind(('', serverPort))
print('The server is ready to receive')
loop infinito → while True:
 lê mensagem do socket UDP, obtendo endereço do cliente (IP e porta do cliente) → message, clientAddress = serverSocket.recvfrom(2048)
 modifiedMessage = message.decode().upper()
 retorna string em maiúsculas para este cliente → serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

# Programação com *sockets* com TCP

## Cliente deve contactar servidor

- processo servidor deve antes estar em execução
- servidor deve antes ter criado socket (porta) que aguarda contato do cliente

## Cliente contacta servidor:

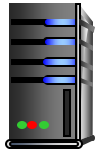
- cria socket TCP local ao cliente, especificando endereço IP, número de porta do processo servidor
- quando **cliente cria socket**: TCP cliente cria conexão com TCP do servidor

- quando contactado pelo cliente, o **TCP do servidor cria um novo socket** para que o processo servidor possa se comunicar com aquele determinado cliente
  - permite que o servidor converse com múltiplos clientes
  - Endereço IP e porta origem são usados para distinguir os clientes (mais no cap. 3)

## Ponto de vista da aplicação

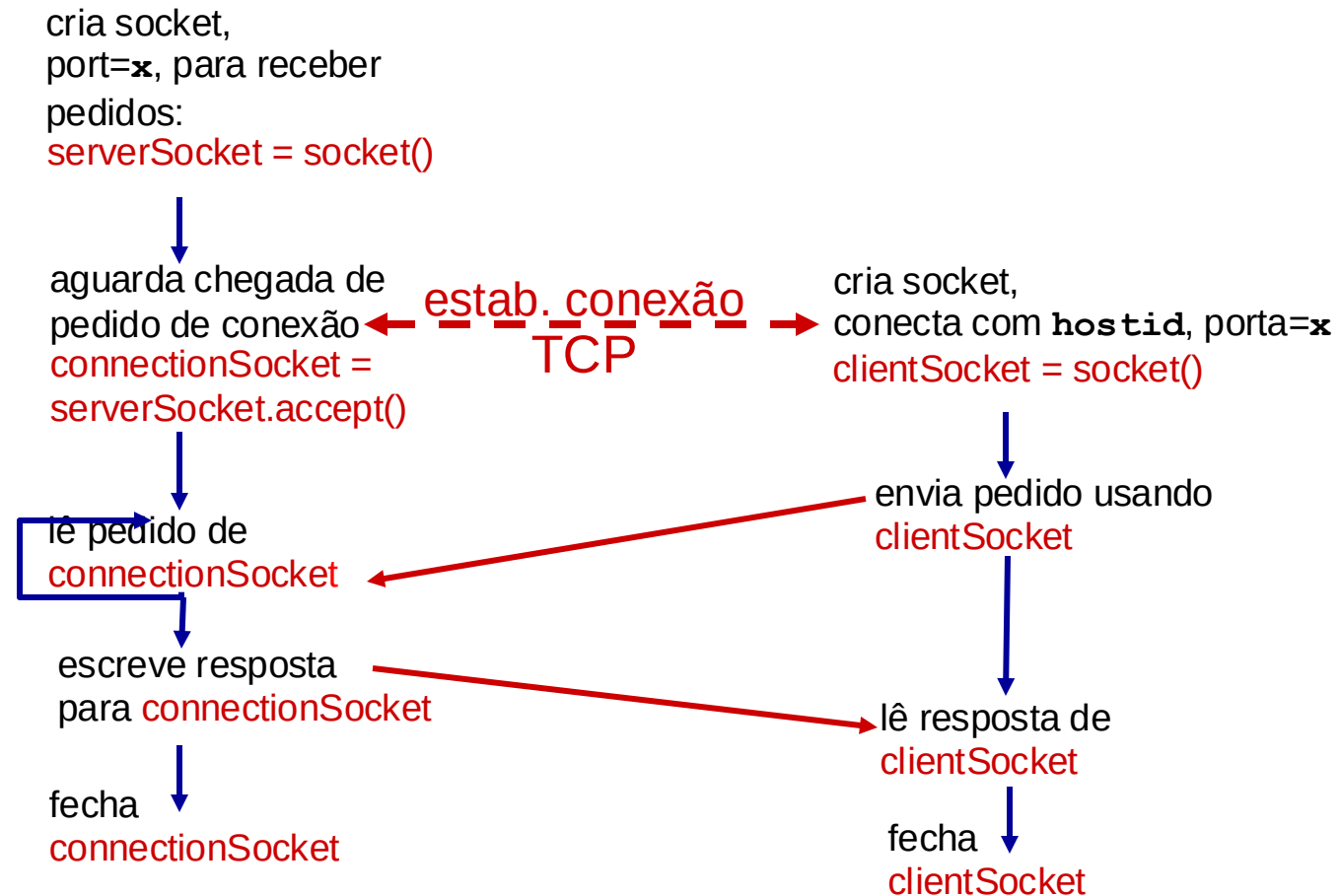
TCP provê transferência confiável, ordenada de bytes (“tubo”) entre processos cliente e servidor

# Interação entre sockets cliente/servidor: TCP



servidor (rodando em hostid)

cliente



# Exemplo: cliente TCP

## *TCPClient em Python*

cria socket TCP para o servidor, porta remota 12000



não há necessidade de especificar nem o nome do servidor nem a porta



```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

# Exemplo: servidor TCP

## *TCPServer em Python*

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
Print('The server is ready to receive')
while True:
 connectionSocket, addr = serverSocket.accept()

 sentence = connectionSocket.recv(1024).decode()
 capitalizedSentence = sentence.upper()
 connectionSocket.send(capitalizedSentence.
 encode())

 connectionSocket.close()
```

cria socket TCP de recepção →

servidor inicia a escuta por  
solicitações TCP →

loop infinito →

servidor espera no accept() por solicitações,  
um novo socket é criado no retorno →

lê bytes do socket (mas não precisa  
ler endereço como no UDP) →

fecha conexão para este cliente (mas  
não o socket de recepção) →

# Capítulo 2: Resumo

nosso estudo sobre aplicações de rede está agora completo!

- arquiteturas de aplicações
  - cliente-servidor
  - P2P
- requisitos de serviço das aplicações:
  - confiabilidade, largura de banda, atraso
- modelos de serviço de transporte da Internet
  - orientado a conexões, confiável: TCP
  - não confiável, datagramas: UDP
- protocolos específicos:
  - HTTP
  - SMTP, IMAP
  - DNS
  - P2P: BitTorrent
- *streaming* de vídeo, CDNs
- programação com sockets: sockets TCP, UDP

# Capítulo 2: Resumo

Mais importante: aprendemos sobre *protocolos*!

- troca típica de mensagens pedido/resposta:
  - cliente solicita info ou serviço
  - servidor responde com dados, código de *status*
- formatos de mensagens:
  - **cabeçalhos**: campos com info sobre dados
  - **dados**: info (carga) sendo comunicada

temas importantes:

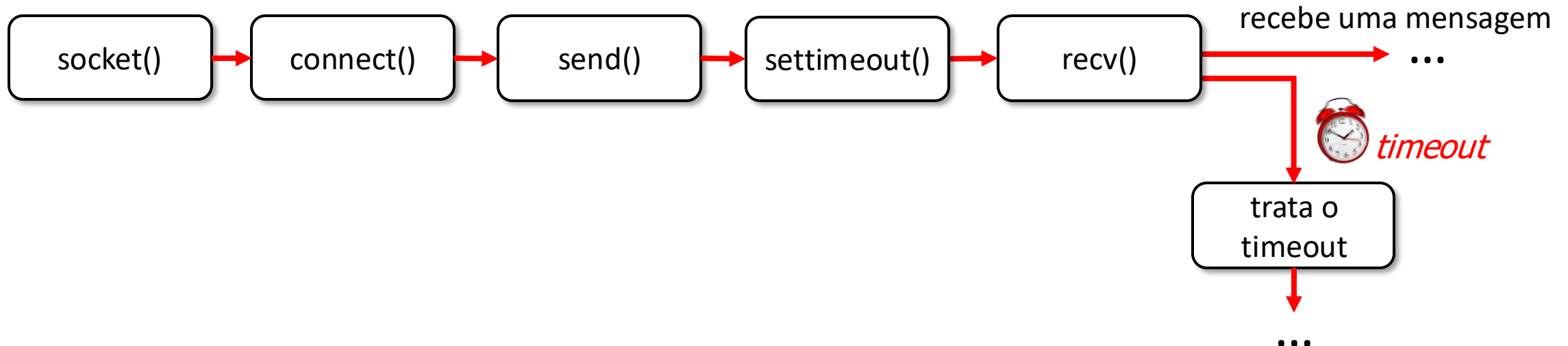
- centralizado vs. descentralizado
- sem estado vs. com estado
- escalabilidade
- transferência de mensagens confiável vs. não confiável
- “complexidade na borda da rede”

# Slides adicionais Capítulo 2

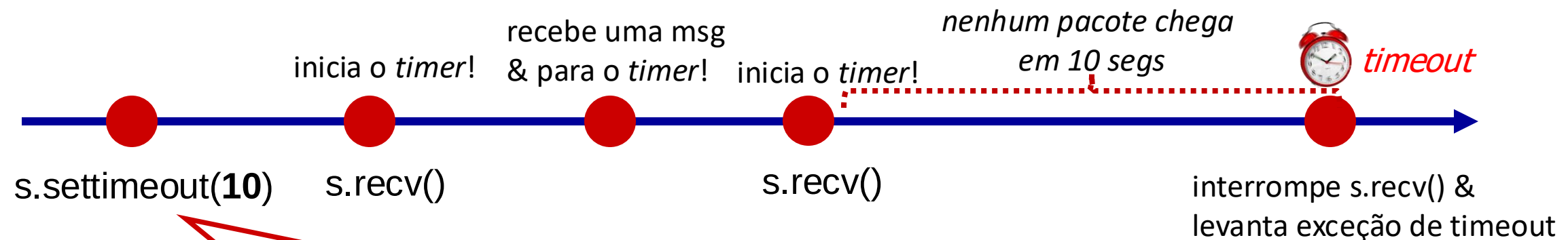
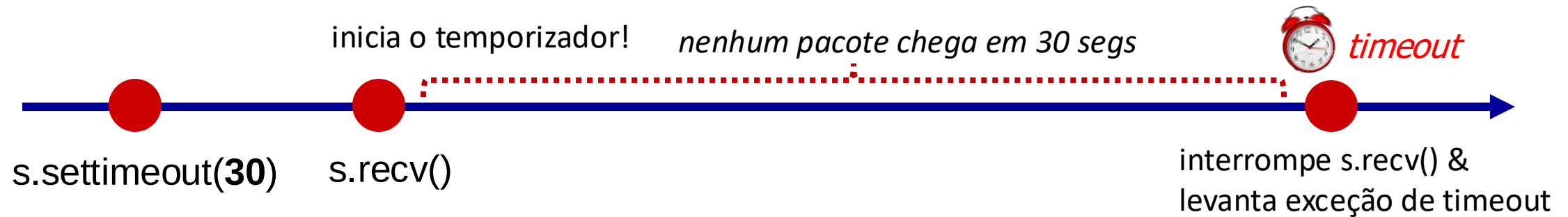


# Programação com Sockets: esperando por múltiplos eventos

- algumas vezes um programa deve **aguardar que um dentre diversos eventos** ocorra:
  - aguarda por (i) uma resposta do outro lado de um socket, ou (ii) estouro de um temporizador
  - aguarda por respostas de diferentes sockets abertos: `select()`, `multithreading`
- temporizadores são muito usados em redes
- usando temporizadores com socket em Python:



# Como funciona o `socket.settimeout()` do Python?



Configura o temporizador em todas as operações futuras de um socket específico!

# Bloco try-except do Python

Executa um bloco de código, e trata “exceções” que podem ocorrer ao executar aquele bloco de código

try:

<faça algo>

except <exceção>:

<trata a exceção>

{ Executando este **bloco de código try** pode causar a ocorrência de exceção(ões). Se ocorrer uma exceção, a execução salta diretamente para o **bloco de código except**

{ este **bloco de código except** é executado apenas se ocorrer uma **<exceção>** no **bloco de código try** (nota: é *obrigatório* um bloco except com um bloco try)

# Programação com sockets: **socket timeouts**

## Exemplo:



- Um jovem pastor cuida do rebanho de seu patrão
- Se ele vir um lobo, ele pode enviar um pedido de socorro para os aldeões usando um socket TCP.
- O jovem acha engraçado conectar ao servidor sem enviar nenhuma mensagem. Mas, os aldeões não acham o mesmo.
- Eles decidiram que se o jovem se conectar ao servidor e não enviar a localização do lobo **dentro de 10 segundos por três vezes**, eles **deixarão de escutá-lo**.

## *Python TCPServer (Aldeões)*

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
counter = 0
while counter < 3:
 connectionSocket, addr = serverSocket.accept()
 connectionSocket.settimeout(10)
 try:
 wolf_location = connectionSocket.recv(1024).decode()
 send_hunter(wolf_location) # a villager function
 connectionSocket.send('hunter sent')
 except timeout:
 counter += 1
 connectionSocket.close()
```

ajusta o temporizador para 10-segundos em todas as futuras operações com o socket. →

timer é iniciado ao chamar recv() e causará uma exceção de timeout se não receber nenhuma mensagem dentro de 10 segundos. →

executa a exceção de timeout do socket →

# Interação SMTP típica

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

# Lista Parcial de TLDs (1.470 no total)

|          |             |          |           |             |
|----------|-------------|----------|-----------|-------------|
| AAA      | ACCENTURE   | AETNA    | AIRTEL    | BOOK        |
| AARP     | ACCOUNTANT  | AF       | AKDN      | BOOKING     |
| ABB      | ACCOUNTANTS | AFL      | AL        | BOSCH       |
| ABBOTT   | ACO         | AFRICA   | ALIBABA   | BOSTIK      |
| ABBVIE   | ACTOR       | AG       | ALIPAY    | BOSTON      |
| ABC      | AD          | AGAKHAN  | ALLFINANZ | BOT         |
| ABLE     | ADS         | AGENCY   | ALLSTATE  | BOUTIQUE    |
| ABOGADO  | ADULT       | AI       | ALLY      | BOX         |
| ABUDHABI | AE          | AIG      | ALSACE    | BR          |
| AC       | AEG         | AIRBUS   | ALSTOM    | BRADESCO    |
| ACADEMY  | AERO        | AIRFORCE | AM        | BRIDGESTONE |

# Domínios .BR por Categorias



## GENÉRICOS

Total **4.930.616** 95,54%

» [Ver evolução](#)

| CATEGORIAS    | QUANTIDADE | %      |
|---------------|------------|--------|
| <b>APP.BR</b> | 15.929     | 0,31%  |
| <b>ART.BR</b> | 10.563     | 0,20%  |
| <b>COM.BR</b> | 4.794.481  | 92,90% |
| <b>DEV.BR</b> | 6.876      | 0,13%  |
| <b>ECO.BR</b> | 8.819      | 0,17%  |
| <b>EMP.BR</b> | 853        | 0,02%  |

» [Ver todas as categorias](#)



## CIDADES

Total **13.782** 0,27%

» [Ver evolução](#)

| CATEGORIAS          | QUANTIDADE | %     |
|---------------------|------------|-------|
| <b>9GUACU.BR</b>    | 8          | 0,00% |
| <b>ABC.BR</b>       | 299        | 0,01% |
| <b>AJU.BR</b>       | 144        | 0,00% |
| <b>ANANI.BR</b>     | 5          | 0,00% |
| <b>APARECIDA.BR</b> | 51         | 0,00% |
| <b>BARUERI.BR</b>   | 57         | 0,00% |

» [Ver todas as categorias](#)



## UNIVERSIDADES

Total **4.676** 0,09%

» [Ver evolução](#)

| CATEGORIAS    | QUANTIDADE | %     |
|---------------|------------|-------|
| <b>BR</b>     | 1.207      | 0,02% |
| <b>EDU.BR</b> | 3.469      | 0,07% |



## PESSOAS FÍSICAS

Total **8.553** 0,17%

» [Ver evolução](#)

| CATEGORIAS     | QUANTIDADE | %     |
|----------------|------------|-------|
| <b>BLOG.BR</b> | 6.485      | 0,13% |
| <b>FLOG.BR</b> | 107        | 0,00% |
| <b>NOM.BR</b>  | 1.087      | 0,02% |
| <b>VLOG.BR</b> | 318        | 0,01% |
| <b>WIKI.BR</b> | 556        | 0,01% |



## PESSOAS JURÍDICAS

Total **111.499** 2,16%

» [Ver evolução](#)

| CATEGORIAS     | QUANTIDADE | %     |
|----------------|------------|-------|
| COM RESTRIÇÃO  |            |       |
| <b>AM.BR</b>   | 71         | 0,00% |
| <b>COOP.BR</b> | 1.391      | 0,03% |
| <b>FM.BR</b>   | 290        | 0,01% |
| <b>G12.BR</b>  | 548        | 0,01% |
| <b>GOV.BR</b>  | 1.618      | 0,03% |
| <b>MIL.BR</b>  | 40         | 0,00% |
| SEM RESTRIÇÃO  |            |       |
| <b>AGR.BR</b>  | 4.524      | 0,09% |
| <b>ESP.BR</b>  | 1.117      | 0,02% |
| <b>ETC.BR</b>  | 1.054      | 0,02% |
| <b>FAR.BR</b>  | 583        | 0,01% |
| <b>IMB.BR</b>  | 3.444      | 0,07% |
| <b>IND.BR</b>  | 22.353     | 0,43% |



# Domínios .BR por Categorias



## PROFISSIONAIS LIBERAIS

Total **91.708** 1,78%

» [Ver evolução](#)

| CATEGORIAS    | QUANTIDADE | %     |
|---------------|------------|-------|
| <b>ADM.BR</b> | 2.510      | 0,05% |
| <b>ADV.BR</b> | 45.899     | 0,89% |
| <b>ARQ.BR</b> | 5.680      | 0,11% |
| <b>ATO.BR</b> | 47         | 0,00% |
| <b>BIB.BR</b> | 32         | 0,00% |
| <b>BIO.BR</b> | 495        | 0,01% |

» [Ver todas as categorias](#)